



École Doctorale de Mathématiques, Informatique,
Science et Technologie de la Matière



Thesis

Submitted for the degree of

Ph.D. in Computer Science

Prepared within the research laboratory
MEDIATRON, Sup'Com

Presented by
Nouredine AMRAOUI

Behavior Profiling-based Approach for The Security of Smart Home Systems

Defended on **June 24th, 2021**

In front of the jury composed of

Mrs. Amel BORG	Professor at ISI	President
Mr. Sofiane OUNI	Associate Professor at INSAT	Member
Mrs. Hanen IDOUDI	Associate Professor at ENSI	Member
Mr. Abderrazek JEMAI	Professor at INSAT	Examiner
Mr. Belhassen ZOUARI	Professor at Sup'Com	Thesis Director

Academic Year: 2020/2021

*This dissertation is dedicated to all my family and loved ones for their
endless love, prayers, and encouragements . . .*

Acknowledgements

I am grateful to Almighty Allah for giving me the strength and ability to fulfill this goal.

This thesis would not have been possible without the patient instruction and constant support of my advisor Professor Belhassen Zouari, who is not only an excellent academic advisor but also a thoughtful professional mentor. His wit, insight, and feedback have immensely enriched this last stretch of my educational experience.

First of all, I would like to thank Doctor Riadh Ksantini and Doctor Amine Besroun for sharing their variant scientific and academic knowledge and contributing to the publication of my research papers.

Secondly, I would also like to thank all the jury's members for accepting to review and judge this thesis. The final version of this dissertation has benefited from their careful reading and valuable comments. I also thank them for accepting to attend the presentation of this work.

Besides, I also want to thank all my friends who support me during this step of my life, especially, Hamza, Adil, Fateh, and Oussama.

Finally, I would like to thank the Algerian Ministry of Higher Education and Scientific Research for granting me the scholarship to accomplish this work. I would also like to thank Mediatron Research Laboratory at Sup'Com for providing the appropriate research atmosphere and partially funding my research papers.

Table of contents

List of figures	xi
List of tables	xiii
Nomenclature	xv
Introduction	1
 Part I : Background, Threat Model, and Positioning to The State of The Art	 13
 1 Securing The Operation of Smart Home Systems: Threat Model and Existing Conventional Approaches	 15
1.1 Introduction	15
1.2 Internet of Things-based Smart Home Systems	16
1.2.1 Internet of Things ecosystem	16
1.2.2 Smart Home Systems	18
1.3 Threat model of SHS operation	23
1.3.1 Security threats related to user-driven operation	24
1.3.2 Security threats related to appified automated operation	25
1.4 Existing conventional approaches for securing user-driven operation	25

1.4.1	Signature-based intrusion detection systems	26
1.4.2	Conventional user authentication	27
1.5	Existing conventional approaches for securing appified automated operation	27
1.5.1	Program analysis-based approaches	27
1.6	Conclusion	29
2	Behavioral Anomaly Detection-based Security: Fundamental Concepts and Existing Approaches	31
2.1	Introduction	31
2.2	Anomaly detection	32
2.2.1	Machine learning-based anomaly detection	32
2.2.2	Taxonomy of anomaly detention techniques	34
2.3	Behavioral anomaly detection-based security	38
2.3.1	Basic operational scheme	38
2.3.2	Data collection	39
2.3.3	Anomaly detection models training	40
2.4	Existing behavioral anomaly detection-based security approaches	41
2.4.1	Existing approaches for securing non IoT-based systems	41
2.4.2	Existing approaches for securing user-driven operation of IoT-based SHSs	43
2.4.3	Existing approaches for securing appified automated operation of IoT-based SHSs	46
2.5	Conclusion	46

Part II : Contributing to The Security and Safety of Smart Home Systems	47
3 Duenna: A Behavioral Anomaly Detection-based Conceptual Security Framework	49
3.1 Introduction	49
3.2 Operational architecture	50
3.2.1 Initial operation	52
3.2.1.1 Target logs collection	52
3.2.1.2 Target regular behavior enrollment	54
3.2.2 Interactive operation	57
3.2.2.1 Basic interactive operation	57
3.2.2.2 Enhanced interactive operation	60
3.3 Conclusion	63
4 Use Case 1: Continuous Authentication of Smart Home System Users	65
4.1 Introduction	65
4.1.1 System model and security goals	65
4.1.2 Duenna instantiation	66
4.2 Initial operation	68
4.2.1 Raw logs collection	68
4.2.2 User regular behavior enrollment	70
4.2.2.1 Logs preparation	71
4.2.2.2 Probabilistic models construction	72
4.2.2.3 Behavioral scores extraction	74
4.2.2.4 Training anomaly detection models	75
4.3 Interactive operation	76

4.3.1	ICA: Implicit and Continuous Authentication	77
4.3.1.1	ICA-Anomaly Analyzer (ICA-AA)	77
4.3.1.2	ICA-Action Manager (ICA-AM)	77
4.3.2	MTR-ICA: Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication	81
4.3.2.1	MTR-ICA-Action Manager (MTR-ICA-AM)	81
4.3.2.2	MTR-ICA-Anomaly Analyzer (MTR-ICA-AA)	82
4.3.3	SA-MTR-ICA: Self-Adaptive, Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication	84
4.4	Empirical performance evaluation	85
4.4.1	Evaluation datasets	86
4.4.2	Selection of anomaly detection model and evaluation methodology	88
4.4.3	Tuning of inactivity threshold and day segmentation	92
4.4.4	Authentication performance evaluation	93
4.4.5	Comparison with similar works	96
4.4.6	Lockout threshold tuning	97
4.4.7	Evaluation of computational complexity	100
4.5	Conclusion	103

5 Use Case 2: Maintaining a Safe Appified Automated Smart Home System

Operation		105
5.1	Introduction	105
5.1.1	System model and security goals	105
5.1.2	Duenna instantiation	106
5.2	Initial operation	106
5.2.1	SmartApps raw logs collection	107
5.2.2	SmartApps regular behavior enrollment	108

5.2.2.1	SmartApps operation log segmentation	108
5.2.2.2	Probabilistic models construction	109
5.2.2.3	Behavioral scores extraction	109
5.2.2.4	Training anomaly detection models	110
5.3	Interactive operation	111
5.3.1	Anomaly Analyzer	111
5.3.2	Continuous Safety (C-Safety)	112
5.3.3	Continuous, Trust-based, and Maintainable Safety (CTM-Safety)	114
5.4	Empirical performance evaluation	116
5.4.1	Evaluation dataset	117
5.4.2	Selection of anomaly detection model and evaluation methodology	118
5.4.3	Day segmentation tuning	118
5.4.4	Anomaly detection performance	119
5.5	Conclusion	122
Conclusion		123
References		127

List of figures

1	A scenario of a legitimate/malicious user-driven operated smart window . .	6
2	A scenario of a legitimate/malicious automated smart window operation . .	7
3	Dissertation roadmap	9
1.1	Relationship between physical and digital worlds in an IoT environment . .	17
1.2	Architecture of a user-driven operated SHS	21
1.3	Architecture of an appified automated SHS	23
2.1	A trivial example of an SVM separating hyperplane	35
2.2	A trivial example of an OCSVM separating hyperplane	36
2.3	Basic operational scheme of a BAD-based security approach	39
3.1	Operational architecture of Duenna	51
3.2	Workflow of target regular behavior enrollment	54
3.3	Workflow of Basic Anomaly Analyzer (BAA)	58
3.4	Workflow of Basic Action Manager (BAM)	59
3.5	Workflow of target-initiated command execution and models update	60
4.1	Instantiation of Duenna on a user-driven SHS architecture	67
4.2	Workflow of user regular behavior enrollment	70
4.3	Workflow of ICA-Anomaly Analyzer (ICA-AA)	78
4.4	Workflow of ICA-Action Manager (ICA-AM)	78

4.5	Workflow of user login	79
4.6	Workflow of user-initiated command execution and models update	80
4.7	Workflow of MTR-ICA-Action Manager (MTR-ICA-AM)	81
4.8	Workflow of MTR-ICA-Anomaly Analyzer (MTR-ICA-AA)	83
4.9	Workflow of SA-MTR-ICA-Action Manager (SA-MTR-ICA-AM)	84
4.10	Training and testing process for our adapted dataset	91
4.11	Day segmentation and inactivity threshold tuning using our dataset	94
4.12	Day segmentation and inactivity threshold tuning using DS2OS dataset	95
4.13	Comparison of use case 1 with similar works by anomaly detection accuracy	98
4.14	Comparison of use case 1 with similar works by false rejection rate	98
4.15	Lockout threshold tuning using our dataset	101
4.16	Lockout threshold tuning using DS2OS dataset	102
4.17	Log of training time per model for different sizes of data	102
4.18	Log of training per model for different length of user sessions	103
5.1	Instantiation of Duenna on an appified automated SHS architecture	107
5.2	Workflow of SmartApps regular behavior enrollment	108
5.3	Workflow of Anomaly Analyzer	112
5.4	Workflow of Continuous-Safety-Action Manager (C-Safety-AM)	113
5.5	Workflow of CTM-Safety-Action Manager (CTM-Safety-AM)	115
5.6	Obtained ROC curves using different day segmentation candidates	120

List of tables

1	Publications resulting from thesis	11
1.1	IoT-based systems and examples of enabling devices	19
1.2	Recent real-world reported malicious SHS devices operation	24
1.3	Summary of existing program analysis-based approaches	28
2.1	Summary of existing approaches for securing non-IoT-based systems	42
4.1	Structure of user/SHS raw logs	70
4.2	Structure of user/SHS prepared logs	71
4.3	User Behavioral Model (User-BM)	73
4.4	SHS Behavioral Model (SHS-BM)	73
4.5	Types of AD models	83
4.6	Description of our adapted dataset for evaluating use case 1	87
4.7	Frequency distribution of considered attacks	88
4.8	Day segmentation candidates	93
4.9	Authentication performance results using our adapted dataset (N/A means that metric could not be calculated for primary vs itself)	96
4.10	Authentication performance results using DS2OS dataset	96
4.11	Comparison of use case 1 with similar works	97
4.12	Obtained AGNA and ANIA results (∞ means a big value)	100

5.1	SmartApps probabilistic models	109
5.2	Description of evaluation dataset of use case 2	118
5.3	Obtained results of TPR, FAR, and FRR for subject 1 vs subject 2	121
5.4	Obtained results of TPR, FAR, and FRR for subject 2 vs subject 1	122

Nomenclature

Acronyms / Abbreviations

AA Anomaly Analyzer

AD Anomaly Detection

AM Action Manager

BAD Behavioral Anomaly Detection

C-Safety-AM Continuous-Safety-Action Manager

CTM-Safety-AM Continuous, Trust-based, and Maintainable-Safety-Action Manager

DMA-AD Degraded Mode Activity Anomaly Detection

ICA-AA Implicit and Continuous Authentication-Anomaly Analyzer

ICA-AM Implicit and Continuous Authentication-Action Manager

ICA Implicit and Continuous Authentication

IDS Intrusion Detection System

iOCSVM incremental One-Class Support Vector Machine

IoT Internet of Things

ML Machine Learning

MTR-ICA-AA Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication-
Action Analyzer

MTR-ICA-AM Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication-
Action Manager

MTR-ICA Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication

NMA-AD Nominal Mode Activity Anomaly Detection

OCSVM One Class Support Vector Machine

S-AD Starting Anomaly Detection

SA-MTR-ICA Self-Adaptive, Maintainable, Trust-based, Risk-aware, Implicit, and Contin-
uous Authentication

SHS Smart Home System

Introduction

Recent developments in information and communication technologies related to computer networks, embedded systems, and artificial intelligence have led to the emergence of a new technology called the Internet of Things (IoT). The overall aim of IoT is to interconnect the physical world with the digital one. This vision is achieved by the deployment of sensor devices that can measure the physical world. The digital measurements are then translated into processable data which is translated into commands executed by actuator devices [60]. According to Gartner, there will be over 25 billion connected IoT devices by the end of 2021 [94]. The increasing attention gained by IoT results in the proliferation of many applications. The latest 2020 analysis shows that most IoT projects are happening in a plethora of verticals such as manufacturing/industrial, transportation/mobility, energy, cities, healthcare, and agriculture [96].

Smart Home Systems (SHSs), also called Home Automation Systems, Connected Homes, or Domotics, represent a class of the most prevailing IoT applications [129]. The vision of the smart home is an old idea but no real-world systems have existed before the emergence of IoT. Due to the growing interest in smart home environments, the number of systems designed to support them has risen considerably [60]. The goal of an SHS is to provide interoperability and communication among IoT devices commercialized by different vendors, manage data flows, support application development, etc. [80]. An SHS is thus able to provide several intelligent services to consumers such as energy-saving, physical security and safety, and elderly people assistance.

To take advantage of different intelligent services, a consumer can operate an SHS in several ways. On one hand, SHSs may provide consumers with companion applications and web portals that can be run on end-user devices such as tablets, smartphones, etc. so consumers can operate their devices on their own either from the inside when connected to the Local Area Network (LAN), or from any outside location via the Internet. On the other hand, many SHSs also allow consumers to install and delegate authorization to third-party applications (called SmartApps) to autonomously operate devices without user intervention. SmartApps use simple trigger-action rules where the operation action of a given device is only performed when the triggering event has occurred [30]. For instance, a ‘Welcome Home’ SmartApp sets the mode to home when the light in the living room is turned on.

Problem statement

While bringing a significant convenience to consumers, unauthorized and unexpected operation of SHS devices brings new security and safety concerns that have a great impact compared to the malicious operation of a classical computer system. In other words, the maximal damage a computer attacker could inflict is limited to data loss. With the IoT-based SHSs, attackers can have physical effects in the world, such as opening doors, causing fake fire alarms, and disrupting electricity supply [51].

On one hand, when the SHS operation is driven by the user, several vectors could allow an attacker to operate SHS devices. First of all, recent results reveal that IoT devices have been suffering from the open-port problem, weak encryption, and lack of authentication/authorization to their web interfaces [86]. Besides, end-user control devices can easily be compromised if they are not secured properly [54]. Moreover, the account which an SHS owner uses to access companion applications could be compromised in several ways such as reverse engineering, password guessing, malware infection, etc. [78].

On the other hand, while SHS automation is supposed to be operating regularly, intentional/unintentional issues would make SmartApps deviate from their regular behavior, thus putting the SHS owner's security at risk and create unsafe or damaging conditions. First of all, poor configuration by novice users (e.g., parents and kids) at the installing stage of SmartApps can transition the SHS to unsafe physical states due to the conflicting logic of common SmartApps [92]. For instance, a SmartApps misconfigured to be triggered by the action of another one will unexpectedly be triggered once the second SmartApp is in action [30]. Secondly, the trigger-action model of appified SHSs provides flexibility for the attacker to embed their malicious logic into the SmartApps using available triggering events (e.g., home mode changing) [37]. Lastly, faulty events caused by sensor defects or physical interference as well as fake events maliciously injected by attackers may incorrectly trigger SmartApps and cause consumer confusion.

Current literature

The plethora of security threat vectors that can be exploited requires robust security schemes to prevent the malicious operation of SHS devices.

In the context of securing user-driven operation, two types of conventional security mechanisms have gained a particular attention viz., signature-based Intrusion Detection Systems (IDSs) [81, 137] and user authentication [76, 110]. However, these approaches suffer from several limitations. First of all, although existing signature-based IDSs assume that legitimate and anomalous traffic signatures are notably different, operation commands sent via a compromised end-user device will have the same IP addresses and packets' signatures [132]. Besides, although recent works have been combining authentication factors to identify SHS users (e.g., passwords, smart-cards, fingerprint, etc.) [127], these factors remain vulnerable to social engineering attacks and the overall usability of the SHS decreases since users are forced to carry around specific authentication tokens [110].

In the context of securing automated SHS devices operation, recent works have been proposing the enforcement of policies that describe the security and safety properties that refer to regular operation of the SHS [30, 31, 120, 130]. In particular, the adherence of the SHS automated operation is continuously checked with the properties defined by the policy, and the operation commands causing the policy violations are blocked. Unfortunately, the pre-definition of the policy is the major problem facing this type of system. In other words, general-purpose policies (e.g., defined by security experts) are not personalized and may not suit all SHSs automation configurations. Moreover, SHS users often do not know exactly what to expect from the system when acquiring it, thus if the policy definition is left for the user him/herself, security and safety properties may not be well defined.

Motivation and research question

Recently, the limitations of conventional security mechanisms have been warranting researchers to integrate the behavioral analysis of both SHS and its users to devise new schemes that (i) are personalized for each SHS configuration and support self-learning, and (ii) allow more intelligent authentication/authorization decisions.

Tracing then assessing the behavioral patterns of users and entities to secure cyber systems is better known as Behavioral Anomaly Detection (BAD) [99]. The basic idea behind such an approach is to first build the behavioral models of profiled targets i.e., systems, users, applications, etc. and, then, attempt to identify security threats and behaviors that are not known and do not match the predetermined patterns [88, 119]. BAD-based security has been recently considered as the alternative to respond to the limitations of conventional security approaches. Although some works have been leveraging BAD-based approach to secure the operation of SHSs [56, 104, 132, 138], such an approach is still not well addressed in this context.

In light of the discussion of the state-of-the-art approaches addressing the security problems of SHSs, this thesis is motivated by the research question: *How to ensure a secure and safe operation of SHSs?*

This research question can be broken down into the following questions:

- How to address the lack of relevant techniques that leverage the BAD-based approach to secure the operation of SHS?
- How to continuously authenticate SHS users?
- How to maintain a safe applied automated operation?

To respond to these research questions, we consider the following hypotheses. On one hand, SHS users usually operate their home IoT devices in typical and distinctive patterns that can be described by many behavioral features such as rate (i.e., number of devices operated in a time window), sequence (i.e., the order of operation). Also, operating SHS devices is generally related to the daily activities of inhabitants. For instance, a user usually turns off the heater in the mornings when nobody is home (i.e., no physical activity). Therefore, correlating the cyber-operation with the physical-activity is a strong behavioral pattern.

In Fig. 1.1, we present a motivating example to show how the integration of the user and SHS behavior can efficiently help in preventing the unauthorized/malicious operation of devices. In this example, we consider the scenario (which has been reported in real-world [29]) of a smart window open command performed by a legitimate and an adversary. The operation command issued by the legitimate user is considered as authorized and executed since he/she frequently opens the window in the morning of weekends when the home is occupied, and after closing the door. An adversary, however, did not follow such a legitimate behavioral pattern and attempts to open the window to break into the house in a different situation i.e., when the home is not occupied and before opening the door, etc. Thus, the operation command is considered malicious and rejected.

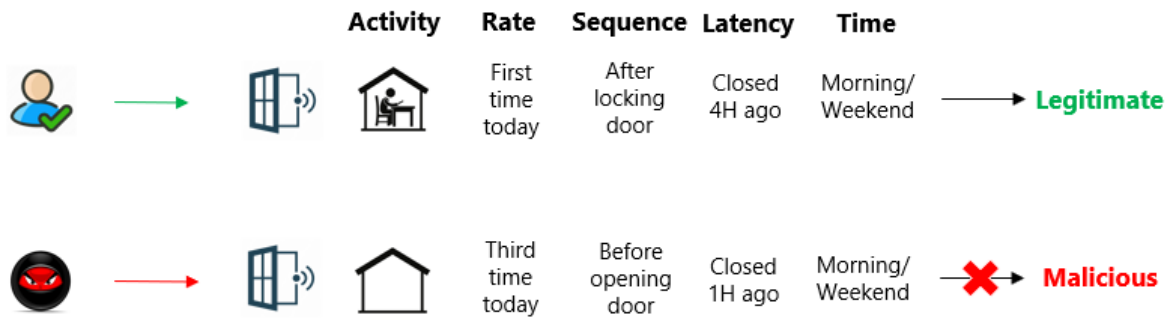


Fig. 1 A scenario of a legitimate/malicious user-driven operated smart window

In contrast to afore discussed conventional techniques, authenticating users based on their behaviors has many advantages [126]. Indeed, user behavioral patterns could be discreetly collected and monitored since they depend on mental characteristics and do not require user interaction. Consequently, neither authorized nor unauthorized users are aware of being forensically monitored, thus they do not interfere with their experience. Additionally, user behavior-based verification could be continuously performed throughout the entire user's operation session.

On the other hand, given that the SmartApps leverage the trigger-action automation model for operation, they follow a frequent pattern when they are triggered by the occurred events to operate the SHS devices. In particular, the occurred events are the result of the daily living activities of the home inhabitant (e.g., door opening). And because the inhabitant tends to follow frequent patterns when living and performing variant activities inside the home (e.g., every evening he/she back home from work), triggering events also occur in the same particular patterns. On the other hand, since SHS devices are operated by these events (e.g., door opening triggers the light to turn on), SmartApps also operate the devices in such a recurrent pattern. This set of regular patterns can be described by several behavioral features such as the occurrence probability of an event, the probability of the SmartApps to operate a given device while being triggered by the occurred event, etc. Thus, any deviation of the SHS automated operation from such regular behavior could be detected based on the analysis of the SmartApps behavior.

In Fig. 1.2, we present a motivating example to show how the integration of the SmartApps behavior can efficiently help in preventing the unexpected/malicious automated operation of devices. In this example, we consider the scenario of automated operation of a smart window. In the first case, the opening of the window is considered legitimate and authorized to be executed since during the evening of weekday, the reaching of temperature to 30 is a probable event and the operation of windows triggered by this event is also probable, etc. However, in the second case, the SmartApp did not follow its legitimate behavioral pattern and attempts to open the window in a different situation i.e., door is unlocked when the home is not occupied and before opening the door, etc. Thus, the automated operation command is considered malicious and rejected.

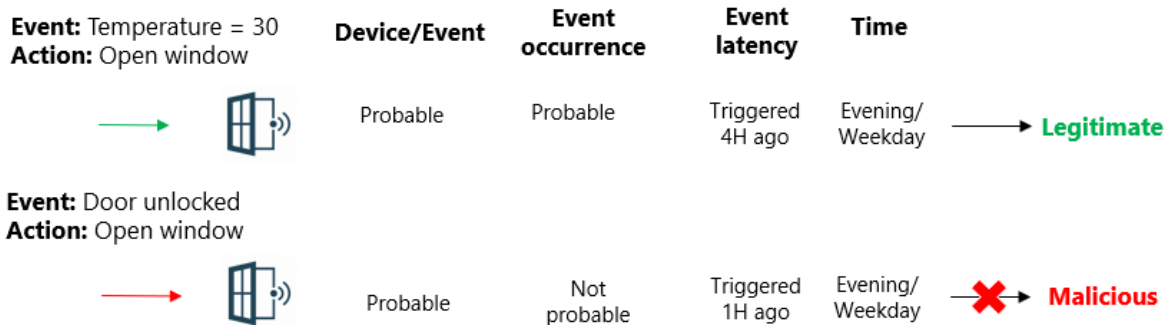


Fig. 2 A scenario of a legitimate/malicious automated smart window operation

Compared to predefined policy-based systems, preventing unexpected automated operations based on SmartApps behavior analysis is customized for each SHS configuration. Besides, behavioral patterns can be automatically collected and assessed, thus require lesser user intervention.

Results and contributions

Considering its advantages compared to conventional techniques, in this thesis, we leverage the BAD-based approach to respond to the raised research questions. The contribution of this thesis is three-fold as explained in the following.

Duenna: A Behavioral Anomaly Detection-based Conceptual Security Framework

We introduce Duenna - a BAD-based conceptual and methodological framework to guide cybersecurity practitioners and developers in securing the operation of SHSs. Duenna is an abstraction framework that provides a detailed operational architecture to illustrate different operation phases, modules, workflows, behavioral features, data structure, etc. that would be devised and implemented to achieve such an aim. Duenna could be relevant for multiple use cases whether the profiled target is an SHS user or entity such as an IoT device or an application.

This thesis claims that Duenna is an exemplar for leveraging the BAD-based approach and methodology towards the design of secure and safe SHSs. To validate this claim and show the concrete operation and the feasibility of Duenna, we apply it to two use cases; hence, we make two other research contributions as follows.

Continuous Authentication of Smart Home System Users

In the first use case, user-driven SHS operation is considered. In particular, Duenna is instantiated to make sure that only authorized users are allowed to operate SHS devices. The profiled and monitored targets in this use case are the SHS users which are continuously authenticated to prevent unauthorized user-issued operation commands. This use case is validated through a detailed empirical evaluation where both SHS security and user experience have been ensured.

Maintaining a Safe Appified Automated Smart Home System Operation

In the second use case, appified automated SHS operation is considered. In particular, Duenna is instantiated to maintain a safe SHS automation. The profiled and monitored targets in this

use case are the automation apps (called SmartApps) which are continuously assessed to prevent unexpected SmartApps-issued operation commands. This use case is also validated through a detailed empirical evaluation where both SHS safety and user experience have been ensured.

Dissertation structure

Fig. 1.3 provides a graphical roadmap of this dissertation, describes how its structure maps to the contributions, and how these appeal to the research questions posed in Section 1.3. As depicted in the figure, this dissertation is broken down into two main parts. The first part provides the necessary background, presents considered security threats, and situates the dissertation within the state-of-the-art. Whereas, the second part is devoted to present the main contributions in detail. In the following, we briefly present the content of the two parts.

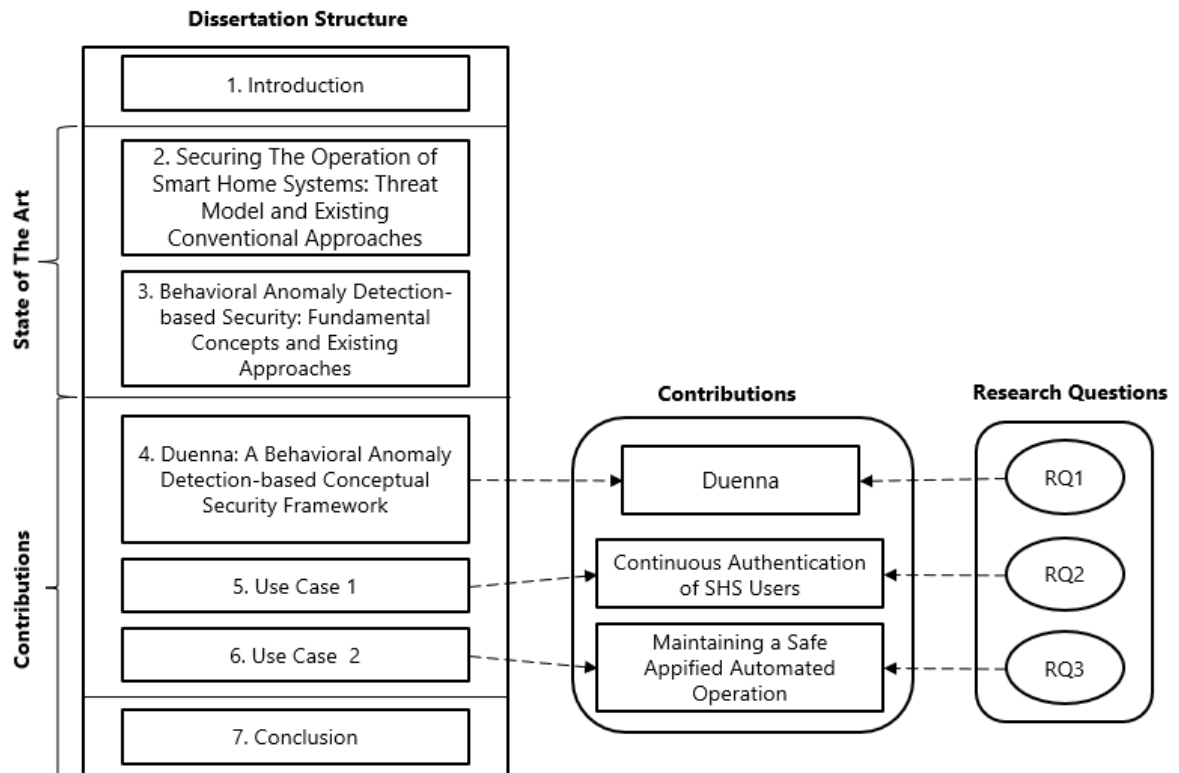


Fig. 3 Dissertation roadmap

Part I: Background, Threat Model, and Positioning to The State of The Art

Chapter 2 represents the first part of our review of the state-of-the-art approaches for securing the operation of SHSs. We first provide a theoretical background to understand IoT-based SHSs. Then, security threats that may lead to unauthorized/unexpected user-driven/automated operation of an SHS are presented. After that, conventional security techniques that do not leverage the BAD-based security approach are discussed. Finally, based on the review, some concluding remarks are drawn to motivate the conduction of the second review addressed in Chapter 3.

Chapter 3 represents the second part of our review with a focus on the state-of-the-art BAD-based security approaches. The necessary background to understand such an approach is first provided. Then, we review existing approaches for securing both IoT-based SHSs as well as non-IoT-based systems. Finally, based on the review, some concluding remarks are drawn to motivate the proposition of our framework addresses in Chapter 4.

Part II: Contributing to The Security and Safety of Smart Home Systems

Chapter 4 introduces our proposed framework called Duenna which leverages the BAD-based security approach. We first discuss the motivation behind the proposition of Duenna which is drawn from the conducted state-of-the-art review. Then, we provide a detailed description of Duenna's operational architecture that would be instantiated and implemented to secure the operation of an SHS.

Chapter 5 presents the first use case to show the feasibility of our proposed framework. In particular, Duenna is instantiated to continuously authenticate SHS users and prevent unauthorized devices' operation. This use case is then validated through a detailed empirical performance evaluation.

Table 1 Publications resulting from thesis

Publication	Related chapter
Amraoui N, Besrour A, Ksantini R, Zouari B (2019, March). “Implicit and continuous authentication of smart home users”. In: 2019 33rd International Conference on Advanced Information Networking and Applications (AINA). Springer, pp. 1228-1239. (B-ranked). https://doi.org/10.1007/978-3-030-15032-7_103	5
Amraoui N, Besrour A, Ksantini R, Zouari B (2020, October). “Securing Smart Homes using a Behavior Analysis based Authentication Approach”. In: 2020 8th International Conference on Communications and Networking (ComNet). IEEE. (Unranked). https://doi.org/10.1109/ComNet47917.2020.9306081	5
Amraoui N, Zouari B (2020, September). “Behavior-based Anomaly Detection for Securing Smart Home Systems Automation”. In: 2020 28th International Conference on Software, Telecommunications and Computer Networks (SoftCom). IEEE. (B-ranked). http://doi.org/10.23919/SoftCOM50211.2020.9238199	6
Amraoui N, Zouari B (2020, November). “An ML Behavior-based Security Control for Smart Home Systems”. In 2020 15th International Conference on Risks and Security of Internet and Systems. Springer. (C-ranked). https://doi.org/10.1007/978-3-030-68887-5_7	6

Chapter 6 presents the second use case to show the feasibility of our proposed framework. In particular, Duenna is instantiated to maintain a safe appified SHS automation by preventing unexpected operation caused by misbehaving SmartApps. This use case is then validated through a detailed empirical performance evaluation.

Finally, Chapter 7 provides a general conclusion of this thesis by summarizing our flagship contributions and outlines future research directions.

Publications arising from this work

Table 1.1 describes the elements of this dissertation which have been published in peer-reviewed international conference proceedings.

Part I :

Background, Threat Model, and Positioning to The State of The Art

Chapter 1

Securing The Operation of Smart Home Systems: Threat Model and Existing Conventional Approaches

1.1 Introduction

As the adoption of any new information and communication technology is usually hindered by the security challenges it brings [13], the success of IoT-based SHSs is no doubt related to the confidence degree of consumers towards how their systems are sufficiently secure and safe.

Even though the market of SHSs has been prospering in recent years, their security condition is not cheerful [129]. Current designs of SHSs suffer from many security problems that may manifest at all layers of their software/hardware stack as well as at end-users devices [51]. In particular, design and implementation flaws that could be exploited by adversaries may exist on devices firmware, connectivity protocols, as well as on system-level. Open ports and the lack of robust authentication are some examples of these flaws.

Among these latter, security flaws that lead to the malicious operation of SHS devices are considered as the most serious security and safety concern since such unwanted operation has a great impact compared to the malicious operation of a classical computer system. In other words, the maximal damage an attacker could inflict is limited to data loss. With the IoT-based SHSs, attackers can have physical effects in the world, such as opening doors [50], causing fake fire alarms [50], and disrupting electricity supply [116], etc. [51]. As a consequence, security is still one of the major obstacles for widespread adoption of IoT-based SHSs [56]; thus, robust security mechanisms should be taken in place.

1.2 Internet of Things-based Smart Home Systems

1.2.1 Internet of Things ecosystem

The Internet of Things (IoT) is a technical realization of ubiquitous computing as imagined in 1991 by Mark Weiser [128], where technology will gradually become effective in the users' environment and integrated inside daily life objects. The first machine to implement this idea was a soda dispenser installed at Carnegie Mellon University (CMU) within the computer science department in 1982 [69]. By querying it remotely via its IP address, it was possible to tell if there were any cans left in the machine.

Originally, the term Internet of Things was first used in 1999 by Kevin Ashton to describe objects equipped with Radio Frequency Identification (RFID) chips that are readable by other machines [15]. Since then, a large number of objects have acquired addresses to communicate and the term IoT has been adopted without an exact definition. It was indeed more of an idea than a physical reality.

Later, the IoT concept has evolved and has spread to an approach consisting of connecting a very large number of everyday objects to the Internet. With IoT, technology is no longer represented by a single object, the personal computer, but instead comes in the form of

specialized and easy-to-use devices. These devices are capable of communicating through several types of wired and wireless networks [17, 58].

The overall aim of IoT is to interconnect the physical world composed of our daily life objects (e.g., doors, refrigerators, etc.) with the digital world provided by information and communication technologies [20]. In particular, as depicted in Fig. 2.1, the physical world is measured by sensors and translated into processable data that has to be translated into commands to be executed by actuators [60]. For instance, a set of sensors monitoring the state of road traffic in real-time would control the alarm clock in a home to adapt the alarm time according to the traffic.

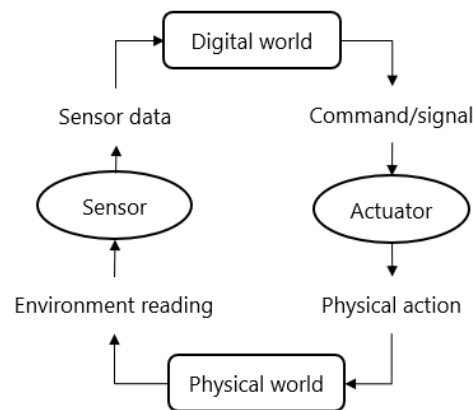


Fig. 1.1 Relationship between physical and digital worlds in an IoT environment

Although the original concept of IoT is certainly interesting, today we are far from an automated and unified ecosystem that has been initially imagined. Currently, the majority of IoT developments consist of adding communication capabilities to an object that initially lacks them. This object can then send data to a server or receive commands to interact with the environment in which it is installed. We are therefore seeing a multitude of small environments associated with LANs of objects rather than a true Internet of Things. The only difference with traditional LANs is that objects store their data on servers in the cloud and therefore on the Internet.

The economic interest associated with the possibility of acquiring information from data collected by hundreds of thousands of objects is pushing manufacturers to develop many connected objects. According to Gartner, there will be over 25 billion connected IoT devices by the end of 2021 [94]. All environments such as homes, enterprises, industries, cities, and public spaces, are indeed integrating these devices more and more massively to meet their specific needs [96].

As a consequence, rather than an Internet of Things, we consider the notion of the connected environment or Internet of Things-based system. This latter is defined as a computer system dedicated to a limited environment made up of IoT devices that have the possibility of exchanging information with other devices in the environment directly or via the Internet. The characteristics of these devices, their roles, as well as their means of communication depend on the context in which they are used. Table 2.1 summarizes the most common IoT-based systems and the types of devices enabling each one.

1.2.2 Smart Home Systems

Smart Home Systems (SHSs), also called Home Automation Systems, Connected Homes, or Domotics, represent a class of the most prevailing IoT-based systems [129]. The vision of the smart home is an old idea but no real-world implemented systems have existed before the emergence of IoT. Homes have been considered as one of the main environments for the widespread of IoT devices compared to other ones such as enterprises and cities [58].

Indeed, consumers are transforming their homes into smart spaces with Internet-connected sensors, lights, and appliances. As predicted by Gartner, there will be more than 500 IoT devices deployed in a typical household by 2022 [122]. Due to the growing interest in smart home environments, the number of systems designed to support them has risen considerably [60]. The goal of an SHS is to provide interoperability and communication among IoT devices

Table 1.1 IoT-based systems and examples of enabling devices

Domain	System	Application	Examples of devices
Industrial	Logistic and product management	Products management (integrity, deterioration)	RFID tags/readers, beacons
		Intelligent shopping systems	
	Agriculture and breeding	Agriculture and irrigation monitoring	Sensors, sprinkler nodes
		Animals monitoring	Sensors, integrated chips
Smart city	Smart mobility	Traffic management and V2V communication	Mobile sensors
		Smart parking	Mobile sensors, GPS, beacons
	Smart grid	Electricity efficiency	Smart meters
	Home/building automation	Optimizing energy consumption	Thermostats, smart meters
		Home control	Security cameras, thermostats, doorbells
Healthcare	Healthcare	Patient remote monitoring	Wearables, pacemaker

commercialized by different vendors, manage data flows, support application development, etc. [80].

One of the first prominent SHS prototypes is the Mozer's adaptive house, described in 1998 [89]. Other earlier SHS projects are MavHome [38], CASAS [121], and MIT PlaceLab [65]. Nowadays, big technology companies are leading SHSs market such as Samsung's SmartThings [107] and Apple's Home [11].

Provided services

The plethora of services and applications, which SHSs would provide, has been promoting the enthusiasm of consumers towards this type of system. Examples of provided services include:

- Energy-saving: when the heating is modulated with a smart thermostat and the lights go out when the rooms are empty, a consumer can save a lot of electricity, and the home is becoming more energy efficient.
- Physical security and safety: an SHS allows consumers to sleep soundly at all times or when they go on vacation. They can for instance program the lights to turn on by themselves and the roller shutters to go down after dark. Besides, alarms, surveillance cameras, and intrusion detectors are also programmable and remotely controllable.
- Elderly people assistance: an SHS helps to preserve the autonomy of elderly people by helping them to maintain their usual way of life. To do so, an SHS can be configured to automate the routines of elderly people such as waking up, preparing coffee, or sleeping.
- Surveillance: thanks to different sensors that detect flood, fire, gas leaks, etc. an SHS may intervene instantly to cut off the power supplies, raise the blinds, or call emergency numbers.

Operation mechanisms

To take advantage of different intelligent services, a consumer can configure or operate an SHS in several ways. Today's SHSs are either user-driven, programmable, or self-learned. We give more detail in what follows.

User-driven operation Intuitively, a user may prefer to have full control of his/her IoT SHS devices, in a way similar to his/her full control of conventional home appliances, without reliance on external parties [139]. Such preference is currently available in today's SHSs such as SmartThings [107]. In particular, such type of SHS provides consumers with companion applications and web portals that can be run on end-user devices such as tablets, smartphones, etc. so consumers can operate their devices on their own either from the inside when connected to the LAN, or from any outside location via the Internet.

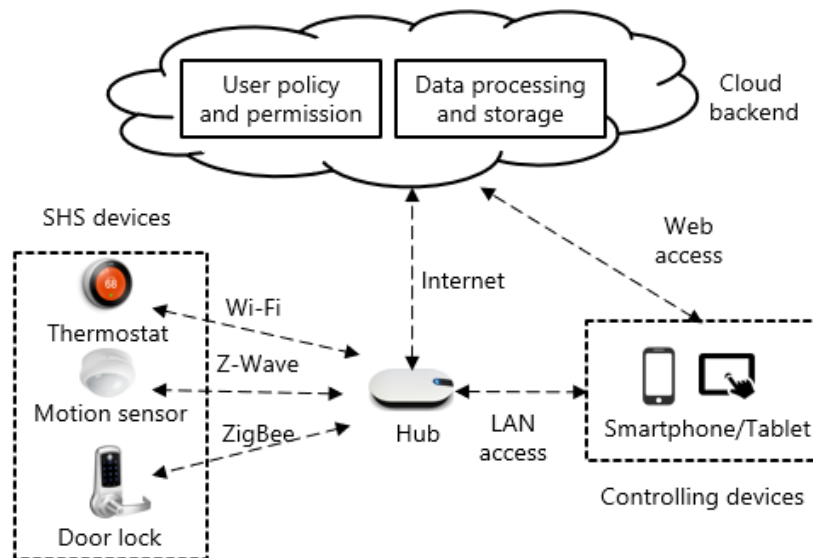


Fig. 1.2 Architecture of a user-driven operated SHS

A typical architecture of a user-driven SHS is depicted in Fig. 2.2 comprising four basic building blocks [113]. The first block is formed by the IoT devices that could be a sensor (e.g., motion detector) or an actuator (e.g., door-lock). Since there is no generic interoperability standard, SHS devices are connected via a smart hub that works as a coordinator among

them and provides a common access point for all the entities in the SHS. End-user devices (smartphones, tablets, etc.) can be either connected to the hub through the LAN or to the cloud backend through the Internet for local or remote operation, respectively. Moreover, the hub is also connected to the cloud to offload the big processing and data storage, in addition to managing security policy and end-user permissions.

Appified automated operation Recently, multiple cloud-backed SHSs are now allowing third-party developers to publish their automation apps (called SmartApps in Samsung SmartThings [107]). An SHS consumer can install and authorize these apps to autonomously monitor and operate his/her home devices. SmartApps use simple trigger-action rules where the operation action of a given device is only performed when the triggering event has occurred [30]. For instance, a ‘Welcome Home’ SmartApp sets the mode to home when the light in the living room is turned on.

In this thesis, we consider the SmartThings SHS since it has the largest number of supported SmartApps among all the SHSs. As depicted in Fig. 2.3, SmartThings uses a cloud backend to abstracts physical SHS devices into device handler instances. These software wrappers handle the real underlying communication between the cloud and the physical devices. SmartApps can subscribe to the events fired by a set of instances of device handlers and issue commands to operate the device’s handler. SmartThings also provide a smartphone companion app for users so they can install SmartApps published in the store, and configure and authorize the SmartApps that support the capabilities provided by their devices.

The permission model is the security architecture that governs the access of a SmartApp to the commands and attributes provided by the devices’ handlers. Commands represent ways in which a device can be operated (e.g., turn on/turn off). Attributes represent the state information of a device (e.g., on/off) [50]. When a user installs a SmartApp, an enumeration process is triggered that scans all the physical devices currently paired with the user’s hub, and that supports the commands and attributes claimed by the SmartApp. Once the user

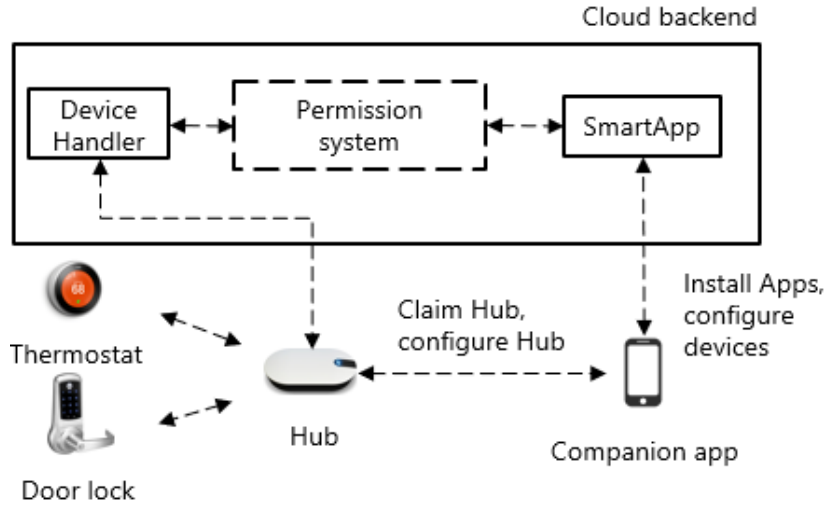


Fig. 1.3 Architecture of an appified automated SHS

chooses one of the suggested devices, the SmartApp is authorized to operate the selected device.

Self-learning automated operation SHSs can also be configured to be semi or fully autonomous without any consumer intervention. To do so, this type of SHS first records consumer usage information. Then, it applies artificial intelligence techniques to the accumulated data to extract new knowledge, anticipate consumers' needs, and automate their daily tasks [22, 59]. Similar to user-driven and appified automated ones, self-learning SHSs are also hub-based with a cloud-backend for artificial intelligence processing. An example of existing self-learning SHSs that only actuate under consumer's permission is [64]; whereas [124] is a fully autonomous SHS.

1.3 Threat model of SHS operation

Although an SHS could be operated through three types of mechanisms namely, user-driven, appified automated, or self-learning automated operation, in this thesis, we mainly focus on

the first two types of mechanisms. In the following, we present different security threats that could be exploited to maliciously operate an SHS for each type of operation.

1.3.1 Security threats related to user-driven operation

When an SHS is operated by users, several security threat vectors could lead to a malicious operation. These threats may manifest with different types and at different levels of an SHS architecture such as:

- Direct remote operation: recent results have indicated that 53% in a total of 1,541 devices firmware have publicly open ports [129].
- End-user device compromise: end-user devices can easily be compromised if they are not secured properly [54]. Besides, they could be manipulated by informed adversaries such as friends, coworkers, etc. Moreover, mobile end-user control devices such as smartphones could be stolen or dropped and found by adversaries.
- User account compromise: the account which an SHS owner uses to access control applications could be compromised in several ways such as reverse engineering, password guessing, malware infection, etc.

Table 2.2 presents some real-world malicious SHS devices operation attacks that have been recently reported.

Table 1.2 Recent real-world reported malicious SHS devices operation

Device	Example
Smart door	The attacker opens a smart door to break into the house [85]
Smart heater	An attacker operates the heater to overheat the bedroom [68]
Smart stove	An attacker operates the stove to cause fire [67].
Smart window	An attacker opens the smart window to break into the house [29]
Philips Hue smart light	An attacker uses a drone to turn off smart light to cause a blackout [40]
Smart toilet	An attacker remotely operates a smart toilet to cause water overflow [27]

1.3.2 Security threats related to appified automated operation

When an SHS is automatically operated via applications (i.e., SmartApps), anomalies whether due to attacks or devices' malfunctions, may cause SmartApps to deviate from their expected behavior and thus lead to severe safety consequences [53]. Such an unexpected operation could be resultant from several threat vectors such as:

- **Permission misuse:** once a user permits an application to operate a particular device, it can use that permission whenever it executes thereafter if no security control is in place [101].
- **Embedded malicious logic:** trigger-action model provides flexibility for attackers to embed their malicious logic into the SmartApps using available triggering events (e.g., home mode changing) [37]. The activation of malicious logic makes the SmartApp deviating from its past regular behavior since it starts to perform unexpected automation actions.
- **Faulty events:** incorrect reported values caused by sensor defects or physical interference, such as mysterious door-knocking events and motion events [63], may incorrectly trigger SmartApps and cause user confusion.
- **Fake commands:** an attacker may inject fake commands towards IoT devices. For example, Sonos smart speaker [71] and WeMo Smart switch [93] accept commands from the LAN without authenticating their origins [79].

1.4 Existing conventional approaches for securing user-driven operation

The plethora of security threat vectors requires robust security schemes to prevent malicious SHS devices operation [12]. In this context, current research works have addressed particular

to two types of conventional mechanisms namely, signature-based intrusion detection systems and user authentication. In the following, we present some of the existing works for each type of mechanism.

1.4.1 Signature-based intrusion detection systems

Intrusion Detection Systems (IDSs) are a typical countermeasure against attacks targeting IoT devices [132]. An IDS detects attacks and malicious operation of SHS devices based on the analysis of outbound/inbound traffic between the IoT devices and the external world i.e., Internet [98, 115]. A signature-based IDS looks for attack signatures or specific patterns that usually indicate malicious or suspicious intent. It is essential for the working of such an IDS to maintain an updated database of attack signatures [114].

In this context, Martin et al. proposed a comprehensive home network defense method against attacks on home IoT devices. This method uses honeypot to find attacks by the signatures-based method and changes settings of firewalls to drop the attacking packets [81]. Besides, Zarpelao et al. presented an IDS to detect anomalous traffic over IoT devices by comparing the packets from the observed traffic to predefined rules [137]. A comprehensive survey of existing works could be found in [45].

Although existing signature-based IDSs assume that legitimate and anomalous traffic signatures are notably different, both attackers and legitimate users may send the same types of packets to operate SHS devices. For instance, if an attacker issued an operation command via a compromised end-user device (e.g., a malware-infected smartphone), such type of IDS cannot distinguish between packets sent by the legitimate user and those sent by the attacker based only on the available information (e.g., IP address) [132].

1.4.2 Conventional user authentication

Traditional and well-known authentication factors have been also leveraged to identify SHS users and prevent unauthorized operation. Factors include some secret that a user knows (e.g., passwords or PIN codes), some token that a user has (e.g., smart-cards), or something that a user is (e.g., fingerprint or face recognition) [91]. A comprehensive review of these schemes could be found in [76, 110].

While the combination of these factors has led to a great improvement known as multi-factor authentication [110, 127], these systems still suffer from several limitations. First of all, while user's credentials are not trivial to guess and are safely stored, they remain vulnerable to social engineering attacks. Moreover, technologies measuring user's biometric characteristics are often intrusive and expensive as well as they are not always available on control device which SHS user uses.

1.5 Existing conventional approaches for securing appified automated operation

To detect and respond to the plethora of threat vectors related to automated SHS operation and which may lead to severe safety consequences, research works have been focusing on well-known program analysis-based techniques. We present some of these works in the following.

1.5.1 Program analysis-based approaches

Program analysis has been applied, either statically or dynamically, on many different settings such as mobile apps [30]. In static analysis, the source code of an SHS automation application (called SmartApp) is analyzed without running it. Whereas in dynamic analysis, the code is run, possibly under-instrumented conditions, to see if there are likely problems [4, 30].

As summarized in Table 2.3, existing program analysis-based approaches to secure SHS appified automated operation have been focusing on the enforcement of policies that describe the security and safety preferences of the SHS.

Table 1.3 Summary of existing program analysis-based approaches

System	Approach	Policy definition	Enforcement technique	Purpose	Analysis type
SmartAuth [120]	Personalized	User-defined and extracted from SmartApps description	Not specified	Permission misuse prevention	Static
Soteria [31]	General purpose	Extracted from SmartApps and trigger-action rules	Model Checking	Abuse prevention	Dynamic
IoTGuard [32]			Reachability Analysis	Policy violation prevention	
Expat [130]		User-defined	Policy decision function		

Tian et al. proposed SmartAuth, an authorization policy-based system that learns about the SmartApp's actual functionality by analyzing their source code and the description provided by developers [120]. Then, the discrepancies between the SmartApps description and their programmed logic are pointed out and displayed to the user through an automatically generated interface. After that, SmartAuth retrieves the user's explanation and approval for the extracted discrepancies using natural-language-generation techniques and enforces the policy by blocking unauthorized commands. Celik et al. proposed Soteria, a model checking based-system to verify whether installed SmartApps adhere to security and safety properties. The enforced properties are a set of systematically developed policies that represent the physical behavioral specifications of users' expectations about the safe and secure behavior of an SHS [31].

IoTGuard is another policy-based authorization system that retrieves SmartApps information (e.g., events and actions) at runtime and stores them in a dynamic model that consists of transitions and states [32]. The dynamic model represents the runtime execution behavior of

the SmartApp. Using the reachability analysis technique, this model is then evaluated against the same policies used by Soteria [31]. Moreover, Expat allows a user to check the desired properties (e.g., consistency, entailment) of them; which due to their formal semantics can be easily discharged by an SMT solver [130].

Although the proposed systems consider additional design and security features beyond the existing authorization models in current SHS automation (e.g., SmartThings Permission Model), they suffer from a major problem related to the pre-definition of the security policy. Indeed, general-purpose policies as proposed by [130] and [32] are not personalized and may not suit all SHSs automation configurations. Moreover, as leveraged by SmartAuth [120], users may not be able to accurately explain their specific security preferences.

1.6 Conclusion

In this chapter, we have provided a theoretical background to understand the IoT-SHS technology with a focus on its technological aspect, provided services, and operation mechanisms. This background allows us to (i) expose security threats that may manifest at different levels of an SHS architecture, and (ii) review the state-of-the-art related work contributing to the design of secure and safe SHS operation. Summary points from this review are the following:

- The plethora of threat vectors that could be exploited requires robust security schemes to prevent malicious/unexpected SHS devices' operation.
- Conventional signature-based IDSs cannot distinguish between operation commands' packets sent by legitimate users and attackers.
- Conventional user authentication schemes suffer from many limitations related to efficiency and usability.

- Conventional AC models (e.g., CapBAC) fail to prevent the misuse of legitimate privileges by a malicious user.
- Existing policy-based program analysis approaches are hindered by the pre-definition of the security policy.

We may conclude that there is a growing need (i) for integrating user behavioral analysis to make more intelligent authentication/authorization decisions, and (ii) for a new scheme that is personalized for each SHS automation configuration and supports self-learning.

Chapter 2

Behavioral Anomaly Detection-based Security: Fundamental Concepts and Existing Approaches

2.1 Introduction

Recently, the limitations of conventional security techniques to efficiently secure SHS devices from malicious/unexpected operation have been warranting researchers to integrate the behavior profiling of both SHS and its users to make more intelligent security decisions. Multiple appellations could be found in the literature to designate such an approach, namely, behavior profiling, behavior modeling [112], behavior analysis [74], behavior-based analysis [100], etc. In this dissertation, we will be using the term Behavioral Anomaly Detection (BAD) [99].

BAD-based security is the coupling of two approaches namely Behavior Profiling (BP) and Anomaly Detection (AD). Such an approach differs from traditional security methods (e.g., signature and rule-based) by building a reference behavioral profile. This latter may be the regular behavior of the target (i.e., users, applications, devices, etc.), or on the contrary, an

attack situation, that will subsequently allow the detection of abnormal behaviors [88, 119]. BAD-based security provides an automated control that identifies something that is not right with a user account, a machine, an application, or a network, and helps security teams investigate and respond to the incident.

In the following sections of this chapter, we first provide a background on BAD-based security including its basic concepts and enabling techniques. Then, we go deep into the details of such an approach and its applications to securing IoT-based as well as non-IoT-based systems.

2.2 Anomaly detection

Anomaly Detection (AD) is considered the main enabler of a BAD-based security approach. AD is a branch of data mining that deals with the identification of atypical records or rare occurrences in the data [33]. In other words, AD is about finding objects that are different or inconsistent with the majority of objects in a dataset. In the literature, the atypical objects detected are called anomalies and are also called, depending on the application context, exceptions, novelties, or outliers [3].

2.2.1 Machine learning-based anomaly detection

A straightforward anomaly detection approach is to define a region representing normal data and declare any observation that does not belong to this region as an anomaly [33]. This approach can be realized through the use of the Machine Learning (ML) paradigm. In particular, the definition of the region representing normal data can be obtained through the training of an ML algorithm on initial data. Then, future data instances can be recognized as normal or anomalous by applying the trained algorithm.

Possible setups

There are multiple possible setups when following an ML-based AD. On one hand, the setup to be used may depend on the availability of labels indicating the actual nature of the initial training data [57]:

- Supervised anomaly detection: describes the setup where the data comprises fully labeled training and test data sets. The objective is to match input data to output labels based on examples already labeled. In this case, the objective is to learn a model that can generalize to unreached inputs and successfully match them to suitable outputs.
- Semi-supervised anomaly detection: also uses training and test datasets, whereas training data only consists of normal data without any anomalies.
- Unsupervised anomaly detection: is the most flexible setup which does not require any labels.

On the other hand, the setup to be used may also depend how data is fed to learn an anomaly detection model [25]:

- Offline (batch) learning: in this setup, the model is trained using the entire dataset that is available at a certain point in time. Once we have a model that performs well on the test set, the model is shipped for production and thus learning ends.
- Online (incremental) learning: in this setup, data streams (either individually or in mini-batches) are fed to train and update the model. Online learning is ideal in situations where data is generated continuously in time, and we need to use real-time data samples to build an anomaly detection model. This setup also allows the trained model to be updated with new data when it is available, without being re-trained from scratch.

Anomaly detection output

An important aspect for any anomaly detection technique is how the anomalies are reported [33]. The output of an ML-based AD can be categorized into two types:

- **Binary label:** a binary output indicating whether an instance is an anomaly or not. For instance, 0 indicates a normal instance while 1 indicates an anomaly.
- **Anomaly score:** a score qualifying the level of "outlierness" of each datapoint e.g., a score in the range of [0,1]. While an anomaly score can also be converted into a binary label using an appropriate threshold [57], it provides more information than a binary label.

For a supervised anomaly detection setup, the output is often a label and it should not be always binary. However, for semi-supervised and unsupervised anomaly detection algorithms, scores are more common.

2.2.2 Taxonomy of anomaly detection techniques

Numerous ML algorithms are suitable for anomaly detection. Researchers have been proposing many taxonomies of ML-based AD techniques. For instance, Chandola et al. [33] classify them into: Classification-based, (2) Nearest neighbors-based, (3) Clustering-based, (4) Statistical-based, (5) Information theoretic-based, and (6) Spectral-based. Whereas, Bergman and Hoshen [21] classify them into: (1) Reconstruction methods, (2) Distributional methods, and (3) Classification-based methods.

Based on the existing taxonomies, in this section, we categorize AD techniques into four categories: (1) Classification-based, (2) Statistical-based, (3) Distance-based, and (4) Density-based techniques. In the following, we present a brief and high-level description of each category and illustrate them with some examples of representative techniques. For an exhaustive list of existing techniques, we refer the reader to detailed reviews such as

[33, 34, 141]. Besides, for more statistical and linear algebra background, we also refer the reader to original publications of proposed techniques.

Classification-based anomaly detection

Classification-based AD trains a classifier that separates between regions containing normal data instances from the rest of regions i.e., outliers. This technique can be grouped into two broad categories: multi-class and one-class AD techniques. The former technique assumes that the training data contains labeled instances belonging to normal as well as anomalous classes. However, such supervised AD setup is more constraining in some cases where the access to samples from anomalous observations is not available [44]. Examples of multi-class classification-based AD include Support Vector Machine (SVM) [26] as well as neural networks classifier [42] and its variants such as multi-layered perceptrons [18].

SVM is essentially a two-class algorithm that seeks to create one or more linear separators to isolate each class of the dataset. These separating hyperplanes are trained on labeled data so that the margin between the different classes is as large as possible. Maximizing the training margin means minimizing the risk of classification error when the separator is used on unlabeled data. An illustration of this type of separator is given in Fig. 3.1.

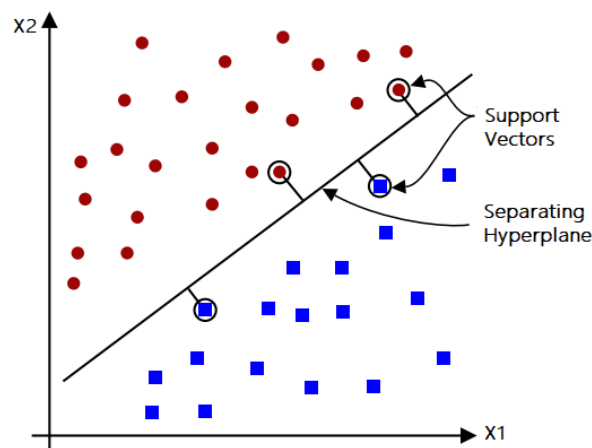


Fig. 2.1 A trivial example of an SVM separating hyperplane

On the other hand, One-class AD only assumes that the training instances belong to one class i.e., the normal class in most cases. The aim is then to learn a discriminating boundary around the normal instances using a one-class classification algorithm such One-Class Support Vector Machine (OCSVM) [109]. OCSVM is an extension of SVM that handles training datasets containing only one class (i.e., positive class). In particular, OCSVM considers the origin of the space to be the only instance of the negative class. The goal, therefore, is to find a hyperplane that separates the examples of the target class from the origin, and which maximizes the margin between the two. Fig. 3.2 shows, in two dimensions, an example of an OCSVM separating hyperplane.

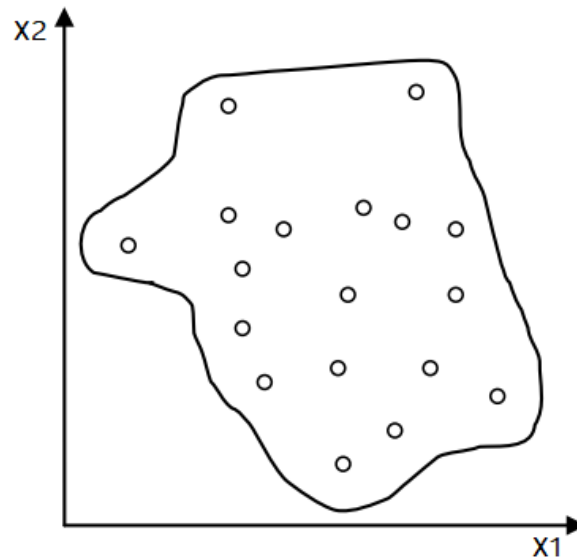


Fig. 2.2 A trivial example of an OCSVM separating hyperplane

The approach of an OCSVM-based AD is to learn a separating hyperplane over the available normal training data. Then, the trained model is used to classify new observations as either normal (if they fall inside the hyperplane, called inliers) or abnormal (if they fall outside the hyperplane, called outliers).

Statistical-based anomaly detection

Statistical-based techniques consist of developing flexible probabilistic statistical models that represent the distribution of the datasets such as Gaussian models [131] and regression models [2]. The degree of the anomaly of a particular data instance is evaluated against its conformity to the established model and receives an anomaly score which characterizes its deviation from the model. A high score indicates a high probability that the object in question is an anomaly.

Moreover, in the literature, we find other statistical methods for detecting anomalies such as histograms [49] and kernel functions [24]. Specifically, the histogram-based techniques consist of developing a frequency profile of the data. Whereas, kernel functions provide an approximation of the data distribution density [135]. Then, an observation that belongs to the less dense area of the data distribution is identified as an anomaly.

Distance-based anomaly detection

Distance-based AD techniques consist of calculating the disparity between instances in a dataset. To measure such heterogeneity, several metrics can be used such as Euclidean distance and Manhattan distance. An instance is considered an anomaly if it is remarkably distant from the majority of objects. In particular, distance-based techniques such as k-Nearest Neighbors (KNN) [102] and weighted k-NN [10] assign an anomaly score to each instance based on its k-nearest neighbors. In this way, being distant, anomalies receive high scores and normal instances receive low scores. Then, anomalies are distinguished by sorting the scores in ascending order and selecting the instances with the lowest scores.

Density-based anomaly detection

Density-based AD techniques measure the degree of the anomaly of an instance by considering the local density of its neighborhood. An example of such a technique is the Local Outlier

Factor (LOF) [28]. LOF inspired its foundation from DBSCAN- a partitioning density-based partitioning method that identifies both clusters and outliers [47]. In LOF, the local density of each instance is calculated to its k nearest neighbors. In particular, the set of distances of a particular instance to its k nearest neighbors are used to calculate its local density. Then, the local densities of all instances are evaluated. The regions of similar density are normal; whereas; outliers have remarkable low local densities compared to their neighborhoods.

2.3 Behavioral anomaly detection-based security

The definition of anomaly takes on more meaning when it is related to a specific context or application. In the context of cybersecurity, a detection of an anomaly may be a sign of an attack-situation such as a network intrusion or a fraudulent transaction in online banking. In these applications, anomaly detection is often coupled with behavioral profiling where the regular behaviors of users, devices, programs, etc. are profiled to detect abnormal ones. Such coupling is also known as Behavioral Anomaly Detection (BAD)-based security [99].

2.3.1 Basic operational scheme

As depicted in Fig. 3.3, a BAD-based security solution operates in two main stages:

- **Profiling:** in this stage, the baseline models are first constructed over the regular conduct of the profiled targets (i.e., users or system entities). The target could be the system user or other entities such as system applications, servers, routers, and data repositories. Regardless of the AD technique, this stage also includes two main processes: (i) collecting historical data on the profiled target, and (ii) building baseline profiles by training AD models.
- **Analysis:** in this stage, the behavior of users/entities is continuously monitored and compared to the established baseline AD models to detect abnormal behaviors, rec-

ognize security threats, and take security actions according to the output of the AD model (i.e., score, label) as discussed previously (Section 3.2.1).

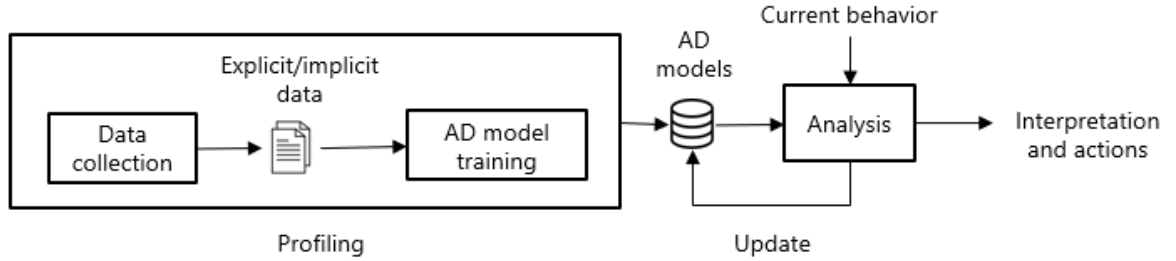


Fig. 2.3 Basic operational scheme of a BAD-based security approach

In the following sections, we provide a detailed presentation of the two BAD-based approach stages. Then, we discuss different challenges facing such an approach.

2.3.2 Data collection

Acquiring data about the regular behavior of the profiled target can be performed through two different methods, namely, the explicit acquisition and the implicit acquisition as detailed in the following.

Explicit data collection Explicit (also called static) data collection is the process of acquiring the profiled target’s static and predictable characteristics [70]. Such data comes usually by filing online forms or from surveys etc. when the target is human or from exiting static information when the target is a system entity. However, this type of data collection has many disadvantages. First of all, users are not interested to reveal their information to anyone as they are concerned about their privacy. Besides, the form filling process might be tedious which the user tries to avoid. On the other hand, static information of entities such as applications and devices may not be always publicly available.

Implicit data collection Implicit (also called dynamic) data collection is about no longer fetching the profiled target explicit data, but rather finding third-party sources and data,

making it possible to extract more unknown/unavailable knowledge. Examples of dynamic data include logs, audits, user-generated content, etc.

Hybrid data collection This method combines the advantages of both implicit as well as explicit data collection. In particular; it takes into consideration both target's static characteristics and also goes retrieving the dynamic information. This approach makes behavioral profiling more efficient and maintains the accuracy of temporal information as it gets updated temporally.

Data pre-processing The collected data may contain many inconsistencies such as incomplete data (i.e., lack of important values or attributes), biased data (i.e., presence of errors produced during entries or automatic collection data), non-coherence (different names or codings in the data, etc.). Besides, raw collected data may not conform to the model or training input format of AD models. Thus, after collecting raw data, it is necessary to implement a preprocessing step before the final step of training AD models.

2.3.3 Anomaly detection models training

Considering the plethora of available anomaly detection techniques (called AD models hereafter), the selection of the appropriate one is not a straightforward task. However, the anomaly detection setup related to the nature and the availability of data limit these choices. Thus, it is important to know if the setup requires a supervised/semi-supervised/unsupervised or an offline/online AD model, as previously discussed.

Due to variations in the behavior of a profiled target, AD modes should be maintained updated. The goal is to remove behaviors from the initial trained model, add new ones or change the behavior weightings without retraining the model from scratch. Updating AD models allows new and supplement information to be added.

2.4 Existing behavioral anomaly detection-based security approaches

A BAD-based security solution could be relevant for multiple cyber systems and environments such as web applications and databases [43]. In particular, it is meant to deal with multiple security threats such as malicious insider, compromised user account, etc. More specifically, through recognizing malicious behaviors, it could: i) identify attacks early on when they are taking their first steps; ii) stop attackers that are already inside and covertly manipulating the system; iii) find the root cause of security incidents and help defend against similar incidents in the future.

In this section, we review the-state-of-the-art BAD-based security approaches. We first review approaches to secure non-IoT-based systems, then, we review those specifically proposed to secure IoT-based SHSs.

2.4.1 Existing approaches for securing non IoT-based systems

To get a broader understanding of the usefulness of a BAD-based approach, we review existing proposed works to secure non-IoT-based systems. This study allows us to understand different thwarted security threats, the types of behavioral analysis features and the data sources used for their extraction, and the AD techniques leveraged, as summarized in Table 3.1.

Web applications When a web user requests a resource (e.g., HTML file), the webserver records these requests in weblogs. By building a baseline over regular requesting patterns, any anomalous request could be then detected. In this context, several methods have been proposed to detect many security threats such as Application layer Distributed Denial of Service (DDoS) attacks. Liao et al. extracted web user behavioral features based on the idea

Table 2.1 Summary of existing approaches for securing non-IoT-based systems

System	Ref.	Security threat	Behavioral features	AD technique	Data source
Web applications	[77]	Application layer DDoS	Requests number per time window, duration between requests	SVDD, RM	Web logs
	[90]		Request rate	PCA	
Databases	[83]	Abuse of privileges	Summarizing queries result tuples in clusters for each user	Clustering	Audit logs
	[84]	Credentials hijacking and abuse of privileges	Rarity, volume, new object accessed	Anomaly scores assignment	
OSN	[125]	Fake and compromised Facebook identities	Number of likes per day, number of likes in different categories	PCA	UGC
	[106]	User account compromise	First activity, action latency, visit duration, browsing sequence	Euclidean distance, self-variance	

that users access and spend more time on pages of their interests [77]. The AD technique used is a hybrid classification algorithm based on Sparse Vector Decomposition and Rhythm Matching (SVD-RM) algorithm. Whereas, Najafabadi et al. applied another anomaly detection technique based on the Principal Component Analysis (PCA) to detect DDoS attacks [90].

Databases Database servers typically maintain an audit log in which they record SQL requests on DB resources (i.e., Tables). By learning the behavior of clients when requesting resources, any changes detected in this behavior could be a sign of abnormal access. In this context, Mathew et al. proposed a clustering-based AD technique to detect users' abuse of privileges [83]. More recently, Mazzawi et al. presented a novel approach to determine whether a DB user activity is malicious or not [84]. The idea consists of checking the user's self-consistency with its previous actions and global consistency with the past actions of similar users.

Online Social Networks (OSNs) OSNs users can be distinguished according to their interaction with several features (e.g., sending messages, commenting, etc.) to detect anomalous behaviors. In this context, Viswanath et al. proposed three types of features to characterize the social behaviors of OSN users such as number of Likes per day and number of Likes in different categories (e.g., politics, sports) [125]. Ruan et al. proposed to characterize a user's social behavior by his/her extroversive activities (e.g., action latency) and introversive activities (e.g., browsing sequence) [106].

2.4.2 Existing approaches for securing user-driven operation of IoT-based SHSs

As discussed in the previous chapter, conventional mechanisms for securing the operation of SHS have been showing several limitations. These latter have been warranting researchers to

integrate the behavioral profiling of both SHS and its users to make more security intelligent decisions.

Continuous Authentication (CA) is one of the main emergent techniques from the BAD-based security approach. Also known as permanent authentication, CA is supposed to increase the level of security by keeping SHS users authenticated permanently and enhance the users' quality of experience by being non-intrusive and minimizing the usage of credentials during the authentication processes [108]. Existing CA approaches for securing SHS operation can be grouped into two categories viz., Physical Behavior-based and Cyber Behavior-based. We present them in the following.

Physical behavior-based continuous authentication

Physical behavior profiling-based CA aims to remedy the limitations related to intrusive authentication biometrics as keystroke dynamics, touchscreen dynamics, etc. [76]. The acquisition of data related to the physical behavior of SHS users could be done through several techniques such as:

- Vision-based: vision-based solutions record an individual's gait patterns when walking via facility cameras. Then, background segmentation techniques are used to extract features from recorded images to verify user identities. However, the vision-based solutions are subject to environments including illumination and camera angle. Furthermore, the high computation consumption and privacy concerns make vision-based solutions infeasible for continuous authentication.
- Voice assistant-based: makes use of the sounds in the home to provide additional context information to decide whether to execute the command, prompt for confirmation or reject the command entirely. However, this feature can be circumvented as voice can be spoofed and users might not be comfortable with their voices being recognized due to privacy concerns [5].

Cyber behavior-based continuous authentication

Given the limitations of physical user behavior-based techniques, researchers have been exploring new opportunities for user authentication by leveraging the behavioral features extracted from user interactions with IoT devices [95]. User cyber behavior-based authentication provides a safer and more convenient way to identify users based on their behavioral interaction with the SHS.

Rath and Colin [104] proposed an access control framework to authenticate the operation of SHS devices in case of user account compromise using association rules as a means to learn user behavior. However, the framework does not use any behavioral features that may efficiently describe SHS user behavior.

Yamauchi et al. [132] proposed a method to detect the exceptional operation of SHS devices. The method first learns sequences of events performed by the user to construct a tree as a baseline. Then, anomalous event sequences are detected by checking whether the sequence is included in the constructed tree. However, the proposed method uses the operation sequence as the only user behavioral feature, thus it cannot accurately identify single commands for which related commands are not observed. Moreover, the proposal only considers the SHS devices separately and does not have a global view of the SHS.

Ghosh et al. [56] proposed SoftAuthZ, a framework for estimating the confidence associated with a device access request. SoftAuthZ computes the belief on a requester based on his/her historical request patterns for a particular device type using a linear regression model. However, SoftAuthZ uses variability in device access requests as the only user behavioral feature besides other non-behavioral attributes such as environmental context, nature of the requested device, etc. Moreover, operation commands are not transformed into feature-based numerical data and are only treated with their original categorical nature.

2.4.3 Existing approaches for securing appified automated operation of IoT-based SHSs

Compared to user-driven operation, little effort has been made to secure the appified automated operation using the BAD-based security approach. The only work that one could find in this context is HoMonit [138]. This proposed system detects misbehaving SmartApps based on a Deterministic Finite Automaton (DFA) matching algorithm. In particular, HoMonit first extracts the expected DFA logic of the installed SmartApps from the source code or their text description. Then, it monitors the behavior of SmartApps from the wireless traffic between SmartThings hub and devices and then matching it with their current working logic using the DFA algorithm. However, the proposed approach does not use any behavioral features that may efficiently monitor the behavior of SmartApps.

2.5 Conclusion

In this chapter, we have provided a detailed theoretical background on the Behavior Anomaly Detection (BAD)-based approach. Then, we have reviewed the existing literature on leveraging such an approach to secure different non-IoT-based systems as well as IoT-based SHSs. In light of this second review, we may conclude that although BAD-based approach has been recently showing its efficiency to secure different types of cybersystems, there is still more effort needed in the context of securing the operation of SHSs.

Part II :

Contributing to The Security and Safety of Smart Home Systems

Chapter 3

Duenna: A Behavioral Anomaly

Detection-based Conceptual Security

Framework

3.1 Introduction

In previous chapters 2 and 3, we have conducted a review of state-of-the-art approaches supporting the design of the secure and safe operation of SHSs. We have drawn our conclusion that conventional security techniques are not mature enough and suffer from providing an efficient security and safety level. Thus, the integration of behavioral analysis of both SHS and its users has been considered as the alternative approach by recent works, yet, these latter are still not sufficient.

Security frameworks lay out the ground for a conceptual, systematic, and consolidated approach to implementing cybersecurity strategies. Many conceptual frameworks that support the design of secure IoT-based systems [82] as well as non-IoT-based ones [16, 48] could be found in the literature. However, to the best of our knowledge, none of such types of

frameworks that are based on a BAD-based security approach has been proposed to secure IoT-based SHSs.

To address this issue, in this chapter, we introduce Duenna: a BAD-based conceptual framework in the hand of the security practitioners (e.g., researchers, developers) for the design and implementation of secure and safe SHSs. Duenna is an abstraction framework [35] that provides a detailed operational architecture to illustrate different operation phases, modules, workflows, behavioral features, data structure, etc. that would be devised and implemented to achieve such an aim. Duenna could be relevant for multiple use cases whether the profiled target is an SHS user or an entity such as an IoT device or an application.

In the following sections of this chapter, we provide a detailed description of the operational architecture of our proposed framework.

3.2 Operational architecture

Fig. 4.1 depicts the operational architecture of our proposed framework. In particular, Duenna operates on an enhanced architecture beyond the basic BAD-based approach (cf. Fig. 3.3):

- Initial operation: once being deployed, Duenna has to be initiated to be ready for the interactive operation. Such initiation is driven by two sub-stages viz., Target Logs Collection and Target Regular Behavior Enrollment. In the former sub-stage, the historical data is collected. Such data should contain relevant and rich traces of the target's activities while interacting with or operating the SHS. In the latter sub-stage, the collected data is prepared and processed to feed the training of AD models.
- Interactive operation: once the initiation operation has finished, Duenna starts monitoring the target's behavior by analyzing the legitimacy/anomaly of its cyber activity (e.g., issued operation commands). This monitoring analysis is driven by two modules viz.,

Anomaly Analyzer (AA) and Action Manager (AM). The former module is responsible for retrieving the trained AD models and applies them to the requested operation commands and produces an Anomaly Score (AS). Whereas, AM is responsible for handling target cyber activities, triggering AA, and taking security actions according to the analysis score produced by the AA.

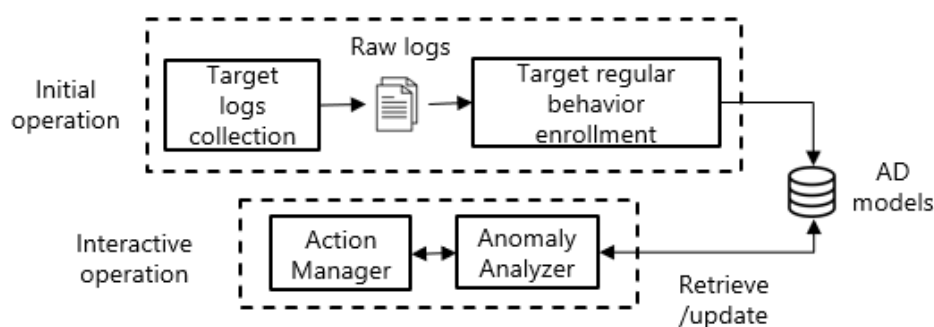


Fig. 3.1 Operational architecture of Duenna

During the interactive operation, Duenna ensures the four following security features:

- **Implication and continuity:** based on a set of behavioral scores that can be discreetly calculated and assessed, Duenna can continuously evaluate the target's behavior during the entire SHS operation.
- **Trust-based verification:** Duenna calculates a confidence score that can be continuously evaluated to ban targets that show anomalous behavior.
- **Risk-awareness:** Duenna switches to a degraded operation mode where highly sensitive SHS devices are disabled from being operated. This functionality makes Duenna aware of the risk that may come with false executed commands when the legitimacy of the current target is not certain.
- **Maintainability:** when the target fails in the trust-based verification, Duenna resets SHS devices to their initial states saved before the execution of the last operation commands.

This functionality prevents dangerous consequences of false executed commands (e.g., left the door unlocked).

In the subsequent sections, we give more details on the two operation modes.

3.2.1 Initial operation

3.2.1.1 Target logs collection

The first step towards the building of target regular behavioral patterns is the collection of its historical data. This data can be directly related to the cyber activities of the target i.e., operating or interacting with the SHS. It can be also related to other SHS components or the physical environment (e.g., user physical activity).

A key source of such type of data is the log files. A log file is a common way to trace the history of systems operation and resources' access by authenticated and authorized users/entities. Collecting and analyzing this data provides a rich understanding of what system users and entities have been doing, when, and how they have been using system resources.

Given the wide range of information that can be collected in a home environment, it is challenging to identify the relevant information to be traced in the target log so the behavioral analysis will be done efficiently [136]. In the following, we recommend the salient information to be traced:

- Target ID: an identifier of the target that issues the SHS operation command e.g., user, an application, etc.
- Device ID: an identifier of the device is being operated, e.g., a door lock.
- Action: the action of the operation command issued by the target on the SHS device e.g., open the smart window, turn on the camera, etc.

- **Trigger:** the concrete/abstract event that triggers the target to act on the resource (if the target is not human). A concrete triggering event happens in the home environment e.g., a motion sensor reports a motion-active event. An abstract triggering event only happens in the system e.g., timer-event or predefined condition such as home mode (home, away, etc.).
- **Device state:** a variable (or a list of variables) that indicates the state of the SHS device. It captures the condition(s) of the device (or other devices) under which the operation command was issued. Examples of such states include a smart window opened, a camera turned on, a smart oven turned off, etc.
- **Environment variables:** a list of variables describing the physical home environment under which the operation command was issued e.g., temperature, humidity, CO2 density, etc.
- **Timestamp:** a combination of the hour of the day, minutes, and seconds, indicating the time condition under which operation command was issued.

It should be noted that the physical deployment of the module responsible for collecting target logs is dependent on the application and its requirements. However, some points should be taken into consideration.

- If the collected logs capture the target inputs (e.g., user inputs), the collector would be better deployed on the input device itself (e.g., an end-user device such as a computer/smartphone, etc.). The input data can be transferred and saved after that to other modules in the system.
- If the collected logs are not directly related to the target (e.g., system events, sensory data, etc.), the collector would be better deployed on a central component (e.g., SHS hub).

- It is not mandatory to use a single log file. Many types of logs could be collected according to the type of collected data; thus, many instances of log collectors could be created.

3.2.1.2 Target regular behavior enrollment

After collecting relevant target logs as recommended, the next step is to train the AD models on the behavioral patterns seen in this data. However, these patterns may not be seen explicitly; thus, the data should be prepared and processed to extract them. In this regard, several pre-steps are performed during this stage called Target Regular Behavior Enrollment as depicted in Fig. 4.2 namely, (1) Raw log preparation, (2) Probabilistic models construction, (3) Behavioral scores extraction, and (4) AD models training. We explain them in detail in what follows.

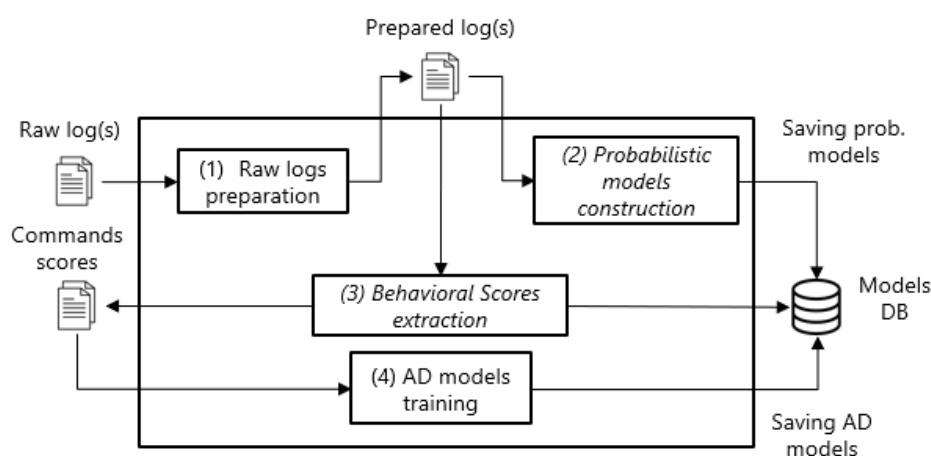


Fig. 3.2 Workflow of target regular behavior enrollment

Raw logs preparation Before being used in the construction of probabilistic models and the training of AD models, the collected logs need first to be prepared. This task allows the extraction of more information about the target patterns from the collected raw logs. In addition to the basic data pre-processing such as cleaning and discretization, this task may also include:

- **Instances selection:** the initial data instances captured by logs to be processed and used in the training of AD models should be first selected.
- **Separation:** when a log file captures the data related to multiple targets, or when logs are gathered from a variety of sources, it would better to separate them for each particular target or data source.
- **Segmentation:** for some use cases, target logs would be segmented by particular information (e.g., by timestamp, IP address, etc.).
- **Fusion:** target raw logs may be correlated to make them more meaningful and distilled into summaries that provide rich context. For instance, associating IP addresses with users, associating cyber activities with physical ones, etc.

Probabilistic models construction Again, behavioral patterns of the target cannot be explicitly seen in the collected logs. To extract them, logs should be processed. To do so, Duenna recommends constructing the so-called probabilistic models. In particular, the construction is performed through the extraction of multiple probabilities from the prepared logs in the form of vectors and matrices. These models ease the extraction of behavioral scores in the subsequent process. Multiple models could be constructed.

Behavioral scores extraction Define what characterizes a target's regular behavior is an important and challenging task [19] since the target behavior analysis accuracy during the interactive operation is strongly dependent on the selection of behavioral features [133, 134]. We call these latter the Behavioral Scores (BSs).

Extracting BSs consists of calculating a tuple of numeric values for each target activity seen in the selected collected logs using the constructed probabilistic models, in addition to other scores that can be calculated independently. To this end, target traced activities are

transformed from categorical data (i.e., a tuple of multiple attributes viz., operated device, action, timestamp) to feature-based numerical data that is suitable for an AD model training.

The recommended scores that could be used to evaluate the behavior of the target are, but not limited to:

- Operation starting probability: when the target operates a series of devices, it is recommended to know the probability by which device the target starts with.
- Intra-operation latency: delay between consecutive operation commands.
- Frequent state probability: probability for a given device to be in a given state.
- Transition probability: the probability of transition between two operation commands.
- Rate: number of operated devices in a time window.
- Sequence: the probability of a sequence of operation commands.

Anomaly detection models training Training AD models on the set of extracted regular Behavioral Scores is the fruit of all the previous enrollment sub-processes. Again, Duenna is a BAD-based security framework that discriminates targets legitimate SHS operation from anomalous ones by detecting abnormal target behaviors. However, in this AD scenario, anomalous SHS targets' behaviors are often unknown at the beginning of the behavior enrollment stage, and only legitimate SHS targets' behaviors are available. As previously discussed in chapter 3 (Section 3.2.1), this setup is recognized as semi-supervised or often as an unsupervised AD.

Moreover, in this scenario, target operation commands need to be analyzed as they are issued in real-time and the initialized AD models during the enrollment stage need to be updated. Yet, retraining an AD model on all new and past data is an intensive task in terms of time and computational resources especially when the update is done on resource-constrained

devices (e.g., hub). As also previously discussed in chapter 3 (Section 3.2.1), this setup is recognized as an online (incremental) AD. Consequently, Duenna employs incremental unsupervised AD models to be trained in the enrollment stage.

3.2.2 Interactive operation

Once the target behavior enrollment is finished, two modules are responsible for handling the interactive operation. The Anomaly Analyzer (AA) is responsible for retrieving the trained AD models and applies them to the target operation commands to analyze their anomaly. AA is also responsible for maintaining the AD models up-to-date while operation commands are being analyzed. Whereas, the Action Manager (AM) is responsible for handling target-issued commands, triggering AA, and taking security actions according to their commands' analysis results.

Duenna may operate in two interactive operation modes. The choice of mode is dependent on the threat model for a specific use case or application e.g., the assumption related to the maliciousness of adversarial target i.e., strong or weak. In the following, we present both interactive modes in detail.

3.2.2.1 Basic interactive operation

In a basic interactive operation, essential security features are provided.

Basic Anomaly Analyzer (BAA) Fig. 4.3 depicts the basic process of AA. Upon receiving a command from AM, the anomaly analysis process is triggered and proceeds as follows:

1. BAA first pulls data collection modules to get real-time data related to operation command (e.g., home environmental variables).
2. Using the command's parameters (e.g., operated device, timestamp, etc.) in addition to the obtained real-time data, AA calculates the corresponding Behavioral Scores (BS).

3. AA passes the calculated BSs to the AD model after being retrieved producing a probabilistic value called Anomaly Score (AS).
4. AA sends AS and BSs back to AM.

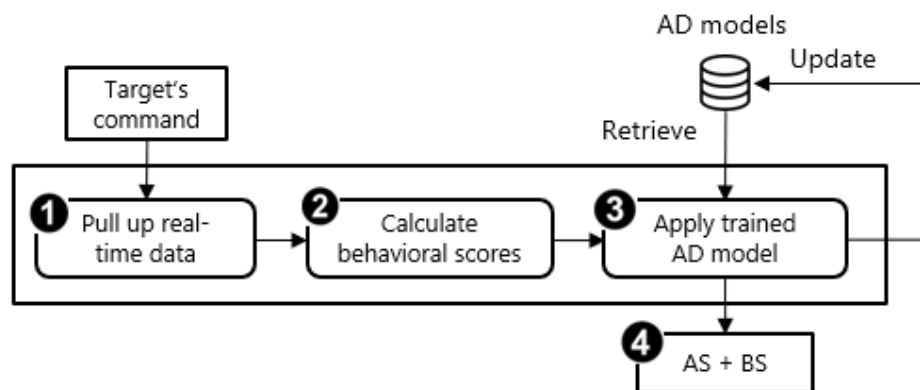


Fig. 3.3 Workflow of Basic Anomaly Analyzer (BAA)

Basic Action Manager (BAM) Fig. 4.4 depicts the basic process of AM. This latter is only triggered by target-issued operation command; thus, it stays at waiting mode when no commands are issued. Upon receiving a target command:

1. BAM handles the command and sends it to BAA. As previously presented, this latter sends the command's BSs and AS back to BAM.
2. BAM verifies the legitimacy/anomaly of the command using the obtained AS.
3. If AS is equal or above a predefined threshold called Anomaly Threshold (AT), the command is sent to be executed.
4. Otherwise, if AS is below AT, the requested command is considered malicious and a confirmation is requested to the target (e.g., reauthentication, a user prompt, etc.). More detail about confirmation choices is given hereafter.
5. If the target fails in the confirmation, the command is rejected and the workflow ends.

6. Otherwise, if the target succeeds in the confirmation, the command is sent to be executed.

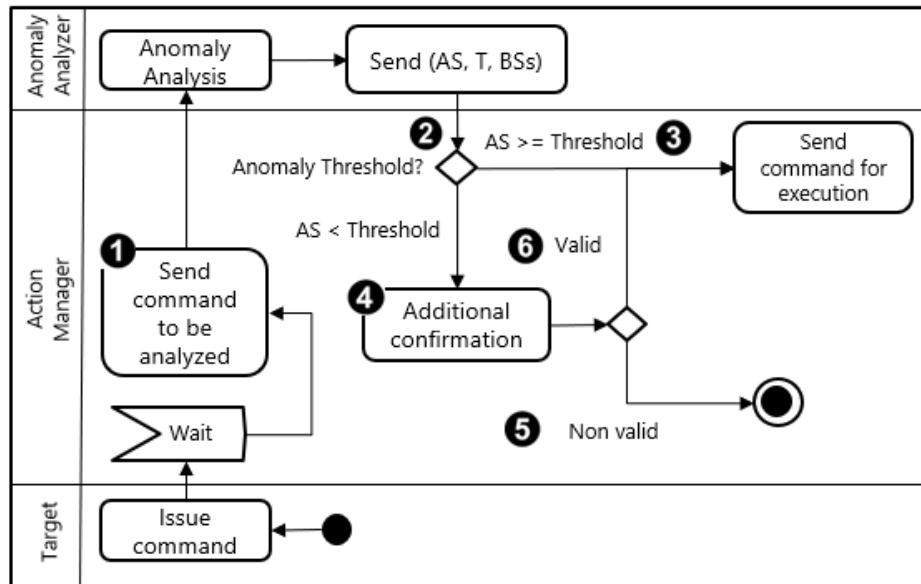


Fig. 3.4 Workflow of Basic Action Manager (BAM)

To tolerate the false rejection of AD models, several choices to get the target's confirmation are recommended.

- User re-authentication: when the profiled target is a user, we recommend that he/she would be explicitly re-authenticated. The user could be prompted to provide a password, a one-time code, etc. Re-authentication is a common technique used in implicit authentication schemes when there is uncertainty about the current user identity [72].
- User prompt: when the profiled target is not an SHS user, we recommend that this latter would be correctly prompted without being habituated or annoyed. Some researchers state that a secure system should either keep humans out of the loop [39]. However, others have also been arguing that a user must be able to retain control of their devices and the option of letting the system autonomously dealing with malicious incidents is not always feasibility since an AD model is not correct all the time due to false

rejection, especially in the SHS context [41]. In this thesis, we believe in the second statement as long as the SHS consumer/user is precisely and correctly involved in the management of security incidents.

- Remembering previous target decisions: a mapping could be established between the context of previous decisions and the context of the requested operations to minimize annoyance (if the target is human), overhead, and delay.

As depicted in Fig. 4.5., upon executing the accepted commands, both Probabilistic and AD models should be updated incrementally without being retrained from scratch. In particular, Probabilistic models are updated from the parameters of the executed commands (i.e., time, device, action, and the period of the day). This guarantees that Behavioral Scores of future requested commands are calculated with recent target behavior. Whereas, AD models are incrementally updated directly with the already calculated Behavioral Scores. This guarantees that recent target behavior has been taken into consideration.

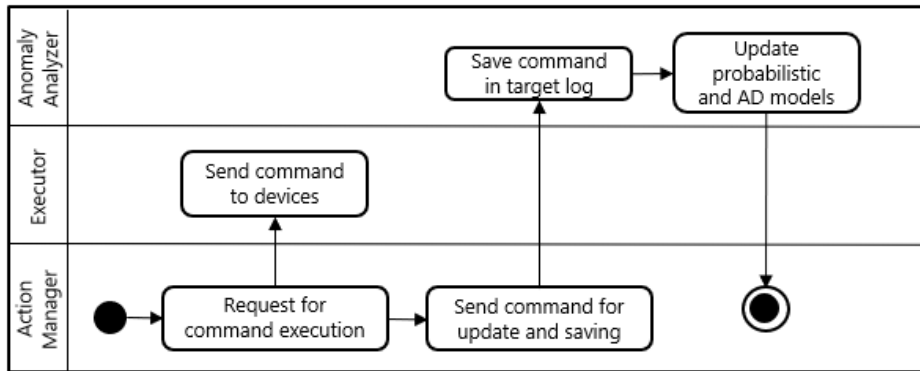


Fig. 3.5 Workflow of target-initiated command execution and models update

3.2.2.2 Enhanced interactive operation

When the assumption related to the maliciousness of the adversarial target is stronger, some additional functions are highly recommended. These latter can be adapted to the requirements of a specific use case or application.

Trust-based verification To prevent the target from bypassing confirmation (i.e., a potential attacker compromising credentials) while continuously showing an anomalous behavior, from keep operating SHS, Duenna recommends a trust-based verification. The motivation is based on the idea that the trust value towards the target can be reflected by his/her behavior [36].

In particular, if the target succeeds in the confirmation, the confidence towards this target is first verified before executing the operation command. While the trust value is still above the allowed level of trust called Lockout Threshold (LT), the command is executed. However, if it drops below LT, the requested command is rejected, and the workflow ends (e.g., the user is logged out). It should be noted that target commands executed after confirmation should not be used to update AD models (cf. Fig. 4.5) since they belong to temporary trust-based verification.

To follow the confidence level related to target behavior, Duenna leverages existing trust models [87]. In particular, a trust value is calculated from Anomaly Score (AS) produced from analyzing the anomaly of the requested command. Hence, the confidence towards the current target may increase or decrease according to the anomaly result of the command. To calculate the change in target trust, we adopt the sigmoid-like formula described in [87], where:

$$\Delta_{Trust}(AS_i) = \min\left(\frac{D(1 + \frac{1}{C})}{\frac{1}{C} + \exp(-\frac{AS_i - AT}{B})} - D, C\right) \quad (3.1)$$

- AS_i is the produced Anomaly Score of the current operation command i .
- AT is the predefined Anomaly Threshold (AT).
- B is the width of the sigmoid function denoting the value of AS in which the maximum value of penalty/reward is given.
- C and D are the upper bound values of the reward and penalty, respectively.

The change of target trust (i.e., $\Delta Trust_i$) is thus related to the AS_i :

- If $AS_i = AT$, then $\Delta Trust = 0$ (i.e., no change in trust).
- If $AS_i > AT$ then $\Delta Trust > 0$ meaning that a reward is given
- If $AS_i < AT$ then $\Delta Trust < 0$ meaning that the trust decreases because of a penalty.

The new value of the trust (i.e., $Trust_i$) is calculated in terms of the current change of trust (i.e., $\Delta Trust_i$) and the last value of the trust (i.e., $Trust_{i-1}$) as described in the following equation (cf. Equation 4.2). In this equation, the minimum value is taken after calculating the change in trust. This choice allows preventing an adversary from taking advantage of the high system trust obtained by the legitimate target before compromising the controlling device/app. Moreover, whenever the target is locked out, the trust value is reset to the trust upper bound which is equal to 100 to simulate a new session starting after the set of actions that lead to this lockout.

$$Trust_i = \min(\max(Trust_{i-1} + \Delta Trust(AS_i), 0), 100) \quad (3.2)$$

Risk-awareness If the target can pass the confirmation and still shows an acceptable level of trust, the requested commands are executed, yet its behavior is still considered suspicious (e.g., potential adversary compromising credentials). To reduce the impact of suspicious executed commands during the trust-based verification, Duenna recommends a risk-sensitive behavior analysis [66] wherein a degradation technique to a restricted operation mode. We inspired this idea from the fault-tolerance systems that usually downgrade to a specific mode (called Degraded Mode) in which they continue to operate but with minimal functionalities when abnormal operating conditions are detected. Once normal conditions are satisfied, the system switches back to its normal operation (called Nominal Mode) [105].

Since an SHS may be configured with multiple IoT devices with different sensitivity degree, devices such as surveillance camera, front door-lock, etc. can be considered highly

sensitive since they have a dangerous impact when are maliciously operated (e.g., a burglar unlocks the front door-lock to enter the house, raises the heating degree on the thermostat to cause a fire, etc.). On the other hand, other devices such as lights, TV, etc. can be considered as lowly sensitive since their malicious operation have no dangerous impact (e.g., switching on/off lights is disturbing but not dangerous).

To this end, when switching to the Degraded Mode, the Action Manager disables highly sensitive devices and only allows the current target to operate lowly sensitive devices. This scheme makes the Action Manager aware of the risk that may come with false executed commands, as the legitimacy of the current target is not certain. The switch-back to the Nominal Mode of operation is done when the target's trust returns to the upper bound of trust (equal to 100 in Equation 4.2).

SHS Maintainability When AM workflow ends due to the target's fail in the trust-based verification, the target's previous operations commands have been executed since AM was uncertain about their maliciousness. To prevent dangerous consequences of false executed commands, such as turning off the camera, raising the temperature, left the door unlocked, etc., a maintainability technique is proposed and recommended wherein SHS devices are reset to their initial states saved before. In particular, devices' states (i.e., on/off, locked/unlocked, etc.) are saved in three points: 1) after successful login; 2) when the operation session expires; and 3) after executing a requested command analyzed as legitimate.

3.3 Conclusion

Motivated by the lack of a conceptual BAD-based framework to secure SHSs, this chapter introduces our proposed framework called Duenna. We present a detailed operation architecture including different workflows and recommended security features. It should be noted

that Duenna should not be considered as a substitute for basic monitoring systems such as intrusion detection systems. It should complement the traditional monitoring infrastructure.

In the next two chapters, we present two use cases to show the applicability of our proposed framework. In chapter 5, we instantiated Duenna to continuously authenticate SHS users. Whereas, in chapter 6, we instantiate Duenna to maintain a safe appified automated SHS operation. The feasibility of both use cases is validated through detailed empirical evaluations.

Chapter 4

Use Case 1: Continuous Authentication of Smart Home System Users

4.1 Introduction

4.1.1 System model and security goals

In this use case [6, 7], we consider a user-driven operated SHS; thus, the SHS model which Duenna will be stated on a user-driven SHS architecture (cf. Fig. 2.2). The overall security objective of applying Duenna is to thwart security threats related to the user-driven operation and make sure that only authorized users are allowed to operate their SHS devices where the profiled and the monitored target is the SHS user.

By applying Duenna, we consider the following additional design and security features:

- Implication and continuity: thanks to the behavioral anomaly detection-based approach, an SHS user can be discreetly and continuously monitored during his/her entire operation session of IoT devices.

- User activity-devices operation correlation: by recognizing the user activity during the operation of devices, it is possible to determine if this operation is legitimate or malicious by considering the historical correlation between operation and activities. For instance, the inhabitant always turn-on the TV while relaxing, turning on the surveillance camera while leaving the house, etc.
- User activity-SHS state correlation: A SHS state in a given time is the status of the hub-connected devices. Same to operation-activity correlation, SHS states are related to a particular activity (or activities) of users. For instance, all doors must be locked while nobody is at home.
- Risk-sensitivity: devices are logically segmented into highly sensitive and lowly sensitive devices according to their impact level on user's privacy, safety, etc.
- Maintainability: SHS would have a restore point when performed operation commands have been recognized as unwanted.
- Trust-based verification: the confidence of an SHS toward users can be continuously assessed to ban those showing an anomalous behavior.

4.1.2 Duenna instantiation

Fig. 5.1 shows the distribution of Duenna's modules on a user-driven SHS architecture (cf. Fig. 2.2).

1. During the initial operation, the historical cyber and physical activities of the user in addition to the historical states of SHS itself are first collected. The collection of the two types of data is performed by the collection modules that are deployed both on the Hub and end-user controlling device.

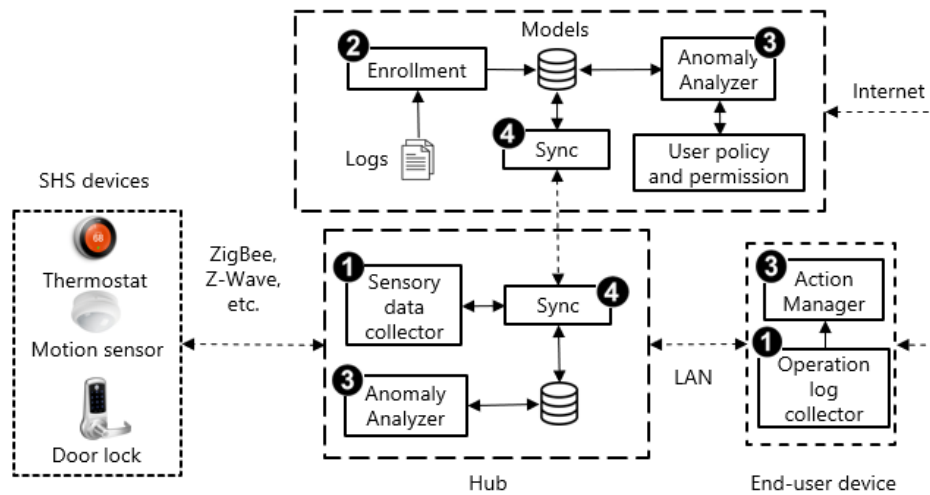


Fig. 4.1 Instantiation of Duenna on a user-driven SHS architecture

2. This data is then used to enroll the regular behavioral patterns of the SHS user seen in these historical data. The enrollment process includes the construction of probabilistic models and the training of incremental AD models on the Behavioral Scores calculated from the probabilistic models.
3. Later in an interactive operation mode, the trained baseline AD models are then used to continuously verify the legitimacy of the user's requested operation commands and take security actions accordingly. This operation is managed by an Anomaly Analyzer deployed both in the hub and the cloud backend as well as Action Manager deployed on the end-user operating device.
4. Once the data collection is completed, a module called Sync is responsible for sending the collected sensory and operation logs to the cloud backend both during the stage of Enrollment as well as during the interactive operation. This module is also responsible for migrating AD models to the hub to be used by the local Anomaly Analyzer when the Enrollment stage is accomplished. Moreover, to keep AD models synchronized on both sides after being updated, the mini-batch used in the update on one side (e.g., hub) is pushed to the other one, and vice versa. This synchronization makes sure

that recent user behavior is always known. For instance, if a user has been remotely operating his/her devices (i.e., cloud-stored AD models have been updated), hub-stored AD models must also be up to date so the user's recent behavior is known if he/she then performs a local operation, later.

Considering the architecture of user-driven operated SHS, one can see that it is reasonable for:

- Enrollment to be performed on the cloud as it is a resource-consuming task.
- Sensory data collector to be deployed on the hub since this latter works as the coordinator of SHS.
- Operation log collector to be operated on the end-user controlling device for collecting user-initiated commands.

4.2 Initial operation

Once Duenna is instantiated and different modules have been deployed, the initial operation is the first process Duenna has to perform to be ready for its interactive operation. The processes executed in this stage include raw logs collection and user regular behavior enrollment as detailed in the following.

4.2.1 Raw logs collection

The first step towards the building of regular user behavioral patterns is the collection of historical data of both the user and SHS itself. As previously presented, this task is accomplished through two modules.

- Sensory data collector: since the hub the coordinator of SHS, this module is deployed on the hub and is responsible for collecting sensory events of IoT devices to trace both

devices' states (i.e., only concerns devices of type actuator, e.g., door-lock) and user physical activities (using devices of type sensor, e.g., motion sensor).

- Operation log collector: this module is deployed on the user controlling app, and is responsible for tracking user operation commands then sending them back to the Sync module on the hub or directly to the cloud backend if commands are requested remotely.

As described in Table 5.1, the collected data consists of different types of logs that trace historical information about both user and devices:

- User operation log: a log file is a common way to trace the history of manipulation/operation of computing resources by their authenticated users by saving information such as who has operated what, and at what time. Analyzing this data provides a rich understanding of what users have been doing, and how they have been using resources. In this work, the Operation Log Collector is responsible for tracking local and remotely executed commands. Both types of commands are saved in one User Operation Log file stored in the cloud backend (cf. Fig. 5.1), since they concern the same user actions.
- User activity log: recent developments in sensing technology have led to the development of wireless sensor networks [72]. These non-intrusive sensors can be used to recognize and trace different human physical activities such as pressure mats to measure sitting on a couch or lying in bed. Collected by Sensory Logs Collector, a User Activity Log saves daily user activities with their start and end timestamps.
- SHS states log: An SHS state in a given time is the status of the hub-connected devices. Collected by the Sensory Logs Collector, this log traces the different status of all devices while being operated at the given timestamp.

Table 4.1 Structure of user/SHS raw logs

Log	Structure	Collector
User operation log	Command ID, operated device, operation action, timestamp	Operation log collector
User activity log	Activity, start time, end time	Sensory data collector
SHS states log	Timestamp, state of device _i , . . . , state of device _n	

4.2.2 User regular behavior enrollment

Once sufficient raw logs are gathered, the baseline models summarizing user patterns seen in the collected raw logs have to be enrolled. The output of the Enrollment process is the trained AD models that are saved to be retrieved in the interactive operation. The Enrollment process is executed on the cloud as it is a resource-consuming task.

Fig. 5.2 depicts the workflow of the Enrollment process including four sub-processes: raw logs preparation, probabilistic models construction, behavioral scores extraction, and AD models training. The subsequent sections explain each sub-process in detail.

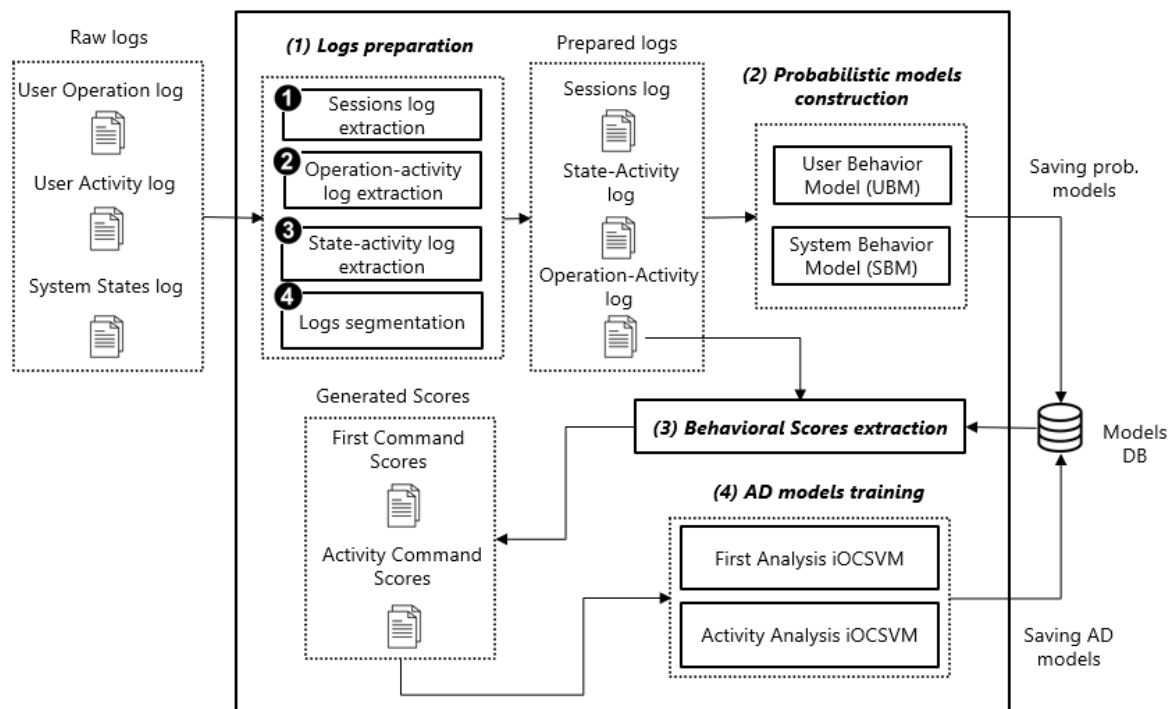


Fig. 4.2 Workflow of user regular behavior enrollment

4.2.2.1 Logs preparation

As recommended by Duenna, the collected user/SHS raw logs need first to be prepared before being used in the construction of probabilistic models and the training of AD models. The structure of the resulting prepared logs is given in Table 5.2 and they are generated by the four following sub-processes:

Table 4.2 Structure of user/SHS prepared logs

Log	Structure
Sessions log	Starting timestamp, period, operation rate, operation sequence
Operation-activity log	Command ID, operated device, operation action, user activity, period, timestamp
State-activity log	Timestamp, period, user activity, state of device _i , ..., state of device _i

1. Sessions log extraction: in the context of SHS, an operation session is the set of operated IoT devices within a time window. In this work, we follow an inactivity-based identification strategy as it is the most commonly used [23]. Specifically, this strategy implies that if there is a break between the user's operation commands which is sufficiently long, it's likely that the user is no longer active and a new session is created when the next operation command is initiated. We use the following information to describe an SHS user session. Starting timestamp: timestamp of the first command of the session, operation rate: number of operated devices during the session, and operation sequence: order of operation during the session.
2. Operation-activity log extraction: since the operation of a particular device is generally related to a particular physical activity (or activities), the mapping of an operation command captured in the user operation log with the corresponding user physical activity (or activities) captured in the user activity log gives a strong insight about the user operation-activity patterns. For instance, the inhabitant always turns on the TV

while relaxing, turning on the surveillance camera while leaving the house, etc. If no activity is performed at the time of operation command, the 'no activity' label is added.

3. State-activity log extraction: the mapping of an SHS state captured in SHS States Log with the corresponding user physical activity (or activities) captured in the activity log also provides an insight about SHS-user activity patterns as the status of device (or a set of devices) is the consequence of particular user activity (or activities). For instance, all doors are locked while the user is not at home.
4. Logs segmentation: segmenting the logs consists of adding the corresponding time interval of the day (i.e., period) to each row record in the logs. Adding this information gives another precision to understand the user's frequent patterns. Indeed, the behavior of an SHS inhabitant through the 24 hours of the day is generally segmented into a set of frequent periods wherein the user has some specific behavioral routines (e.g., waking up and going to work in the morning, sleeping at night, etc.). In recent years, two main time-series segmentation techniques have been used for various purposes (e.g., mobile user data) viz., equal- and unequal-interval strategies [62]. In this work, we follow an unequal interval-based segmentation strategy in which the 24 hours of a day are divided into 2 or more periods with unequal lengths. In the empirical evaluation (cf. Section 5.4), the optimal segmentation that most fits user behavior patterns seen in the logs will be selected among different candidates.

4.2.2.2 Probabilistic models construction

We distinguish two probabilistic models, one related to user behavior viz., User Behavioral Model (User-BM) and another related to SHS behavior viz., SHS Behavioral Model (SHS-BM) as depicted in Tables 5.3 and 5.4, respectively.

Table 4.3 User Behavioral Model (User-BM)

Parameter	Type	Used log	Description
Session Starting	Vector	Sessions Log	Probability values by which a user starts to operate a device at a given period of the day for all existing devices. For example, during the evening, a user frequently starts a session by setting up the temperature on the thermostat with a 0.39 probability.
Intra-Session Transition	Matrix	Sessions Log	Probability values by which a user transits between every two devices. For example, during the morning, a user frequently opens the garage door-lock after unlocking the front door-lock with a 0.6 probability.
Operation-Activity	Vector	Operation-Activity Log	Probability values that a user requests to operate a given device while performing a given physical activity. For example, the user frequently set up the thermostat while outside.

Table 4.4 SHS Behavioral Model (SHS-BM)

Parameter	Type	Used Log	Description
States Transition	Matrix	State-Activity Log	Probability values by which SHS transits between every two states for a given period of the day. For example, when the camera is off, door-lock is never opened.
State-Activity	Vector		Probability values for SHS to be in a given state resultant from executing the requested operation command, while the user is doing a given activity. For example, when the inhabitant is sleeping, doors are locked, lights are off, etc.
Devices Frequent State	Vector		Probability values for each device to be in a specific state at a specific period of the day. For example, in the morning, the door-lock remains locked 80% of the time.

4.2.2.3 Behavioral scores extraction

Extracting the SHS user behavioral scores consists of calculating a tuple of numeric values for each command seen in the operation-activity log using the constructed probabilistic models (i.e., User-BM and SHS-BM) (cf. Fig. 5.2), in addition to other scores that can be calculated independently.

In particular, we distinguish two types of commands each of which is represented with a specific and common set of scores. If the time since the last operation command (called Inactivity Time) is bigger than a specific threshold (called it Inactivity Threshold), the requested command is considered as a Starting Command since it represents the beginning of a new operation session. However, if the Inactivity Time is still below the Inactivity Threshold, the requested command is considered as an Activity Command since it belongs to the same operation session.

Thus, starting command scores are used to describe user behavior when starting an operating session:

- Session starting: probability by which the user starts a session by requesting to operate the given device with the given operation action.
- Inter-session latency: delay between the session starting's requested command and the last command of the previous session that belongs to the same period of the day.
- Operation-activity: probability by which the user requests to operate the given device for the given period of the day while doing the given physical activity.
- State transition: probability by which the SHS transits to the state resultant from the execution of the requested command.
- State-activity: probability by which the SHS transits to the state resultant from the execution of the requested command while the user is doing the given physical activity.

- Device frequent state: probability by which the operated device would be in the state resulting from the requested operation action for the given period of the day.

Activity command scores allow to describe a user behavior when he/she initiate an operation command preceded by other commands belonging to the same operating session:

- Intra-session transition: probability by which the previously operated device would be followed by the device requested in the given command in the same operation session.
- Current sequence: probability to see the current operation sequence.
- Intra-session latency: delay between the requested operation command and its antecedent in the same session.
- Operation rate: number of current operated devices.
- Operation-activity: probability by which the user requests to operate the given device for the given period of the day while doing the given physical activity.
- State transition: probability by which the SHS transits to the state resulting from the execution of the requested command.
- State-activity: probability by which the SHS transits to the state resultant from the execution of the requested command while the user is doing the given physical activity.
- Device frequent state: probability by which the operated device would be in the state resulting from the requested operation action for the given period of the day.

4.2.2.4 Training anomaly detection models

The final step in enrolling SHS user's regular behavior is to train AD models on the extracted BSs. In the use case addressed in this chapter, two types of operation commands are distinguished viz., starting and activity command. Thus, two types of AD models are trained

each of which on the corresponding extracted starting and activity Behavioral Scores called Starting AD model (S-AD), and the Activity AD model (A-AD). The training of both types of AD models is performed on the cloud-backend. Then, the trained models are migrated to the hub via the Sync module. Once AD models training and migration are done, the interactive operation is ready.

4.3 Interactive operation

To handle user-initiated operation commands and only execute authorized ones, we instantiate the two modules responsible for managing the interactive operation. As previously shown in Fig. 5.1, Anomaly Analyzer (AA) is deployed both in the cloud backend and on the hub and is responsible for retrieving the trained AD models and applies them to initiated commands to analyze their anomaly. It is also responsible for user login verification by interacting with the existing user policy and permission module. On the other hand, Action Manager (AM) is deployed in the end-user device and is responsible for handling user-requested commands, triggering AA, and taking security actions according to the anomaly analysis result. Specifically, if an operation command is locally initiated, AM triggers A deployed on the Hub which uses locally stored AD models. However, if an operation command is remotely initiated, the anomaly analysis is performed on the cloud backend using cloud-stored AD models.

As explained in the following, AM and AA may either operate in a basic mode or in an enhanced mode to secure SHS devices from the unauthorized operation. The two modes are dependent on the assumption put on the maliciousness of the SHS adversarial user.

4.3.1 ICA: Implicit and Continuous Authentication

Assuming that an adversarial user may only compromise the controlling device but not the security credentials, Duenna ensures an Implicit and Continuous Authentication called ICA to prevent the assumed adversary from taking control over the controlling device. In the following, we present the ICA operation of the Anomaly Analyzer (ICA-AA) and Action Manager (ICA-AM).

4.3.1.1 ICA-Anomaly Analyzer (ICA-AA)

Fig. 5.3 depicts the workflow of the ICA Anomaly Analyzer (ICA-AA). Upon receiving an initiated operation command from the ICA Action Manager (ICA-AM), the anomaly analysis sub-process is triggered.

1. The Anomaly Analyzer first pulls the hub to get current sensory data and organize it to extract current devices' states and user physical activity.
2. According to its nature (i.e., starting or activity command) which depends on the current user's Inactivity Time as explained earlier, ICA-AA calculates the corresponding Behavioral Scores (BSs) of the command using the extracted operation commands parameters (e.g., operated devices, timestamp, etc.).
3. Then, the corresponding AD model is used for analyzing the calculated BSs (i.e., S-AD or A-AD) producing an Anomaly Score (AS) in the range of $[-1, 1]$.
4. ICA-AA sends back (BSs and AS) to ICA-AM.

4.3.1.2 ICA-Action Manager (ICA-AM)

Fig. 5.4 describes the workflow of ICA Action Manager (ICA-AM) and it includes the following sub-processes:

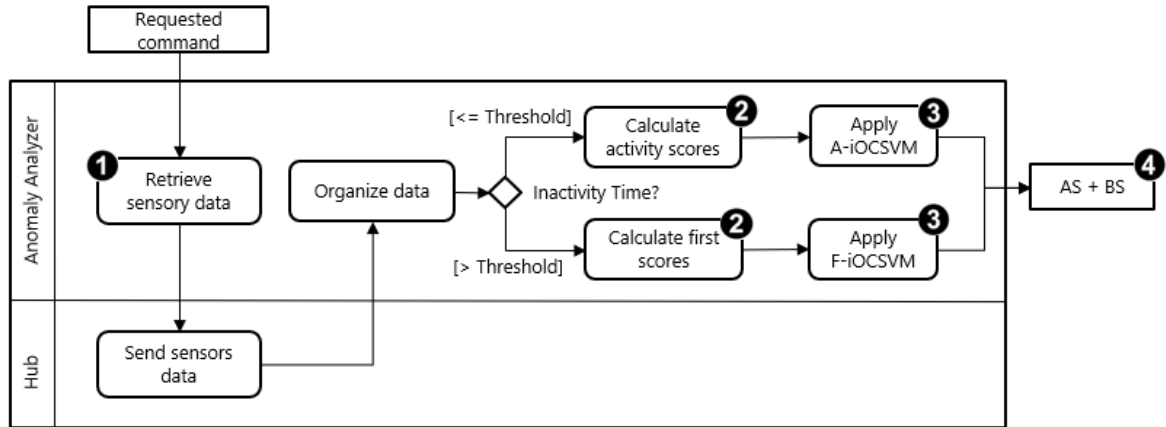


Fig. 4.3 Workflow of ICA-Anomaly Analyzer (ICA-AA)

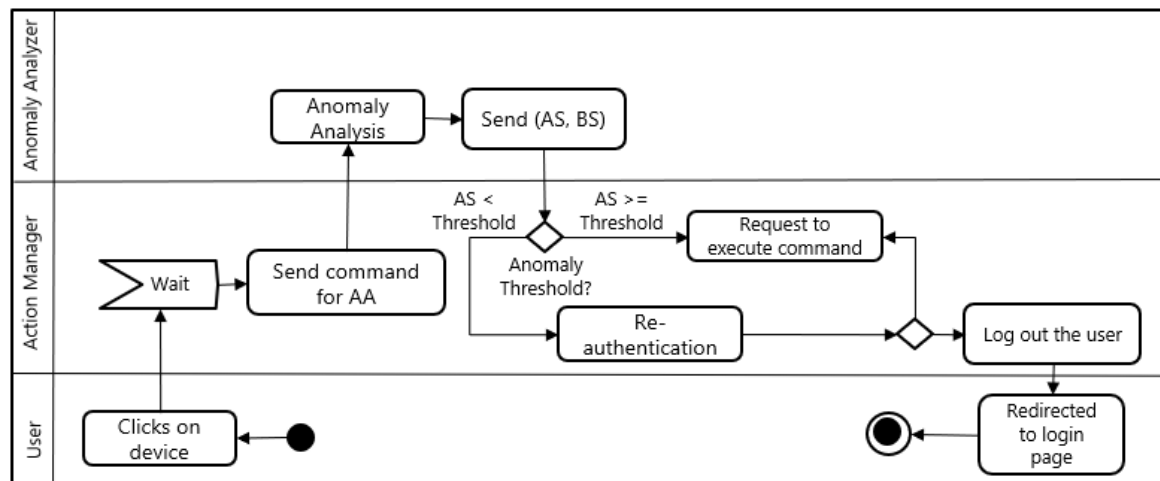


Fig. 4.4 Workflow of ICA-Action Manager (ICA-AM)

User login As depicted in Fig. 5.5, by providing the correct credentials (e.g., e-mail and a password), the user should be authenticated to the controlling app and authorized to operate the SHS devices. In particular, to verify user login, ICA-AM sends user credentials to ICA-AA deployed in the cloud backend which in turn sends them the existing User policy and permission module to be verified. However, if the controlling app has been closed and opened once again, no login is required.

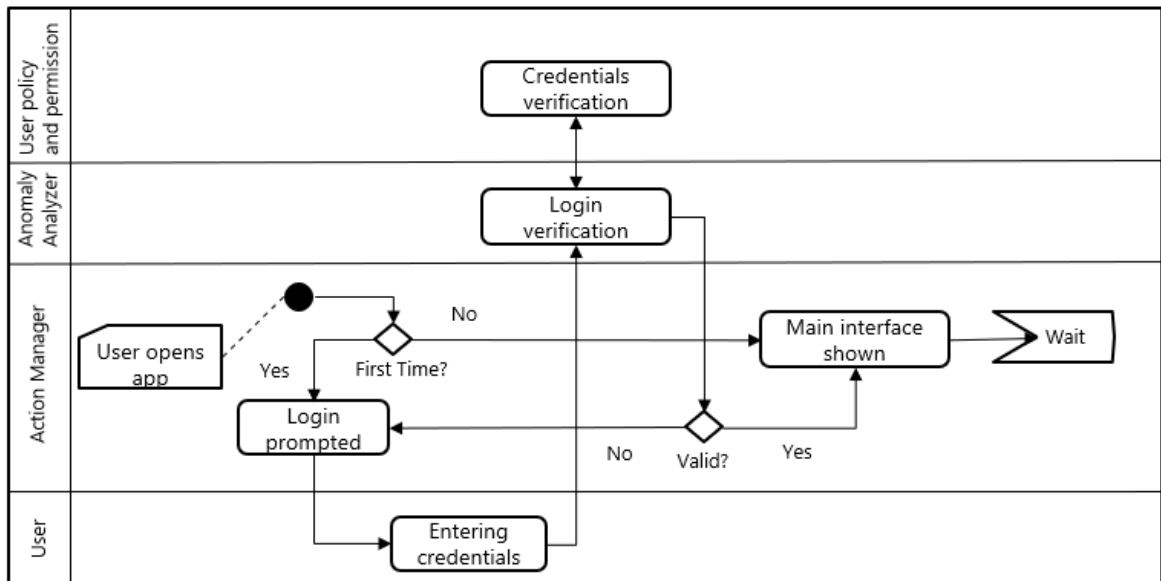


Fig. 4.5 Workflow of user login

Re-authentication Backing to Fig. 5.4, the wait function means that the continuous analysis is triggered by user-requested commands. Once a user requests to operate a device, the Action Manager handles the requested command and sends it to the Anomaly Analyzer which sends back the command's Behavioral Scores (BS) and Anomaly Score (AS) to be verified by the Action Manager. If this score is below a predefined threshold (we call it Anomaly Threshold), the requested command is considered malicious and an explicit re-authentication (i.e., password verification) is prompted to the user. This is a common technique used in implicit authentication schemes when there is uncertainty about the current

user identity [72]. Whereas, if the credentials provided by the user are not valid, he/she is directly logged out from the main interface of the controlling app.

Command execution and models update If the Anomaly Score is above the Anomaly Threshold (AT), or if the user succeeds in the re-authentication, the requested command is sent to the hub to be executed as shown in Fig. 5.6. Upon executing the commands analyzed as legitimate, both Probabilistic and AD models should be updated incrementally without being re-trained from scratch. On one hand, vectors and matrices of user behavior and SHS behavior models (i.e., User-BA and SHS-BA) are both updated from the parameters of the executed command (i.e., time, device, action, the period of the day). Updating probabilistic models guarantees that Behavioral Scores of future requested commands are calculated with recent user behavior. On the other hand, AD models are incrementally updated directly with the already calculated Behavioral Scores.

The update is done after each user session (i.e., a set of consecutive requested commands) considering that the same user never changes his/her operating way in one single session, i.e., cannot be in two places at the same time.

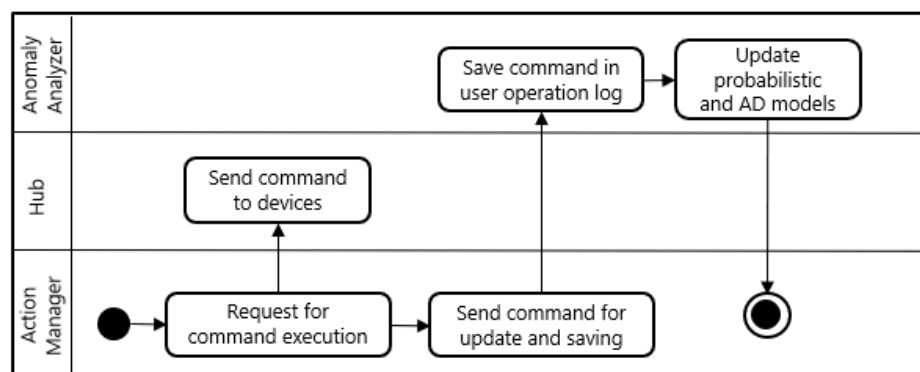


Fig. 4.6 Workflow of user-initiated command execution and models update

4.3.2 MTR-ICA: Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication

Assuming that an adversarial user may either compromise the controlling device or the security credentials, Duenna ensures a Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication called MTR-ICA to prevent such adversary from operating SHS devices. In the following, we present the MTR-ICA operation of the Action Manager and Anomaly Analyzer.

4.3.2.1 MTR-ICA-Action Manager (MTR-ICA-AM)

Fig. 5.7 shows the process of the MTR-ICA Action Manager (MTR-ICA-AM). Compared to ICA Action Manager, in this operation mode, more security features are employed viz., Maintainability, Trust-based verification, and Risk-awareness, as presented in the following.

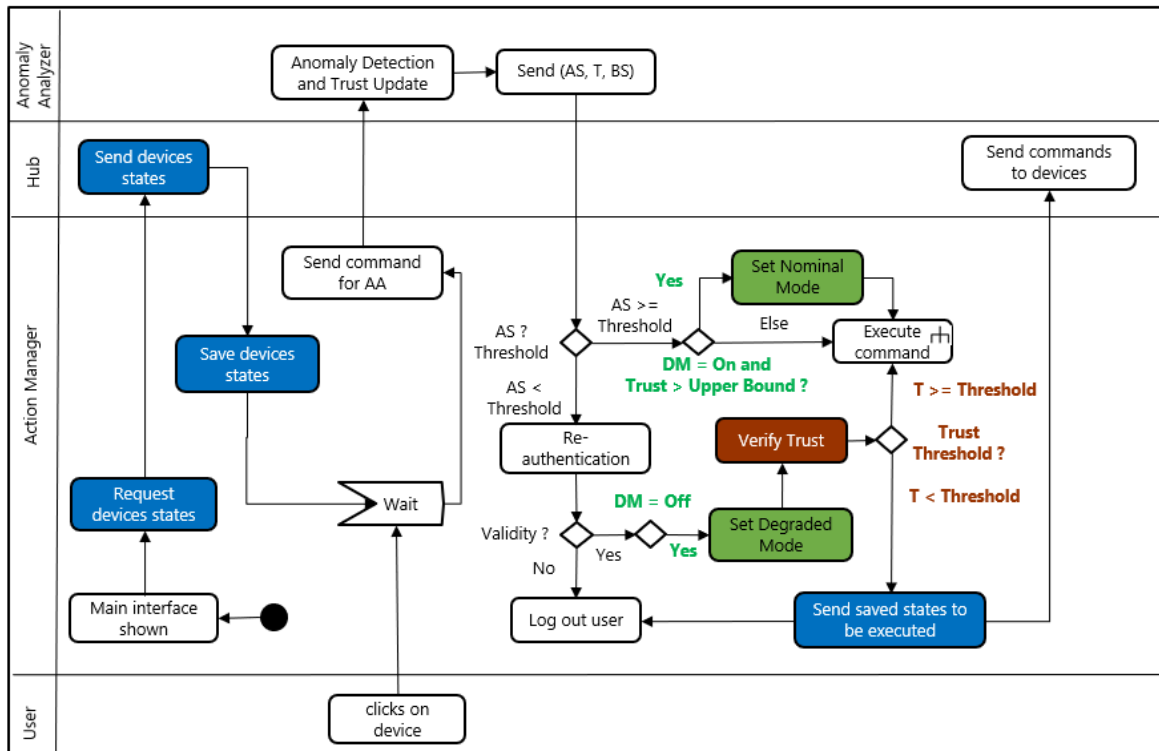


Fig. 4.7 Workflow of MTR-ICA-Action Manager (MTR-ICA-AM)

Trust-based verification (in red) Previously, the Action Manager was executing an initiated operation command directly if the credentials provided by the user are valid. To prevent a user, that can successfully pass the re-authentication (i.e., a potential attacker compromising credentials) while continuously showing an anomalous behavior, from keep using the controlling app, we employ the recommended trust-based verification. The formula (Equation 4.1) is used to follow the confidence level related to user behavior. In particular, if the user succeeds in the re-authentication and the trust value is still below the Lockout Threshold, the command is executed. However, if it drops below LT, the command is rejected, and the user is logged out.

Risk-awareness (in green) To reduce the impact of suspicious executed commands, we employ the risk-awareness feature recommended by Duenna. In particular, during the trust-based verification, the degraded operation mode is enabled where highly sensitive devices are disabled from being operated. Once the trust towards the user returns to the upper bound, the nominal operation mode is re-activated. Otherwise, if the user still shows an anomalous behavior, degraded mode stays enabled until the user is logged out.

SHS Maintainability (in blue) To prevent dangerous consequences of false executed commands, the Maintainability feature recommended by Duenna is employed. In particular, once the user is logged out after failing in the trust-based verification, SHS devices are restored to their initial states. To save initial devices' states, AM requests the hub for current devices' states at three points: 1) after successful login, 2) when the current user session expires, and 3) after executing a requested command analyzed as legitimate.

4.3.2.2 MTR-ICA-Anomaly Analyzer (MTR-ICA-AA)

To ensure the risk-awareness feature, MTR-ICA-Anomaly Analyzer (MTR-ICA-AA) needs to be adapted as shown in Fig. 5.8. For doing so, in addition to the conventional Starting

AD (S-AD), we recognize two types of Activity AD models as depicted in Table 5.5. The Nominal Mode AD (NMA-AD) model is trained on all eight Activity Command analysis scores. However, for the Degraded Mode AD model (DMA-AD), three scores viz., Device Transition Probability, Current Sequence Probability, and State Transition Probability are excluded, and only five are used for the training.

We explain this exclusion as follows. In the Degraded Mode, since highly sensitive devices are disabled from being operated, the probabilities of the three mentioned scores may influence the legitimate user behavior since they are dependent on a particular order and type of device. For example, if a user usually operates the heater in the evening and this device is disabled by the Degraded Mode, then the user will operate another one whose usage probability at this time is low, thus being considered as an anomalous user. In contrast, the rest of the scores can properly describe the regular user behavior even in the Degraded Mode since they did not depend on any order of devices.

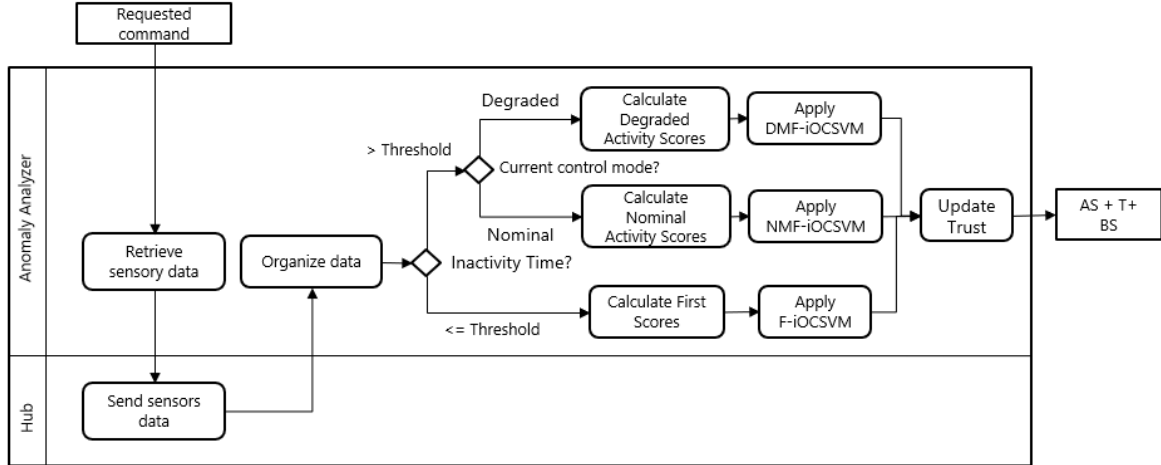


Fig. 4.8 Workflow of MTR-ICA-Anomaly Analyzer (MTR-ICA-AA)

Table 4.5 Types of AD models

Model	Training scores	Number of scores	Operation mode
S-iOCSVM	Starting scores	6	Nominal
NMA-iOCSVM	Activity scores	8	Nominal
DMA-iOCSVM	Activity scores	5	Degraded

4.3.3 SA-MTR-ICA: Self-Adaptive, Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication

Assuming that the legitimate user may change his/her regular behavior, Duenna ensures a Self-Adaptive, Maintainable, Trust-based, Risk-aware, Implicit, and Continuous Authentication. Although it could be possible to assume that the behavior of an SHS user is static over the lifetime of the system and does not change after the enrollment stage, in practice, however, a user may change his/her behavior for several reasons. For instance, during the quarantine caused by Covid-19, people were obliged to stay in their homes. As a consequence, their habits and daily routines have been changed; thus, the way they operate their SHS devices has been also changed. Such drift in the behavior of SHS users makes Action Manager unable to recognize legitimate operation commands since they are deviating from the learned normal behavior. Consequently, it is important for AA to quickly adapt to such a drift.

To do so, we came up with the idea that when the trust-based verification starts, the user under such verification may be either a potential adversary or the legitimate user is behaving differently. As depicted in Fig. 5.9:

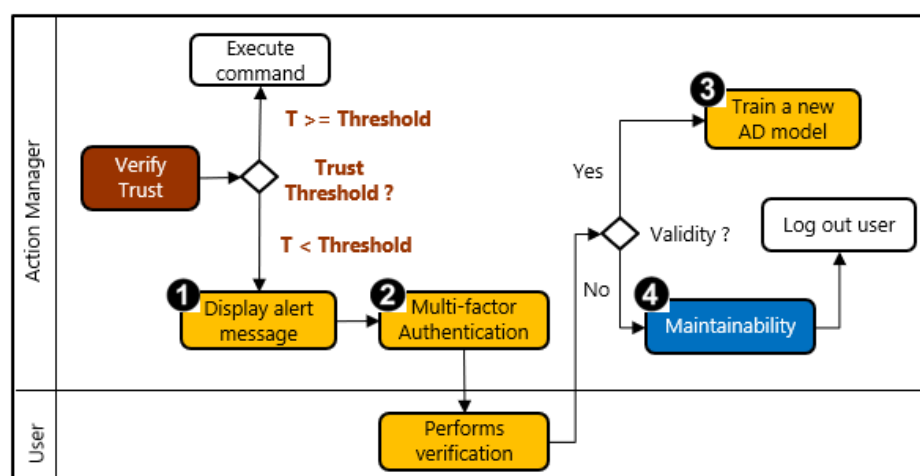


Fig. 4.9 Workflow of SA-MTR-ICA-Action Manager (SA-MTR-ICA-AM)

1. When a user fails in the trust-based verification i.e., trust drops below the allowed threshold, the Action Manager displays an alert message to the user saying: “*An abnormal behavior has been observed, please confirm your identity to consider this behavior as legitimate in the future*”.
2. Since we are also assuming that adversaries may compromise the controlling device or the security credentials, the verification should be performed using multi-factor authentication.
3. If the user succeeds in such verification, a new AD is trained with the accumulated operation commands during the trust-based verification. The choice of keeping the old AD model and training a new one allows preserving the old behavior of the legitimate user when he/she goes back to it e.g., after the end of quarantine caused by covid-19.
4. Otherwise, if the user fails in multi-factor authentication, SHS Maintainability feature is performed and the user is logged out.

4.4 Empirical performance evaluation

In this section, we conduct an empirical evaluation to show the efficiency of this use case of Duenna through the obtained user authentication metrics. The choice of this validation methodology instead of others, such as formal verification [1, 14], is mainly motivated by the adopted BAD-based approach. Indeed, anomaly detection uses a large amount of data collected and processed during both the enrollment and analysis stages, where anomaly detection models are trained and tested, respectively.

In this regard, we address the following research questions:

- RQ1: What is/are the best combination(s) of Inactivity Threshold for user session identification and Day Segmentation to get a better authentication performance? (Section 5.4.3).
- RQ2: Using the obtained combination(s), what is the authentication performance of Duenna in different smart home layouts and devices? (Section 5.4.4).
- RQ3: How this Duenna's use case performs compared to similar frameworks? (Section 5.4.5).
- RQ4: What is the best value for the Lockout threshold for an effective trust-based verification? (Section 5.4.6).
- RQ5: What is the computational complexity of incremental anomaly detection compared to the regular one? (Section 5.4.7).

4.4.1 Evaluation datasets

As described in the enrollment stage (Section 5.2.2), three types of data are used to train probabilistic and AD models viz., user operation log, SHS states log, and user activity log. Although several datasets such as KDDCUP99 [118] and CICD2017 [111] have been created over the past two decades for evaluation purposes of BAD-based security systems, they do not include any specific characteristics of IoT-based systems as these datasets do not contain sensors' reading data or IoT network traffic [46]. Fortunately, to remedy the lack of such type of data, some IoT-specific datasets have been seen in the literature recently.

In this section, we present the datasets which we use to evaluate our proposed framework. In particular, we propose an adaptation of a non-IoT-specific dataset to generate our SHS-specific dataset. Besides, we use another IoT-specific dataset that is publicly available.

Our adapted dataset

Since it has been extensively used in many ambient intelligence applications such as SHS recommender systems [103, 123], the history data of user physical activities inside a smart environment can be easily found in public repositories. To generate an IoT-specific dataset from non-IoT-specific ones, we propose to adapt the history data of manual operation of appliances and objects by inhabitants in real-world home environments and assume that this data belongs to a controlling app manipulation.

The data we will be using for this purpose is the one collected by the University of Amsterdam [123] which contains three datasets (called houses A, B, and C). Each one of the houses was instrumented with wireless sensors (e.g., contact switches to measure open-close states of doors; pressure mats to measure lying in bed, etc.) to record the activities of one single inhabitant during several weeks. The activities include both manual operation of different house appliances (e.g., open door-lock, turn-on microwave) as well as daily living activities (e.g., cooking, sleeping, etc.). Inhabitants of three houses operate their appliances differently while performing different activities, as depicted in Table 5.6.

Table 4.6 Description of our adapted dataset for evaluating use case 1

	House A	House B	House C
# of days	24	27	20
# of physical activities	10	13	16
# of all operation commands	2126	2580	1396
# of operation commands per day (mean)	84	90	68

DS2OS dataset

Distributed Smart Space Orchestration System (DS2OS) is an open-source dataset publicly available on Kaggle [52]. It has been collected in a virtual IoT environment comprising a collection of micro-services during 24 hours. The simulated devices include eight types of devices viz., light controllers (22 instances), temperature controllers (20 instances), movement

Table 4.7 Frequency distribution of considered attacks

Attack	Frequency count	% of total data
Denial of service	5780	57.70
Scan	1547	15.44
Malicious control	889	08.87
Malicious operation	805	08.03
Spying	532	05.31
Data type probing	342	03.41
Wrong setup	122	01.21

sensors (21 instances), washing machines (3 instances), battery sensors (6 instances), door lock sensors (5 instances), 4 heating control sensors (4 instances), and smartphones (3 instances). The size of the collected dataset is 357952 samples including 347935 samples representing normal behavior and 10017 samples representing anomalous data. Table 5.7 shows the frequency count and percentage of the considered attacks.

4.4.2 Selection of anomaly detection model and evaluation methodology

Again, Duenna recommends incremental unsupervised anomaly detection models to be trained in the enrollment stage. While several models could be found in the literature (cf. Section 3.2.2), our selection is based on practical applications in the past and the attention in the scientific community [57]. In particular, in the use case addressed in this chapter, we use the One-Class Support Vector Machines (OCSVM) [109]. This choice is motivated by the fact that it has shown high performance in detecting anomalies in many other application domains compared to other AD models [55]. One incremental version of such model (i.e., iOCSVM) has been recently proposed and its efficiency has been shown compared to contemporary batch and incremental one-class classifiers [73].

In the following two sub-section, we present how iOCSVM is evaluated using each one of the described evaluation datasets.

Evaluation methodology using our adapted dataset

Our adapted dataset contains the data of three users each in one home environment; hence, three baselines (i.e., probabilistic models, operation commands' behavioral scores, and iOCSVM models) are built over the extracted logs of each user (i.e., user operation log, user activity log, and SHS states log).

To test the performance of the Anomaly Analyzer; thus, testing the iOCSVM, we would normally need labels that tell whether an operation command is malicious or not. Since our adapted dataset only contains legitimate users' data and does not include malicious operation commands, we propose to adopt a primary vs adversary strategy. Specifically, since the evaluation dataset contains the data of three users each in one home environment, three baselines (i.e., probabilistic models, operation commands Behavioral Scores, and iOCSVM models) are built over the extracted logs of each user (i.e., user operation log, user activity log, and SHS states log). At each time, one inhabitant is designated as the primary user where his/her iOCSVM models are considered for the evaluation; whereas, the two remaining inhabitants are considered as adversaries and their operation commands Behavioral Scores are considered as the testing data. After that, a part of the Behavioral Scores of the primary user is also included in the Behavioral Scores of the two adversaries. This process is then repeated, designating each of the other inhabitants as the legitimate user in turn.

As described in Fig. 5.10, the training and testing strategy proceeds as follows:

1. Using his/her probabilistic models, the Behavioral Scores of the selected primary user are calculated from his/her operation-activity log and then separated according to the nature of the operation command into starting and activity Behavioral Scores.

To make sure that the performance evaluation results will not be biased or coincidental, 5-fold cross-validation is employed where the four following steps (2-5) are repeated five times.

2. The calculated scores of the primary user are split into two parts (80% and 20%). In particular, scores are split into daily timeframes where the number of days equivalent to the percentage is rounded off. For instance, since the data of House A is recorded during 24 days, 80% of data denotes 19 days; whereas, the remaining 20% denotes 5 days.
3. On one hand, the 80% part (including starting and activity scores) is used to train S-iOCSVM and NMA-iOCSVM models, respectively, which are then saved to be used in the testing stage.
4. On the other hand, at each time, using the probabilistic models of the primary user, the adversarial Behavioral Scores of the two remaining users designated as adversaries are calculated from their operation-activity logs, and then separated into starting and activity scores.
5. The remaining 20% part of primary user scores is then combined with each adversary scores to construct the final testing Behavioral Scores.
6. The previously trained starting and activity iOCSVM models are exported to be tested on the corresponding final scores.
7. The anomaly detection results of both models are combined to be evaluated.

Evaluation methodology using DS2OS dataset

Since the collected DS2OS dataset refers only to one user, a single regular baseline (i.e., probabilistic models, operation commands Behavioral Scores, and iOCSVM models) is built over the extracted logs of this user (i.e., user operation log, user activity log, and SHS states log). The advantage of this dataset is that multiple attack scenarios are considered in the dataset and thus malicious operation commands are available. The behavioral malicious of

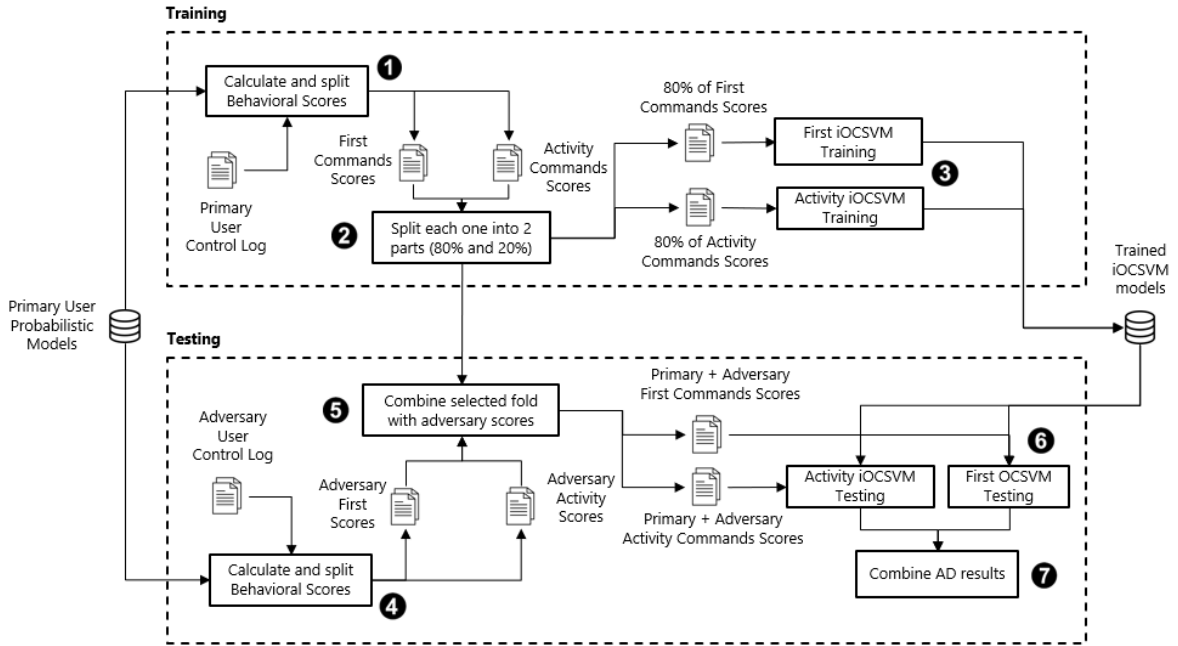


Fig. 4.10 Training and testing process for our adapted dataset

anomalous commands from each considered attack are calculated using the legitimate user baseline.

To test the performance of the Anomaly Analyzer; thus, testing the iOCSVM, we employ a similar 5-folds cross-validation process:

1. The calculated behavioral scores of the legitimate user are split into two parts (80% and 20%). In particular, since the dataset is collected only during 24 hours, the scores are split into hourly timeframes where the number of hours equivalent to the percentage is rounded off. That is, 80% of data denotes 19 hours; whereas, the remaining 20% denotes 3 hours.
2. On one hand, the 80% part is used to train S-iOCSVM and NMA-iOCSVM models, respectively, which are then saved to be used in the testing stage.
3. On the other hand, the remaining 20% part of legitimate user behavioral scores is then combined with the ones of each considered attack to construct the final testing behavioral scores.

4. The previously trained starting and activity iOCSVM models are exported to be tested on the corresponding final scores.

4.4.3 Tuning of inactivity threshold and day segmentation

During the enrollment stage, two variable parameters are used when preparing the history logs for the construction of the probabilistic models. Specifically, since we are following an inactivity-based identification strategy in extracting user session logs, choosing the best Inactivity Threshold to identify a session is challenging. Besides, in log segmentation where the corresponding period of the day is added to each record in the logs, choosing the best day division is also challenging. Since the Behavioral Scores used to train the iOCSVM models are calculated from the probabilistic models, different combinations of the two parameters would give different authentication results.

To find the combination (or the set of combinations) that better fits different history logs of the three users, different candidates of Inactivity Threshold and day segmentation are first applied to calculate Behavioral Scores. Then, the trained iOCSVM on the set of scores is evaluated using the Area Under the ROC Curve (AUC) metric. AUC measures the degree of separability of the iOCSVM between primary and adversary classes using various Anomaly Threshold settings. Finally, the optimal combinations are selected for training the final iOCSVM models to be evaluated. For the Inactivity Threshold, a range of values varying from 1 minute to 1 hour is employed to extract the sessions' logs. While for day segmentation, three candidates wherein the 24-hours-a-day are divided into two (2) or more periods with unequal lengths are used for logs' segmentation, as depicted in Table 5.8.

Fig. 5.11 and Fig. 5.12 show the AUC trend with different day segmentation and inactivity threshold candidates using our adapted dataset and DS2OS datasets, respectively. As one can see in these figures, manipulating different day segmentation and Inactivity Threshold candidates gives different AUC values.

Table 4.8 Day segmentation candidates

Candidate	# of periods	Segmentation
1	2	Night: ['00:00:00','07:59:59'], Day: ['08:00:00','23:59:59']
2	3	Night: ['00:00:00','07:59:59'], Day: ['08:00:00','17:59:59'], Evening: ['18:00:00','23:59:59']
3	4	Night: ['00:00:00','07:59:59'], Morning: ['08:00:00','11:59:59'], Afternoon: ['12:00:00','17:59:59'], Evening: ['18:00:00','23:59:59']

We can see that the AUC value certainly gets improved with small IT values. In particular, IT values smaller than 900 seconds and smaller than 100 seconds are the best values that we can use for extracting user sessions log regarding our adapted dataset and DS2OS datasets, respectively. However, the three segmentation candidates give almost similar AUC values for both datasets and do not have a considerable impact on the user behavioral pattern.

4.4.4 Authentication performance evaluation

After determining the best values of the two parameters, we use this combination to train and test the performance of iOCSVM models following the same training and testing cross-validation process (cf. Fig. 5.9). This is called nested cross-validation (CV) since we are using two CV procedures.

To evaluate the ability of Anomaly Analyzer in identifying adversaries without incorrectly rejecting the legitimate users, we calculate the fraction of adversaries' operation commands that are incorrectly accepted which is known as False Positive Rate (FPR). However, since the profiled target in this use case is a user, we use the term False Acceptance Rate (FAR) that denotes the same fraction because it is more meaningful. On the other hand, to measure the legitimate user convenience level, we calculate the fraction of authorized operation commands that are incorrectly rejected. Similarly, we use the term False Rejection Rate (FRR) instead

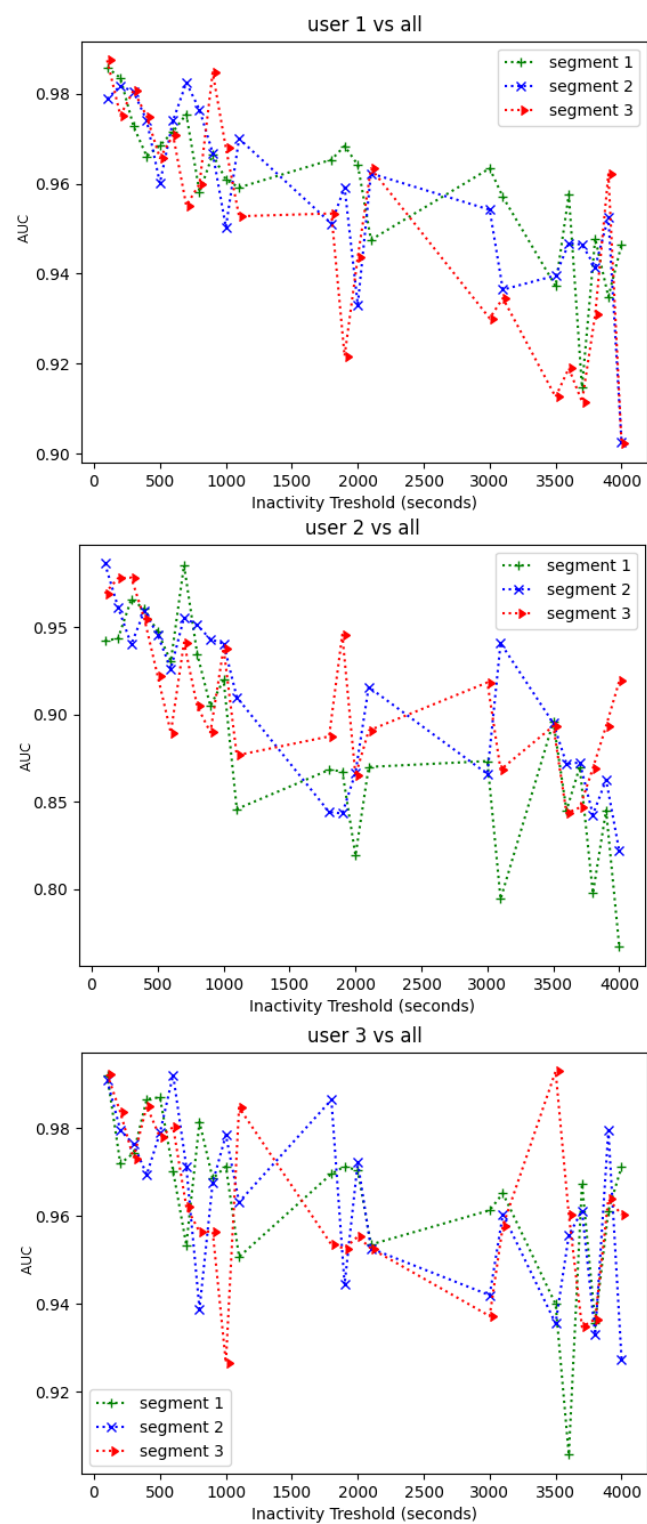


Fig. 4.11 Day segmentation and inactivity threshold tuning using our dataset

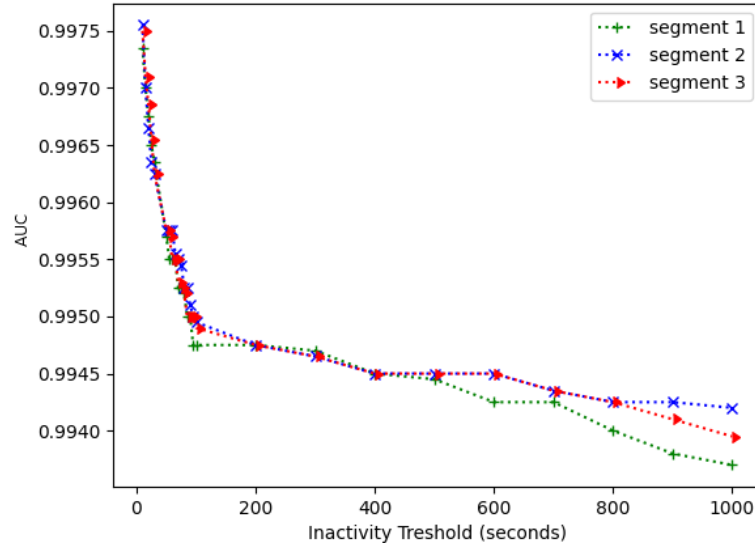


Fig. 4.12 Day segmentation and inactivity threshold tuning using DS2OS dataset

of False Negative Rate (FNR) to refer to such fraction since it is more meaningful. Moreover, we also measure the detection accuracy which denotes is the ratio of correct predictions (legitimate and anomalous) over all the analyzed commands. It should also be noted that the chosen Anomaly Threshold (often called Cut-off in ML terms) is 0 since the Anomaly Score (AS) produced by the iOCSVM varies in $[-1, 1]$.

Table 5.9 and 5.10 show the mean value of authentication metrics among the obtained 5-folds values using our adapted dataset and DS2OS dataset, respectively. In particular, we have chosen a small value of Inactivity Threshold equal to 400 seconds and 100 seconds for the former and the latter dataset, respectively.

We can see that the FAR successfully reaches the optimal value with 0.00% for our adapted dataset while reaching in the worst cases a value of 2.30% for the DS2OS dataset. Such a low rate makes sure that Duenna is efficiently not accepting any adversary thus ensuring a high-security level. On the other hand, the FRR reaches in the worst case a value of 4.84% and 3.23% for our adapted dataset and DS2OS dataset, respectively. Such a low rate ensures a high level of user convenience since a legitimate user is rarely prompted to re-authenticate.

Table 4.9 Authentication performance results using our adapted dataset (N/A means that metric could not be calculated for primary vs itself)

		Primary							
		User 1		User 2				User 3	
Adv.	Acc.	FAR	FRR	Acc.	FAR	FRR	Acc.	FAR	FRR
U1		N/A		1.0000	0.0000	0.0206	0.9720	0.0000	0.0206
U2	0.9831	0.0000	0.0376		N/A		1.0000	0.0000	0.0484
U3	1.0000	0.0000	0.0352	0.9912	0.0000	0.0252		N/A	

Table 4.10 Authentication performance results using DS2OS dataset

Attack	Accuracy	FAR	FRR
Denial of service	0.9776	0.0110	0.0074
Scan	0.9888	0.0000	0.0260
Malicious control	0.9826	0.0050	0.0010
Malicious operation	0.9960	0.0000	0.0104
Spying	0.9796	0.0026	0.0000
Data type probing	0.9941	0.0107	0.0007
Wrong setup	0.9852	0.0230	0.0323
Mean	0.9862	0.0074	0.0109

4.4.5 Comparison with similar works

Some other existing works have been also leveraging BAD-based security approach to secure IoT-based systems. For those specifically designed to protect SHS devices from unauthorized operation, we can find SoftAuthZ [56], [104], and [132] (cf. Section 3.4.2.2). Whereas; other works were generally designed for IoT-based systems viz., [97], [75], and [61].

Table 5.11 summarizes the AD techniques used by these works as well as the datasets and the performance metrics used for their evaluation. In particular, the previous version of Duenna [6], SoftAuthZ [56], [104], and [132], have only conducted simulation-driven experiments for performance evaluation. Whereas, [97], [75], and [61] have been evaluated using the real-world DS2OS dataset. In contrast, our proposed framework has been evaluated using both our real-world adapted dataset as well as DS2OS dataset.

Fig. 5.13 and Fig. 5.14 show the comparison of this use case with the presented similar frameworks by anomaly detection accuracy and FRR, respectively. In terms of accuracy,

Table 4.11 Comparison of use case 1 with similar works

Work	AD technique	Dataset used	Performance metrics
[6]	OCSVM	Own synthetic	FRR=0.0412
[56]	Linear regression		Accuracy=0.80
[104]	Association rules		Accuracy=0.92
[132]	Trees		Accuracy=0.79 FRR=0.1447
[97]	BIRCH Clustering	DS2OS	Accuracy=0.99 FRR=0.2
[75]	Random Neural Network (RaNN)		Accuracy=0.9920
[61]	Random Forest		Accuracy=0.994
This use case	iOCSVM	Our adapted dataset	Accuracy=0.9720 FRR=0.0484
		DS2OS	Accuracy=0.9744 FRR=0.0484

evaluated on our adapted dataset, this use case outperforms similar works that are evaluated on their own synthetic datasets but not those evaluated on DS2OS. However, applied to this latter dataset, this use case outperforms similar work evaluated on such dataset. In particular, in terms of False Rejection Rate (FRR), it outperforms other frameworks when evaluated on both our adapted dataset as well as on DS2OS dataset.

We explain this outperforming by the fact that Duenna uses a multitude of Behavioral Scores in contrast to SoftAuthZ [56] and [132] that only use a single user behavioral feature viz., variability between operation commands and operation sequence, respectively. Besides, incremental OCSVM adopted by Duenna gives higher performance than other AD techniques such as Linear Regression [56], the batch OCSVM [6], Clustering [97], and Random Forest [61].

4.4.6 Lockout threshold tuning

As discussed in the MTR-ICA-Action Manager, a user is logged out when his/her trust reaches a specific value called the Lockout Threshold. It is challenging to choose such a

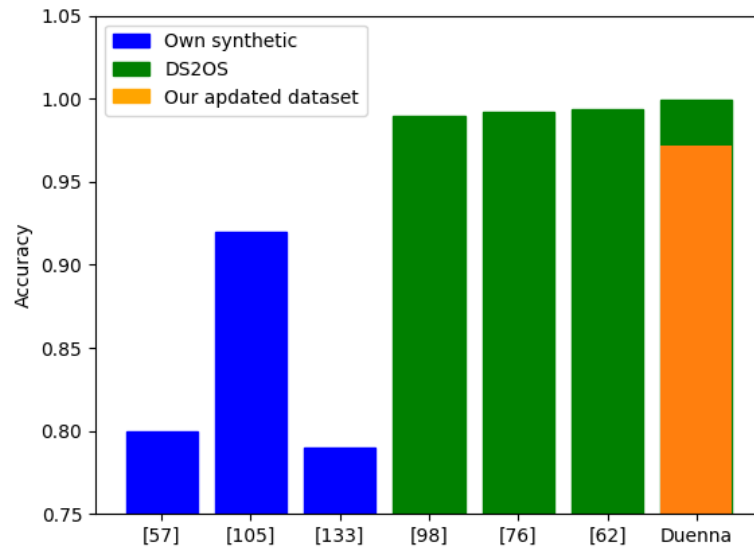


Fig. 4.13 Comparison of use case 1 with similar works by anomaly detection accuracy

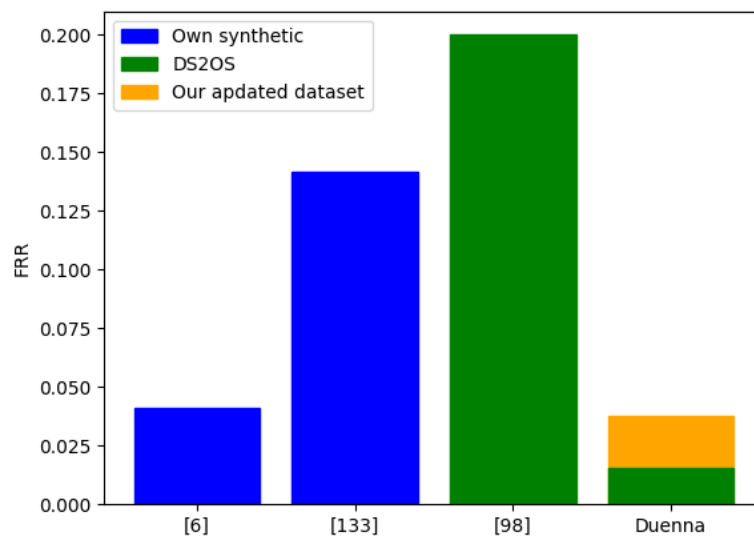


Fig. 4.14 Comparison of use case 1 with similar works by false rejection rate

threshold since a value that is near the trust upper bound (equal to 100 in Equation 4.2) is likely to lock the legitimate user very quickly. This choice makes iOCSVM very sensitive to FRR. On the other hand, a threshold that is distant from the reward upper bound is likely to let adversaries manipulating devices for too long before getting locked making iOCSVM models very sensitive to FAR. Consequently, it is important to choose the best threshold assuring that an impostor user can do as little as possible while not locking legitimate too quickly. For doing so, we measure two metrics (proposed by [87]) to follow the impact of different threshold values: 1) Average Number of Genuine Actions (ANGA) which is the average number of operation commands requested by the genuine user before locked by the system, and 2) Average Number of Impostor Actions (ANIA) which is the average number of operation commands requested by the adversarial user before locked by the system. The goal is obviously to have ANGA as high as possible, while at the same time the ANIA value must be as low as possible.

It should be noted that the parameters' values of the trust-based verification formula (Equation 4.1) have been chosen as follow:

- The parameter AT (Anomaly Threshold) is already chosen to 0 in the authentication performance evaluation (Section 5.4.4).
- For parameter B which is the value of AS in which the maximum value of penalty/reward is given, a value equal to 1 (i.e., the upper bound of AS) is chosen.
- For parameters C and D which are the upper bound value of the reward and penalty, respectively, a value equal to 1 is chosen.

Fig. 5.15 and Fig. 5.16 show the trend of ANGA and ANIA using different values of Lockout Threshold for our adapted dataset and DS2OS dataset, respectively. It should be noted that the values of ANGA and ANIA are scaled in the range of [0, 1] for good visualization. At the beginning of both figures, we can see that as long as the Lockout Threshold

increases, the ANIA value decreases while ANGA stays steady. This is a good indicator since our objective is to maximize ANGA and minimize ANIA as possible. However, ANGA begins to decrease starting from Lockout Threshold = 91, 91, 78 for our adapted dataset and starting from Lockout Threshold = 90 for DS2OS dataset. Since the decrease of AGNA is; we pick the minimum threshold among the three values for the first dataset (i.e., 78) and 90 for the second one. We consider the chosen values as the best thresholds that give a good combination of both ANGA and ANIA values.

Using the selected Lockout Threshold for our adapted dataset, Table 5.12 shows the mean value of AGNA and ANIA for all possible primary vs adversaries (NA means that ANGA could not be calculated as it only concerns the primary user against itself). Whereas, for DS2OS datasets, we have found that the mean value of AGNA and ANIA is 1245 and 11, respectively.

From the obtained results from both datasets, we can see that the big values of ANGA show that the legitimate user is never locked out, whereas, for ANIA we can see that an adversary can perform at most nine (9) actions before getting logged out. We recall that the Action Manager recovers devices to their initial states although these nine operation commands have been executed (cf. SHS Maintainability, Section 5.3.2.1).

Table 4.12 Obtained AGNA and ANIA results (∞ means a big value)

	Primary					
	User 1		User 2		User 3	
Adversary	AGNA	ANIA	AGNA	ANIA	AGNA	ANIA
User 1	∞	N/A	N/A	5.7592	N/A	9.0882
User 2	N/A	5.8848	∞	N/A	N/A	7.1860
User 3	N/A	5.8848	N/A	7.9744	∞	N/A

4.4.7 Evaluation of computational complexity

In this use case, we adopt the iOCSVM which can be updated incrementally. However, compared to regular OCSVM, this interesting feature needs more computational time and

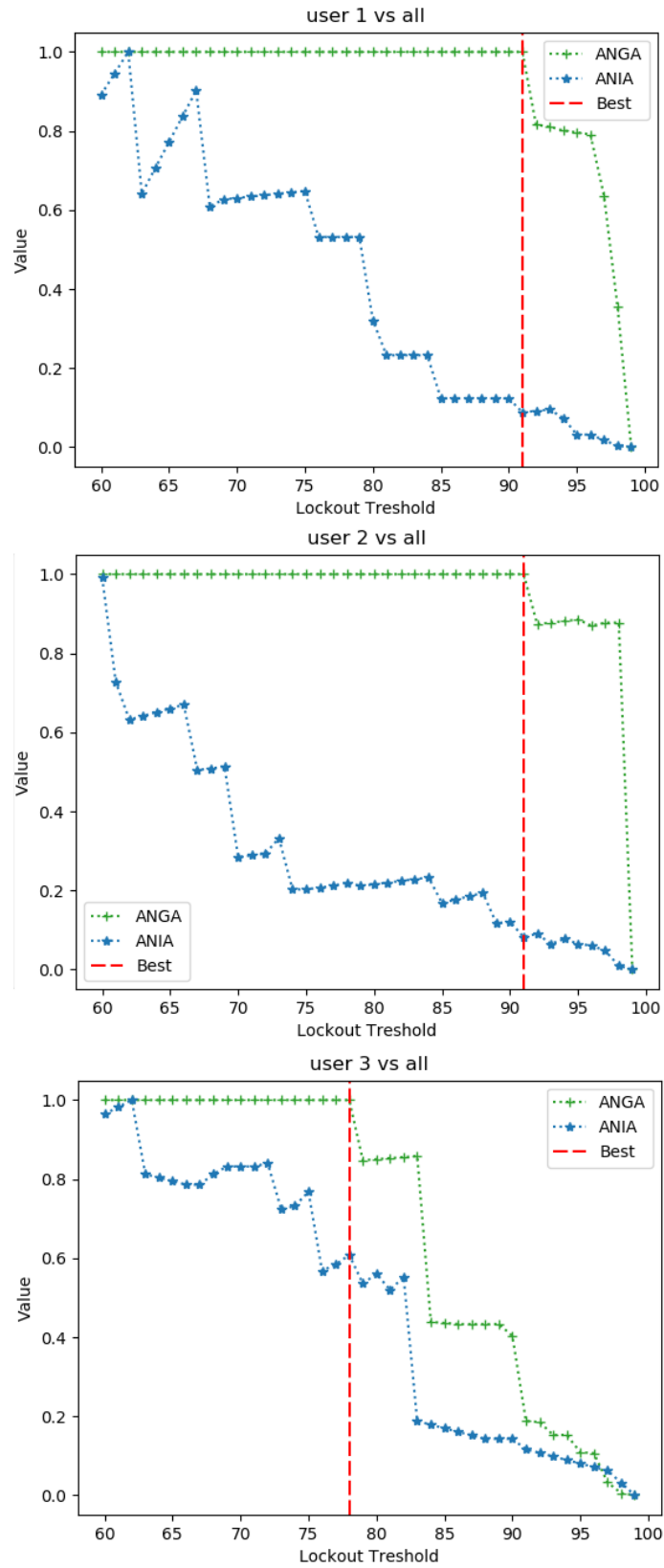


Fig. 4.15 Lockout threshold tuning using our dataset

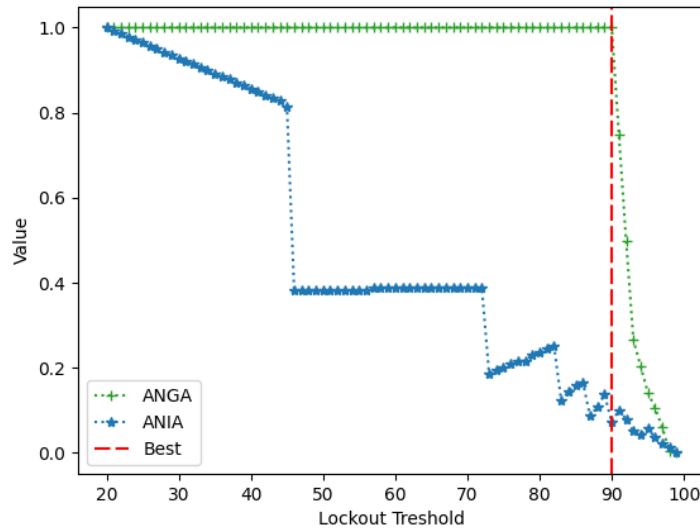


Fig. 4.16 Lockout threshold tuning using DS2OS dataset

resources. Fig. 5.17 shows the required training time for regular OCSVM and incremental OCSVM for different sizes of training data. We can see that iOCSVM exponentially needs more time to be trained when data size increases compared to the regular one.

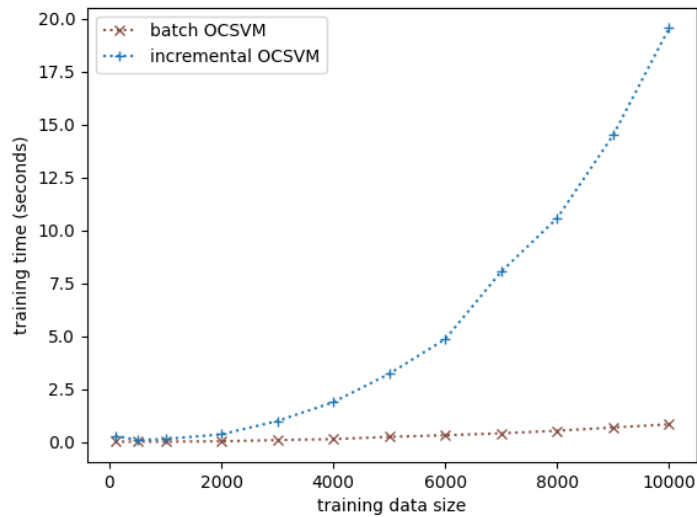


Fig. 4.17 Log of training time per model for different sizes of data

However, we argue that besides the advantage of updating iOCSVM without being re-trained from scratch in contrast to regular OCSVM, the added delay in training time does not pose a problem. On one hand, since the training of iOCSVM models in the first stage is

done offline before user operation commands are being analyzed, the time needed for the training does not affect the user experience and can be accepted. On the other hand, during the interactive operation, iOCSVM models are only updated with new executed commands (i.e., after the user session has finished). Using the optimal values of inactivity threshold and day segmentation, we have found that the maximum length of user sessions does not surpass 50 operation commands. As shown in Fig. 5.18, with such a few training instances, iOCSVM does not need a remarkable training time compared to the regular OCSVM.

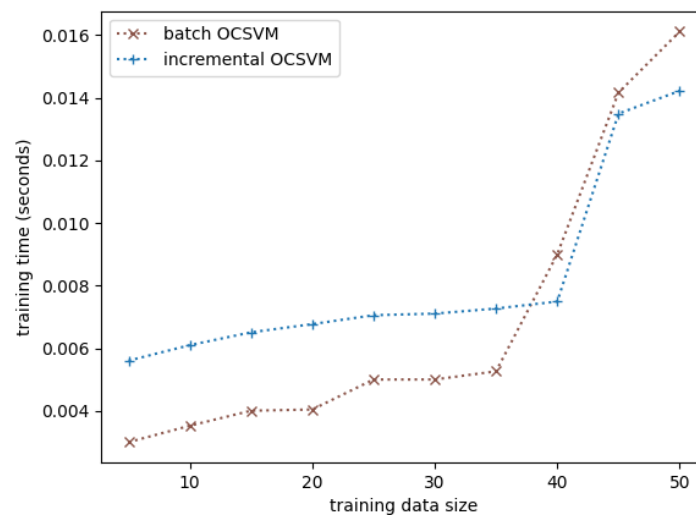


Fig. 4.18 Log of training per model for different length of user sessions

4.5 Conclusion

In this chapter, we have shown the feasibility of Duenna by addressing the use case of authenticating SHS users. In particular, Duenna was instantiated to make sure that only authorized users are allowed to operate SHS devices. The profiled and monitored targets in this use case are the SHS users which are continuously authenticated to prevent unauthorized user-issued operation commands. We have provided a detailed description of how Duenna's modules are instantiated and deployed in a user-driven SHS architecture.

Performance results obtained through an empirical evaluation conducted on real-world SHS datasets validate the ability of Duenna to differentiate between different SHS users. Specifically, Duenna exhibits high authentication rates in terms of False Acceptance Rate (FAR) and False Rejection Rate (FRR); thus, ensuring both security and user experience. These results also show that such a user behavior-based approach is a promising security scheme that could be integrated into existing user-driven SHSs.

Chapter 5

Use Case 2: Maintaining a Safe Appified Automated Smart Home System Operation

5.1 Introduction

5.1.1 System model and security goals

In the use case addressed in this chapter [8, 9], we consider an appified automated SHS (cf. Fig. 2.3) as the system model on which Duenna will be stated. The overall security objective of applying Duenna is to thwart security threats related to the automated operation and maintain a safe SHS against unexpected automation operation commands where the profiled and monitored target is the SmartApp.

By applying Duenna, we consider the following additional design, security, and safety features:

- Continuity: based on a set of behavioral scores that can be continuously calculated and assessed, the behavior of a SmartApp is evaluated during its entire lifecycle.

- Trust-based verification: a confidence score of each installed SmartApps is calculated and assessed to ban the ones showing a steady anomalous behavior.
- Maintainability: SHS would have a restore point when performed operation commands have been recognized as unwanted.

5.1.2 Duenna instantiation

Fig. 6.1 shows the distribution of Duenna's modules on an appified automated SHS architecture (cf. Fig. 2.3).

1. During the initial operation, the historical operation commands of the set of installed SmartApps are first collected. The collection of this data is performed at the cloud back-end via the log collector which intercepts operation commands issued by SmartApps towards Devices Handlers.
2. This data is then used to enroll the regular behavior patterns of installed SmartApps seen in these historical data. The enrollment process includes the construction of the probabilistic models and the training of incremental AD models on the Behavioral Scores calculated from the probabilistic models.
3. Later in an interactive operation mode, the trained baseline AD models are then used to continuously verify the legitimacy of the SmartApps-initiated operation commands and take security actions according to their anomaly analysis result.

5.2 Initial operation

Once Duenna is instantiated and different modules have been deployed, the initial operation is the first process Duenna has to perform to be ready for its interactive operation. The

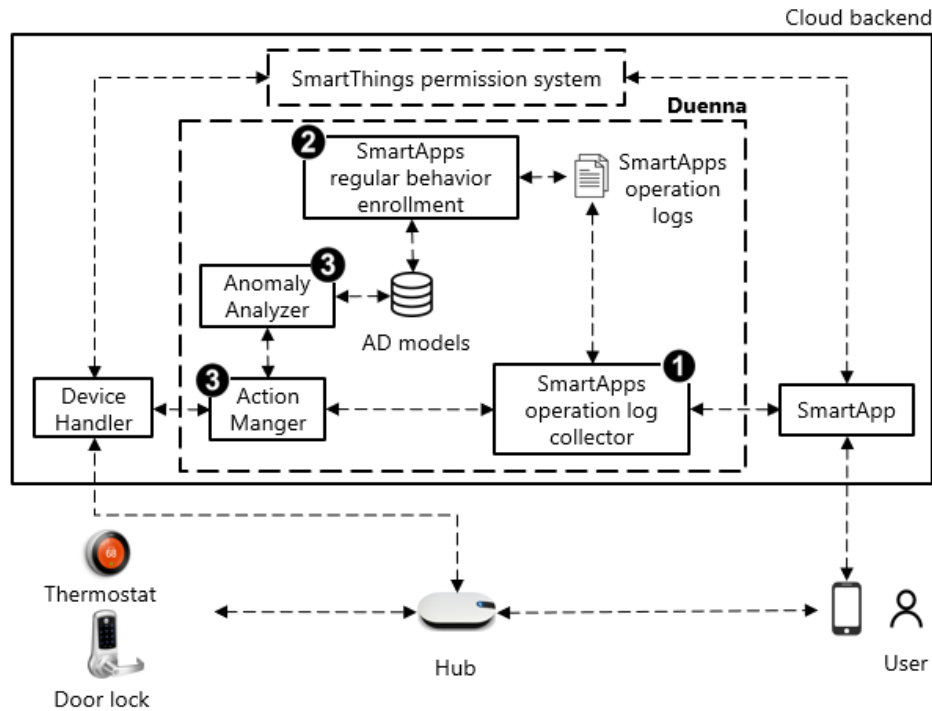


Fig. 5.1 Instantiation of Duenna on an appified automated SHS architecture

processes executed in this stage include SmartApps raw logs collection and SmartApps regular behavior enrollment as detailed in the following.

5.2.1 SmartApps raw logs collection

The first step towards the building of regular SmartApps behavioral patterns is the collection of their historical automation traces. The log collector situates at the cloud backend between the abstraction instances of SmartApps and Devices Handler. It intercepts the automation operation commands initiated by SmartApps to devices and sends them to be saved in the SmartApps operation log file.

The information extracted from an operation command and captured in the SmartApp log includes: SmartApp ID, triggering event, operated device, operation action, and timestamp.

5.2.2 SmartApps regular behavior enrollment

Once the SmartApps operation log is collected, the baseline AD models summarizing the behavioral patterns of each one of the installed SmartApps are trained and saved to be retrieved in the analysis stage. As described in Fig. 6.2, the enrollment process includes four sub-processes as explained in the following.

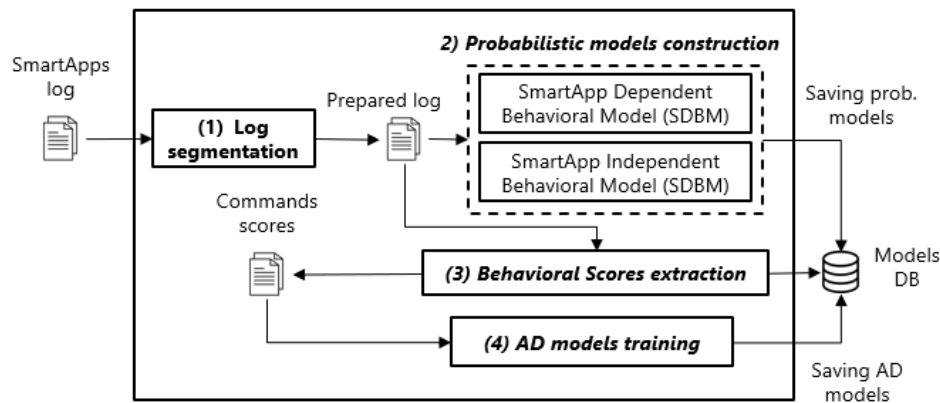


Fig. 5.2 Workflow of SmartApps regular behavior enrollment

5.2.2.1 SmartApps operation log segmentation

Before being used in the construction of probabilistic models and the training of AD models, the collected SmartApps operation log needs first to be prepared. Given that the behavior of an SHS inhabitant through the 24 hours of the day is generally segmented into a set of frequent periods wherein the user has some specific behavioral routines (e.g., waking up and going to work in the morning). And because the SmartApps behavior is related to the SHS owner behavior (i.e., triggering event occurs due to SHS inhabitant physical activities), the log segmentation consists of adding the corresponding time interval of the day (i.e., period) to each record in the SmartApps operation log. Segmentation allows the extraction of more information about the SmartApps patterns and simplifies the construction of probabilistic models in the next sub-process.

5.2.2.2 Probabilistic models construction

A probabilistic model is of a set of vectors and matrices containing different probabilities that describe the SmartApps behavioral patterns seen in the segmented SmartApps operation log. In particular, we distinguish two types of models. The SmartApps Dependent Behavioral Model (SADBM) contains the behavioral probabilities related to each SmartApp itself. Whereas, the SmartApps Independent Behavioral Model (SAIBM) contains the behavioral probabilities related to the set of occurred events and installed SmartApps. Table 6.1 describes the parameter of each model.

Table 5.1 SmartApps probabilistic models

Model	Parameter	Type	Description
SADBM	App Related Event	Vector	Probability values of the set of installed SmartApps to be triggered by related event(s).
	App Controlled Devices	Vector	Probability values of the set of installed SmartApps to control related device(s) given the occurrence of related event(s).
SAIBM	Apps Transitions	Matrix	Probability values of transition between every two SmartApps for all the set of installed SmartApps.
	Apps Transitions Latencies	Matrix	Time interval between the triggering between every two SmartApps for all the set of installed SmartApps.
	Events Transitions	Matrix	Probability values of transition between every two events for all the set of occurred events.
	Events Transitions Latencies	Matrix	Time interval between the occurrence of every two events for all the set of occurred events.

5.2.2.3 Behavioral scores extraction

Extracting SmartApps Behavioral Scores (BSs) consists of calculating a tuple of numeric values for each command seen in the SmartApps operation log using the constructed probabilistic models (i.e., SADBM and SAIBM) (cf. Fig. 6.2).

Hence, a trigger-action command issued by a SmartApps is described by the six following behavioral scores:

- Event occurrence: the probability of the SmartApps to be triggered by the occurred event.
- Device operation given event: the probability of the SmartApps to operate the given device while being triggered by the occurred event.
- SmartApp transition: the probability of the given SmartApp to be triggered after the previous one has been triggered.
- SmartApp transition latency: time interval between the triggering of the given SmartApp and the triggering of the previous one.
- Events transition: the probability of the SmartApp to be triggered by the occurred event after being triggered by the previous one.
- Events transition latencies: time interval between the occurrence of the given event and the previous one.

5.2.2.4 Training anomaly detection models

The final step in enrolling SmartApps regular behavior is to train AD models on the extracted BSs. In the use case addressed in this chapter, each installed SmartApp has its own BSs. Thus, one AD model is trained per SmartApp. The training of this set of AD models is performed on the cloud-backend. Then, the trained models are retrieved to be used in the next interactive stage.

5.3 Interactive operation

To prevent unwanted devices operation resulted from SmartApps behaving against their expected behaviors, the Action Manager (AM) intercepts the commands issued by SmartApps to analyze their legitimacy/anomaly via the Anomaly Analyzer (AA). Then, it takes security actions according to the anomaly analysis result.

As explained in the following, AM may either operate in a basic mode or in an enhanced mode to maintain a safe SHS operation. Whereas, the workflow of AA is the same in both modes. The two modes are dependent on the assumption put on the maliciousness of a misbehaved/adversarial SmartApp.

5.3.1 Anomaly Analyzer

As shown in Fig. 6.3, upon receiving an operation command from the AM, the anomaly analysis sub-process is triggered.

1. AA first extracts the intercepted command's real-time parameters.
2. AA calculates the Behavioral Scores (BSs) from different parameters of the command (e.g., event, operated device, etc.).
3. AA retrieves the trained AD model and applies it to the calculated scores. The application of the AD model produces an Anomaly Score (AS) that varies in the range of $[-1,1]$. AA updates the SmartApps trust (T) using the resulted AS.
4. If AM operates in basic mode, AA sends only AS and BSs back to the AM. However, if this latter operates in enhanced mode AA sends all three parameters i.e., AS, BSs, and T back to AM.

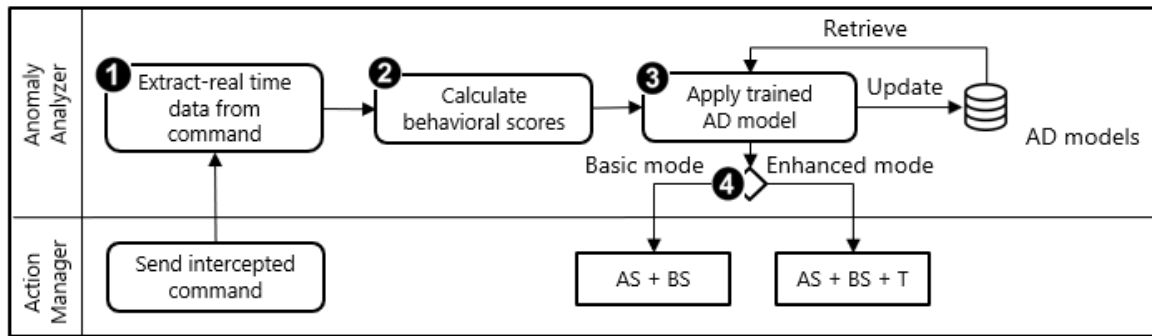


Fig. 5.3 Workflow of Anomaly Analyzer

5.3.2 Continuous Safety (C-Safety)

Fig. 6.4 depicts the process of the Continuous Safety Action Manager (C-Safety-AM).

1. Once a command is issued by a SmartApp, the C-Safety-AM sends it to the AA which sends back three parameters i.e., AS and BSs, as explained before.
2. After that, the C-Safety-AM starts by testing the legitimacy of the issued command from the obtained AS.
3. In particular, if the AS is above a predefined Anomaly Threshold (AT), the C-Safety-AM sends the command to the device handler to be executed.
4. However, if the AS is below the AT, the C-Safety-AM prompts the user to confirm whether the command is suspicious or not. The user's prompt message provides the command's contextual information such as the occurred event and the action on the device; that help the user in his/her decision [140].
5. If the user confirms that the command is suspicious, the C-Safety-AM suggests the SmartApp removal to the user.

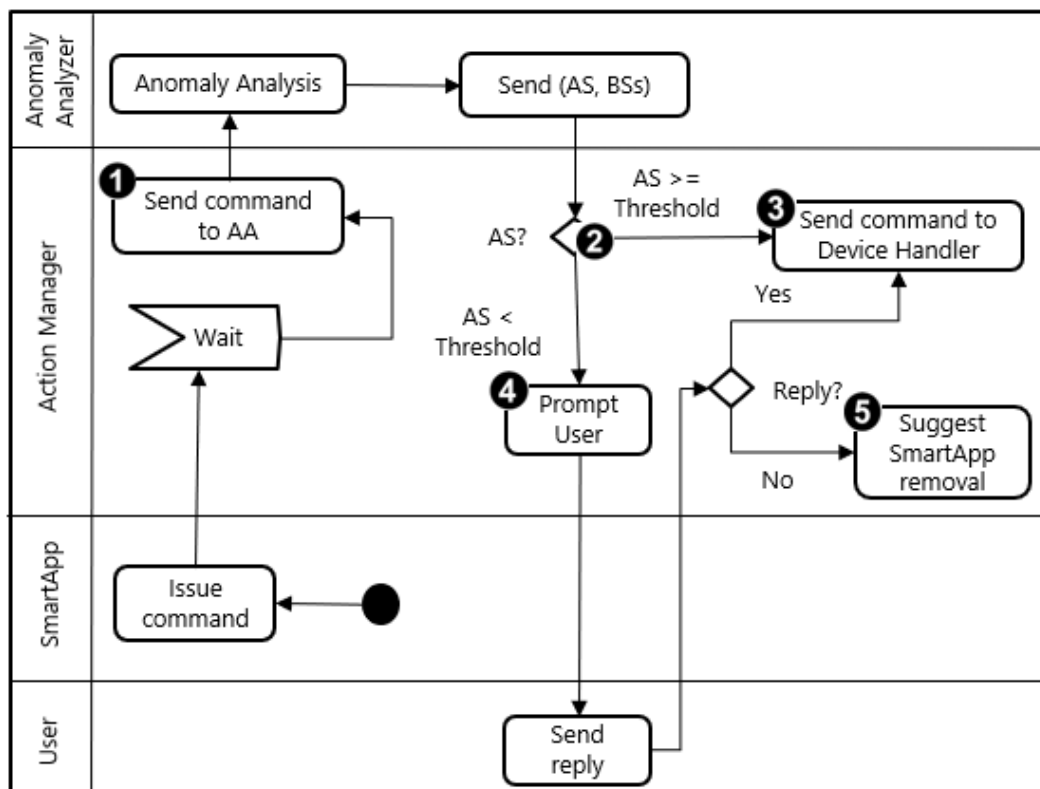


Fig. 5.4 Workflow of Continuous-Safety-Action Manager (C-Safety-AM)

5.3.3 Continuous, Trust-based, and Maintainable Safety (CTM-Safety)

Previously in C-safety-AM, the user has been prompted at each time a SmartApp initiates an operation command recognized as anomalous. This setting makes AD models very sensitive to false alarms; thus, False Rejection Rate (FRR) should be as low as possible so the user is not incorrectly prompted. To remedy this issue, we leverage the feature of remembering previous decisions recommended by Duenna.

To do so, CTM-Safety-AM maintains an authorized permission-context mapping table for each SmartApp. As depicted in Fig. 6.5:

1. When a SmartApp initiates a command analyzed as anomalous, CTM-Safety-AM checks whether the command with the given context has been previously allowed or denied.
2. If not, CTM-Safety-AM prompts the user with a dialog presenting the permission request and the associated context.
3. CTM-Safety-AM adds an entry to the mapping table to store the user's decision (i.e., permission/rejection) as security preferences. The context structure includes SmartApp automation purpose, triggering event, etc.
4. If an entry for the given context has been found, CTM-Safety-AM proceeds to the trust-based verification if the context is allowed.
5. Otherwise, CTM-Safety-AM suggests SmartApps removal if it is denied.

Trust-based verification If the user allows an anomalous-recognized command to be executed, trust-based verification is employed. In particular, the CTM-Safety-AM updates a confidence score for each requested command to evaluate the trust towards the SmartApp. To do so, we leverage the trust-based verification feature provided by Duenna. Using the

recommended formula (c.f., Equation 4.1), the CTM-Safety-AM updates the trust from the obtained AS and sends it to the CTM-Safety-AM. Then, CTM-Safety-AM compares this trust value to the predefined AT. If $T \geq AT$, CTM-Safety-AM sends the command to Device Handler to be executed. Otherwise i.e., $T < AT$, CTM-Safety-AM suggests SmartApp removal.

Maintainability To prevent dangerous consequences of false executed commands, the Maintainability feature recommended by Duenna is employed. In particular, once the SmartApps fail in the trust-based verification and its removal is suggested to the user, SHS devices are restored to their initial states. To save initial devices' states, when SmartApps initiates its first operation command, CTM-Safety-AM requests each device handler for the current state of the attached physical device and saved them locally.

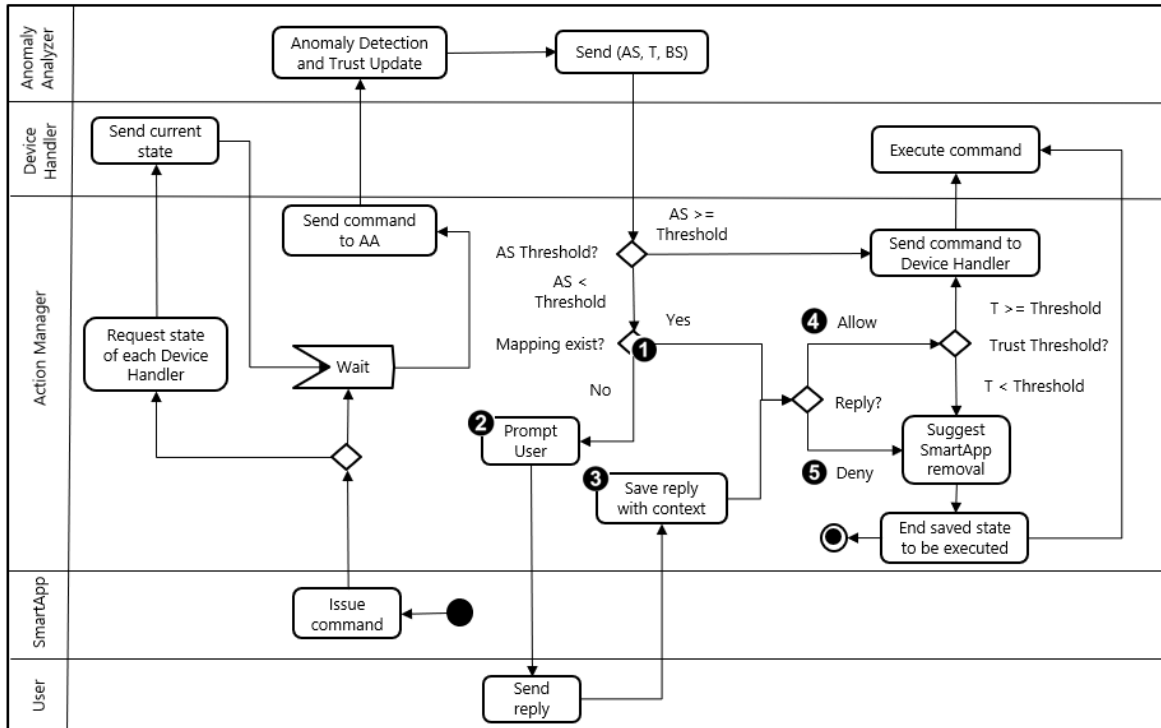


Fig. 5.5 Workflow of CTM-Safety-Action Manager (CTM-Safety-AM)

5.4 Empirical performance evaluation

In this section, we conduct an empirical evaluation to show the feasibility of applying Duenna for maintaining a safe appified automated SHS operation. The choice of this type of validation instead of others is motivated by the same reasons as the first use case (cf. Section 5.4).

In this regard, we address the following research questions:

- RQ1: What is the best segmentation of day (i.e., periods of the day) to get a better authentication performance? (Section 6.4.3).
- RQ2: Using the optimal day segmentation, what is the anomaly detection performance of this Duenna’s use case in different smart home layouts and devices? (Section 6.4.4).

5.4.1 Evaluation dataset

As described in the enrollment stage (Section 6.2.2), the data used to construct probabilistic models and then train AD models is the logging traces of SmartApps when they are operating SHS following the trigger-action model. To remedy such data in public repositories, we propose to follow the same proposition presented in the empirical evaluation of the first use case. Again, we propose to use the history data of the manual operation of appliances and objects by inhabitants in real-world home environments. We assume that the devices operated by inhabitants are operated by the automation SmartApps.

The data we will be using for this purpose is the one collected in the MIT House Consortium [117]. For two weeks, sensors were installed in everyday objects such as drawers and refrigerators to record opening-closing events in two single-person apartments as the two subjects carrying out everyday activities. The recorded inhabitant activities (e.g., eating) are grouped into a set of categories (e.g., personal needs).

Since SHS devices’ automated operation is based on the trigger-action model, we came up with the idea to assume that the SmartApps IDs are the activities categories, the triggering events are the activities themselves, whereas, the operated devices with the particular operation actions and timestamps are extracted as they are. Thus, the obtained SmartApps operation log includes necessary information as needed viz., SmartApp ID, event, device, action, and timestamp. The description of the obtained operation logs for both subjects is given in Table 6.2.

Table 5.2 Description of evaluation dataset of use case 2

	Subject 1	Subject 2
# of installed apps	6	6
# of controlled devices	75	70
# of control commands	5977	4479

5.4.2 Selection of anomaly detection model and evaluation methodology

In the evaluation of the use case addressed in this chapter, we use the same iOCSVM AD model following the same motivations discussed in Section 5.4.2. Our evaluation dataset contains the automated operation history of two SHSs, where six SmartApps are installed in each. Consequently, six SmartApp baselines (i.e., probabilistic models, operation commands Behavioral Scores, and iOCSVM models) are built for each SHS over the extracted operation logs of each SmartApp. To evaluate the ability of the Anomaly Analyzer in discriminating misbehaved SmartApps from regular ones, we follow a primary-vs-adversary strategy. In particular, we first search for common installed SmartApps from the two SHSs. Then, we evaluate each SmartApp OCSVM model of the chosen primary's SHS against the behavioral scores of the corresponding SmartApp from the adversary's SHS, and vice versa.

5.4.3 Day segmentation tuning

During the enrollment stage, SmartApps operation logs are segmented before being used in the extraction of Behavioral Scores (Section 6.2.2.1). In particular, the corresponding period of the day is added to each record in the log. However, choosing the best day division is also challenging since training iOCSVM models on different behavioral scores would give different anomaly detection results.

To find the day segmentation candidate that better fits different history logs of the installed SmartApps, we follow the same method proposed in the evaluation of the first use case addressed in Chapter 5. In particular, different candidates are first applied to

calculate Behavioral Scores. Then, the trained iOCSVM on the set of scores is evaluated using Area Under the ROC Curve. Finally, the optimal candidate is selected for training the final iOCSVM models to be evaluated. In the evaluation of this use case, we also use day segmentation candidates used in the first use case (cf. Table 5.7).

Fig. 6.6 shows the AUC trend with different day segmentation and inactivity threshold candidates for all common-installed SmartApps for both testing scenarios. As one can see, manipulating different day segmentation and Inactivity Threshold candidates gives different AUC values. However, we can notice that the AUC value certainly gets improved when using candidate 3 for day segmentation. In particular, the mean value of AUC among common-installed SmartApps reaches up to 0.72 and 0.74 for subject 1 vs subject 2, and subject 2 vs subject 1, respectively. Consequently, we may conclude that that dividing the day into four periods is the optimal segmentation of SmartApps logs.

5.4.4 Anomaly detection performance

In our adapted evaluation dataset, four common SmartApps have been found among the two SHSs. As discussed in the empirical evaluation of the use case addressed in Chapter 5, we also use the FAR and FRR as the evaluation metrics. In particular, to evaluate the ability of Anomaly Analyzer in identifying malicious operation commands without incorrectly rejecting the legitimate ones, we measure the FAR by calculating the fraction of testing SmartApps-issued operation commands that have been incorrectly accepted. On the other hand, to measure the user convenience level, we calculate the fraction of benign operation commands that have been incorrectly rejected i.e., FRR. Moreover, we also measure the True Positive Rate (TPR), also called Sensitivity or Recall, which denotes the correctly recognized anomalous commands over all the anomalous ones.

Again, the Action Manager (AM) uses an Anomaly Threshold (AT) to discriminate the legitimacy/anomaly of an operation command from the produced Anomaly Score (AS) that

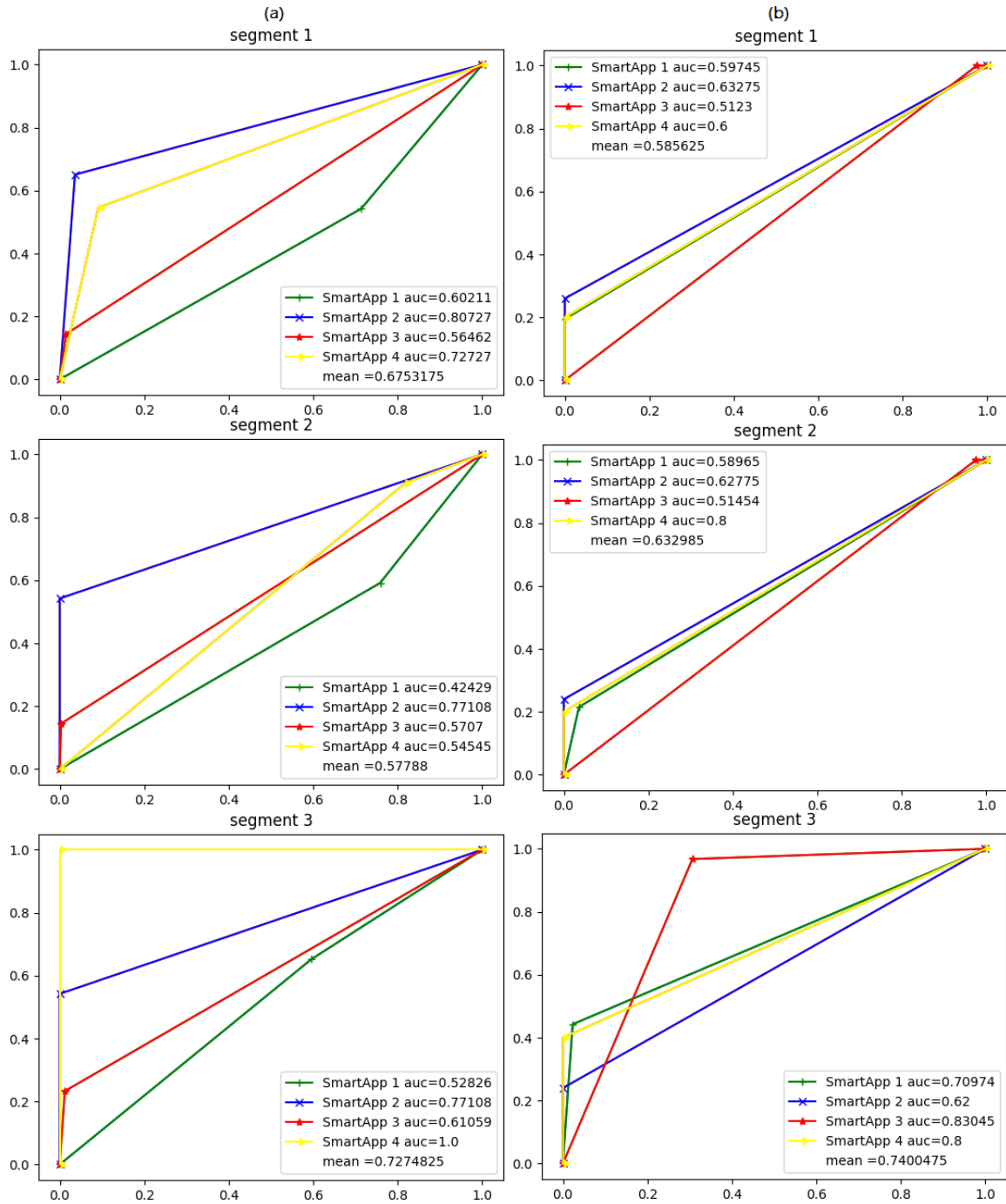


Fig. 5.6 Obtained ROC curves using different day segmentation candidates

varies in the range of $[-1,1]$. It should be noted that the chosen value for AT is 0. If a SmartApp initiates a malicious operation command for the first time, the user is prompted by the AM to confirm the legitimacy/anomaly of this command. However, prompting the user too often cannot satisfy the real-time SHS automation (e.g., users need to be awake to respond). This case implies that the frequency of prompts must be reduced as low as possible and our proposed framework should be more user-friendly than secure. To guarantee such a feature, the FRR should be reduced and prioritized over the FAR, hence the AT value must be chosen as low near to the negative end of the AS range.

By choosing such an AT value, Tables 6.3 and 6.4 give the obtained values for the TPR, FAR, and FRR rates for both testing scenarios (i.e., subject 1 vs subject 2 and the inverse) and all common-installed SmartApps. We can see that the FRR reaches in the worst cases a value of 3.3%. This low rate ensures that benign operation commands are rarely rejected. Hence, a better experience is provided for the user since he/she is not falsely prompted very often. On the other hand, we can see that the FAR reach 7.22% in the worst cases. Such an acceptable rate ensures that malicious operation commands are rarely accepted. Hence, the removal suggestion of malicious SmartApps to the user happens as quickly as possible and these SmartApps were not able to issue many operation commands.

Table 5.3 Obtained results of TPR, FAR, and FRR for subject 1 vs subject 2

	Subj1											
	SA1			SA2			SA3			SA4		
Subj2	TPR	FAR	FRR	TPR	FAR	FRR	TPR	FAR	FRR	TPR	FAR	FRR
SA1	0.992	0.072	0.018									
SA2				0.988	0.072	0.025						
SA3							0.990	0.037	0.010			
SA4										0.985	0.0394	0.022

As discussed in Section 3.4.3, in contrast to the first use case, only HoMonit has leveraged the BAD-based approach to secure the automated operation of SHS based on the analysis of SmartApps behaviors [138]. Authors have evaluated their proposed framework on their own dataset which is not publicly available. Yet, we compare their obtained results to ours for

Table 5.4 Obtained results of TPR, FAR, and FRR for subject 2 vs subject 1

Subj1	Subj2											
	SA1			SA2			SA3			SA4		
	TPR	FAR	FRR	TPR	FAR	FRR	TPR	FAR	FRR	TPR	FAR	FRR
SA1	0.998	0.021	0.013									
SA2				0.987	0.052	0.021						
SA3							0.997	0.047	0.016			
SA4										0.998	0.027	0.033

the sake of information. In particular, the average of TPR they obtained is 0.985, whereas, for our use case, the average of the obtained TPR between both testing scenarios is 0.991. These results show that this use case outperforms HoMonit in the ability to detect anomalous operation commands issued by misbehaved SmartApps.

5.5 Conclusion

In this chapter, we have shown the feasibility of Duenna by addressing the use case of appified automated SHS operation. In particular, Duenna was instantiated to maintain a safe SHS automation. The profiled and monitored targets are the automation apps (called SmartApps) which are continuously assessed to prevent unexpected SmartApps-issued operation commands. We have provided a detailed description of how Duenna's modules are instantiated and deployed in an appified automated SHS architecture.

Performance results obtained through an empirical evaluation conducted on real-world SHS datasets validate the ability of Duenna to detect unexpected automation resulted from misbehaved SmartApps. Specifically, Duenna exhibits high authentication rates in terms of False Acceptance Rate (FAR) and False Rejection Rate (FRR); thus, ensuring both security and user experience. These results also show that such a SmartApp behavior-based approach is a promising security scheme that could be integrated into existing automation SHSs.

Conclusion

Summary of contributions

Behavioral Anomaly Detection (BAD)-based security is an ongoing research topic. However, the design of secure and safe operation of Internet of Things-based Smart Home Systems (SHSs) using such an approach is still not well-addressed. This dissertation addresses this research problem motivated by the research questions (RQs) presented in the first chapter. Our contribution is three-fold and can be summarized in the following key findings.

We consider the **design of a conceptual security framework -called Duenna-** as the missing piece and the first step towards addressing the lack of relevant BAD-based techniques to secure SHSs operation. This claim was realized through two use cases.

In the first, we have **investigated the building of behavioral patterns of SHS users to prevent unauthorized operation.** Through empirical evaluation, such an investigation has been successfully validated as a promising solution to be integrated as an additional security scheme into existing and future SHSs.

In the second, we have **investigated the building of behavioral patterns of automation applications to maintain a safe SHS operation.** Through empirical evaluation, such an investigation has been successfully validated as a promising security scheme to be integrated into existing and future automation SHSs.

Through applying Duenna, we have successfully addressed common BAD-based security challenges, namely, behavior drift and false alarms. For the former, a self-adaptation tech-

nique is proposed, whereas for the latter, target re-authentication and prompt techniques are proposed to tolerate the false rejection.

Future work

The contributions presented in this dissertation open several perspectives for future work. In particular, future researches can be investigated either at the level of framework design or in exploring other use cases.

Duenna design

Multi-user SHSs Duenna can currently be leveraged to secure single-person SHSs. The reason is that the target behavioral scores are personalized to one inhabitant's home environment configuration. Future works may consider the multi-person SHSs and extend Duenna to support such a scenario.

Privacy Recent studies have demonstrated that users are not comfortable with biometric data collection by their IoT-based systems [95]. Future work may thus consider privacy-preserving techniques during the whole operation process of Duenna i.e., data collection, enrollment, analysis, etc.

Use cases

Hybrid-operated SHS Future work may consider leveraging Duenna to secure the operation of SHSs that are both user-driven and appified automated at the same time. In such a use case, the collection of data, enrollment, the analysis should be well adapted.

Other anomaly detection models In the use cases addressed in this dissertation, iOCSVM has been chosen as the AD model to be trained and evaluated. Future works may investigate the usage of other AD techniques (cf. Section 3.2.3).

Investigating other IoT-based systems Duenna provides an open conceptual framework for other use cases either to secure SHSs or for other IoT-based systems and applications such as smart cities and self-driving vehicles.

References

- [1] Abdalla, M., Fouque, P.-A., and Pointcheval, D. (2005). Password-based authenticated key exchange in the three-party setting. In *International Workshop on Public Key Cryptography*, pages 65–84. Springer.
- [2] Aggarwal, C. C. (2005). On abnormality detection in spuriously populated data streams. In *Proceedings of the 2005 siam international conference on data mining*, pages 80–91. SIAM.
- [3] Aggarwal, C. C. (2017). Outlier detection in graphs and networks. In *Outlier analysis*, pages 369–397. Springer.
- [4] Aho, A. V., Sethi, R., and Ullman, J. D. (1986). Compilers, principles, techniques. *Addison wesley*, 7(8):9.
- [5] Alrumayh, A. S., Lehman, S. M., and Tan, C. C. (2020). Context aware access control for home voice assistant in multi-occupant homes. *Pervasive and mobile computing*, 67:101196.
- [6] Amraoui, N., Besrou, A., Ksantini, R., and Zouari, B. (2019). Implicit and continuous authentication of smart home users. In *International Conference on Advanced Information Networking and Applications*, pages 1228–1239. Springer.
- [7] Amraoui, N., Besrou, A., Ksantini, R., and Zouari, B. (2020). Securing smart homes using a behavior analysis based authentication approach. In *2020 IEEE Eighth International Conference on Communications and Networking (ComNet)*, pages 1–5. IEEE.
- [8] Amraoui, N. and Zouari, B. (2020). Behavior-based anomaly detection for securing smart home systems automation. In *2020 International Conference on Software, Telecommunications and Computer Networks (SoftCOM)*, pages 1–6. IEEE.
- [9] Amraoui, N. and Zouari, B. (2021). An ml behavior-based security control for smart home systems. In *Risks and Security of Internet and Systems: 15th International Conference, CRiSIS 2020, Paris, France, November 4–6, 2020, Revised Selected Papers 15*, pages 117–130. Springer International Publishing.
- [10] Angiulli, F. and Pizzuti, C. (2002). Fast outlier detection in high dimensional spaces. In *European conference on principles of data mining and knowledge discovery*, pages 15–27. Springer.
- [11] Apple (2021). Your home at your command. <https://www.apple.com/ios/home/>. [Online; accessed 07-June-2021].

- [12] Arfaoui, A., Cherkaoui, S., Kribeche, A., and Senouci, S. M. (2019). Context-aware adaptive remote access for iot applications. *IEEE Internet of Things Journal*, 7(1):786–799.
- [13] Arias-Cabarcos, P., Almenarez, F., Trapero, R., Diaz-Sanchez, D., and Marin, A. (2015). Blended identity: Pervasive idm for continuous authentication. *IEEE Security & Privacy*, 13(3):32–39.
- [14] Armando, A., Basin, D., Boichut, Y., Chevalier, Y., Compagna, L., Cuéllar, J., Drielsma, P. H., Héam, P.-C., Kouchnarenko, O., Mantovani, J., et al. (2005). The avispa tool for the automated validation of internet security protocols and applications. In *International conference on computer aided verification*, pages 281–285. Springer.
- [15] Ashton, K. et al. (2009). That ‘internet of things’ thing. *RFID journal*, 22(7):97–114.
- [16] Atoum, I., Otoom, A., and Ali, A. A. (2014). A holistic cyber security implementation framework. *Information Management & Computer Security*.
- [17] Atzori, L., Iera, A., and Morabito, G. (2010). The internet of things: A survey. *Computer networks*, 54(15):2787–2805.
- [18] Augusteijn, M. and Folkert, B. (2002). Neural network classification and novelty detection. *International Journal of Remote Sensing*, 23(14):2891–2902.
- [19] Azaria, A., Richardson, A., Kraus, S., and Subrahmanian, V. (2014). Behavioral analysis of insider threat: A survey and bootstrapped prediction in imbalanced data. *IEEE Transactions on Computational Social Systems*, 1(2):135–155.
- [20] Bellman, C. and Van Oorschot, P. C. (2019). Analysis, implications, and challenges of an evolving consumer iot security landscape. In *2019 17th International Conference on Privacy, Security and Trust (PST)*, pages 1–7. IEEE.
- [21] Bergman, L. and Hoshen, Y. (2020). Classification-based anomaly detection for general data. *arXiv preprint arXiv:2005.02359*.
- [22] Bhide, V. H. and Wagh, S. (2015). i-learning iot: An intelligent self learning system for home automation using iot. In *2015 International Conference on Communications and Signal Processing (ICCSP)*, pages 1763–1767. IEEE.
- [23] Birnbach, S. and Eberz, S. (2019). Peeves: Physical event verification in smart homes.
- [24] Bishop, C. M. (1994). Novelty detection and neural network validation. *IEE Proceedings-Vision, Image and Signal processing*, 141(4):217–222.
- [25] Bisong, E. (2019). Batch vs. online learning. In *Building Machine Learning and Deep Learning Models on Google Cloud Platform*, pages 199–201. Springer.
- [26] Boser, B. E., Guyon, I. M., and Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the fifth annual workshop on Computational learning theory*, pages 144–152.

-
- [27] Brad, R. (2014). The ultimate nightmare: Researchers learn how to hack connected ‘smart home’ toilets. <https://bgr.com/2014/06/12/smart-home-toilets-hacked/>. [Online; accessed 07-June-2021].
 - [28] Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. (2000). Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 93–104.
 - [29] Brian, B. (2011). Sustainability hacks: Automatic window control. <https://hackaday.com/2011/09/29/sustainability-hacks-automatic-window-control/>. [Online; accessed 07-June-2021].
 - [30] Celik, Z. B., Fernandes, E., Pauley, E., Tan, G., and McDaniel, P. (2019a). Program analysis of commodity iot applications for security and privacy: Challenges and opportunities. *ACM Computing Surveys (CSUR)*, 52(4):1–30.
 - [31] Celik, Z. B., McDaniel, P., and Tan, G. (2018). Soteria: Automated iot safety and security analysis. In *2018 {USENIX} Annual Technical Conference ({USENIX}{ATC} 18)*, pages 147–158.
 - [32] Celik, Z. B., Tan, G., and McDaniel, P. D. (2019b). Iotguard: Dynamic enforcement of security and safety policy in commodity iot. In *NDSS*.
 - [33] Chandola, V., Banerjee, A., and Kumar, V. (2009). Anomaly detection: A survey. *ACM computing surveys (CSUR)*, 41(3):1–58.
 - [34] Chandola, V., Banerjee, A., and Kumar, V. (2010). Anomaly detection for discrete sequences: A survey. *IEEE transactions on knowledge and data engineering*, 24(5):823–839.
 - [35] CHEN, F., LUO, D., XIANG, T., CHEN, P., FAN, J., and TRUONG, H.-L. (2021). Iot cloud security review: A case study approach using emerging consumer-oriented applications.
 - [36] Chen, Z., Tian, L., and Lin, C. (2018). Trust evaluation model of cloud user based on behavior data. *International Journal of Distributed Sensor Networks*, 14(5):1550147718776924.
 - [37] Chi, H., Zeng, Q., Du, X., and Yu, J. (2020). Cross-app interference threats in smart homes: Categorization, detection and handling. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 411–423. IEEE.
 - [38] Cook, D. J., Youngblood, M., Heierman, E. O., Gopalratnam, K., Rao, S., Litvin, A., and Khawaja, F. (2003). Mavhome: An agent-based smart home. In *Proceedings of the First IEEE International Conference on Pervasive Computing and Communications, 2003.(PerCom 2003)*, pages 521–524. IEEE.
 - [39] Cranor, L. F. (2008). A framework for reasoning about the human in the loop.

- [40] Darlene, S. (2016). Researchers hack Philips Hue lights via a drone; IoT worm could cause city blackout. <https://www.computerworld.com/article/3139860/researchers-hack-philips-hue-lights-via-a-drone-iot-worm-could-cause-city-blackout.html>. [Online; accessed 07-June-2021].
- [41] De Carli, L. and Mignano, A. (2021). Network security for home iot devices must involve the user: a position paper.
- [42] De Stefano, C., Sansone, C., and Vento, M. (2000). To reject or not to reject: that is the question-an answer in case of neural classifiers. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 30(1):84–94.
- [43] Dmitry, D., Elena, P., Anna, C., Tatiana, Z., and Elena, P. (2020). Approaches to anomaly detection in web application intrusion detection systems. In *2020 Ural Symposium on Biomedical Engineering, Radioelectronics and Information Technology (US-BEREIT)*, pages 532–535. IEEE.
- [44] Domingues, R., Filippone, M., Michiardi, P., and Zouaoui, J. (2018). A comparative evaluation of outlier detection algorithms: Experiments and analyses. *Pattern Recognition*, 74:406–421.
- [45] Elrawy, M. F., Awad, A. I., and Hamed, H. F. (2018). Intrusion detection systems for iot-based smart environments: a survey. *Journal of Cloud Computing*, 7(1):1–20.
- [46] Essop, I., Ribeiro, J. C., Papaioannou, M., Zachos, G., Mantas, G., and Rodriguez, J. (2021). Generating datasets for anomaly-based intrusion detection systems in iot and industrial iot networks. *Sensors*, 21(4):1528.
- [47] Ester, M., Kriegel, H.-P., Sander, J., Xu, X., et al. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Kdd*, volume 96, pages 226–231.
- [48] Faily, S. (2011). *A framework of usable and secure system design*. PhD thesis, Oxford University, UK.
- [49] Fawcett, T. and Provost, F. (1999). Activity monitoring: Noticing interesting changes in behavior. In *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 53–62.
- [50] Fernandes, E., Jung, J., and Prakash, A. (2016). Security analysis of emerging smart home applications. In *2016 IEEE symposium on security and privacy (SP)*, pages 636–654. IEEE.
- [51] Fernandes, E., Rahmati, A., and Feamster, N. (2019). New problems and solutions in iot security and privacy. *arXiv preprint arXiv:1910.03686*.
- [52] Francois-Xavier, A. (2018). IoT traffic traces gathered in a the DS2OS IoT environment. <https://www.kaggle.com/francoisxa/ds2ostraffictraces>. [Online; accessed 07-June-2021].
- [53] Fu, C., Zeng, Q., and Du, X. (2020). Hawatcher: Semantics-aware anomaly detection for appified smart homes. ??

-
- [54] Gamundani, A. M., Phillips, A., and Muyingi, H. N. (2018). An overview of potential authentication threats and attacks on internet of things (iot): A focus on smart home applications. In *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, pages 50–57. IEEE.
 - [55] Garcia-Font, V., Garrigues, C., and Rifà-Pous, H. (2016). A comparative study of anomaly detection techniques for smart city wireless sensor networks. *sensors*, 16(6):868.
 - [56] Ghosh, N., Chandra, S., Sachidananda, V., and Elovici, Y. (2019). Softauthz: A context-aware, behavior-based authorization framework for home iot. *IEEE Internet of Things Journal*, 6(6):10773–10785.
 - [57] Goldstein, M. and Uchida, S. (2016). A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data. *PloS one*, 11(4):e0152173.
 - [58] Gubbi, J., Buyya, R., Marusic, S., and Palaniswami, M. (2013). Internet of things (iot): A vision, architectural elements, and future directions. *Future generation computer systems*, 29(7):1645–1660.
 - [59] Guo, X., Shen, Z., Zhang, Y., and Wu, T. (2019). Review on the application of artificial intelligence in smart homes. *Smart Cities*, 2(3):402–420.
 - [60] Guth, J., Breitenbücher, U., Falkenthal, M., Fremantle, P., Kopp, O., Leymann, F., and Reinfurt, L. (2018). A detailed analysis of iot platform architectures: concepts, similarities, and differences. In *Internet of everything*, pages 81–101. Springer.
 - [61] Hasan, M., Islam, M. M., Zarif, M. I. I., and Hashem, M. (2019). Attack and anomaly detection in iot sensors in iot sites using machine learning approaches. *Internet of Things*, 7:100059.
 - [62] He, W., Golla, M., Padhi, R., Ofek, J., Dürmuth, M., Fernandes, E., and Ur, B. (2018). Rethinking access control and authentication for the home internet of things (iot). In *27th {USENIX} Security Symposium ({USENIX} Security 18)*, pages 255–272.
 - [63] Hnat, T. W., Srinivasan, V., Lu, J., Sookoor, T. I., Dawson, R., Stankovic, J., and Whitehouse, K. (2011). The hitchhiker’s guide to successful residential sensing deployments. In *Proceedings of the 9th ACM Conference on Embedded Networked Sensor Systems*, pages 232–245.
 - [64] Indiegogo (2021). Olly - the first home robot with personality. <https://www.indiegogo.com/projects/olly-the-first-home-robot-with-personality/>. [Online; accessed 07-June-2021].
 - [65] Intille, S. S., Larson, K., Beaudin, J., Tapia, E. M., Kaushik, P., Nawyn, J., and McLeish, T. J. (2003). The placelab: A live-in laboratory for pervasive computing research (video).
 - [66] Iskhakov, A. (2019). Adaptive authentication technologies in behavioral analysis solutions of robotic systems. In *VIth International Workshop «Critical Infrastructures: Contingency Management, Intelligent, Agent-Based, Cloud Computing and Cyber Security»(IWCI 2019)*. Atlantis Press.

- [67] Jack, L. (2017). half baked IoT stove could be used as a remote controlled arson device. <https://hackaday.com/2017/04/20/half-baked-iot-stove-could-be-used-as-a-remote-controlled-arson-device/>. [Online; accessed 07-June-2021].
- [68] Jaikumar, V. (2014). Target attack shows danger of remotely accessible HVAC systems. <https://www.computerworld.com/article/2487452/target-attack-shows-danger-of-remotely-accessible-hvac-systems.html>. [Online; accessed 07-June-2021].
- [69] Jun, Z., Simplot-Ryl, D., Bisdikian, C., and Mouftah, H. (2011). The internet of things. *IEEE Commun. Mag*, 49(11):30–31.
- [70] Kanoje, S., Girase, S., and Mukhopadhyay, D. (2015). User profiling trends, techniques and applications. *arXiv preprint arXiv:1503.07474*.
- [71] Kavalakis, S. P. and Serrelis, E. (2014). Security issues of contemporary multimedia implementations: The case of sonos and sonosnet. In *The International Conference in Information Security and Digital Forensics*, pages 63–74.
- [72] Kayacik, H. G., Just, M., Baillie, L., Aspinall, D., and Micallef, N. (2014). Data driven authentication: On the effectiveness of user behaviour modelling with mobile device sensors. *arXiv preprint arXiv:1410.7743*.
- [73] Kefi-Fatteh, T., Ksantini, R., Kaâniche, M.-B., and Bouhoula, A. (2019). A novel incremental one-class support vector machine based on low variance direction. *Pattern Recognition*, 91:308–321.
- [74] Lashkari, A. H., Chen, M., and Ghorbani, A. A. (2019). A survey on user profiling model for anomaly detection in cyberspace. *Journal of Cyber Security and Mobility*, pages 75–112.
- [75] Latif, S., Zou, Z., Idrees, Z., and Ahmad, J. (2020). A novel attack detection scheme for the industrial internet of things using a lightweight random neural network. *IEEE Access*, 8:89337–89350.
- [76] Liang, Y., Samtani, S., Guo, B., and Yu, Z. (2020). Behavioral biometrics for continuous authentication in the internet-of-things era: An artificial intelligence perspective. *IEEE Internet of Things Journal*, 7(9):9128–9143.
- [77] Liao, Q., Li, H., Kang, S., and Liu, C. (2015). Application layer ddos attack detection using cluster with label based on sparse vector decomposition and rhythm matching. *Security and Communication Networks*, 8(17):3111–3120.
- [78] Ling, Z., Luo, J., Xu, Y., Gao, C., Wu, K., and Fu, X. (2017). Security vulnerabilities of internet of things: A case study of the smart plug system. *IEEE Internet of Things Journal*, 4(6):1899–1909.
- [79] Liu, H., Spink, T., and Patras, P. (2019). Uncovering security vulnerabilities in the belkin wemo home automation ecosystem. In *2019 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, pages 894–899. IEEE.

-
- [80] Lucero, S. et al. (2016). Iot platforms: enabling the internet of things. *White paper*.
- [81] Martin, V., Cao, Q., and Benson, T. (2017). Fending off iot-hunting attacks at home networks. In *Proceedings of the 2nd Workshop on Cloud-Assisted Networking*, pages 67–72.
- [82] Masmoudi, F., Maamar, Z., Sellami, M., Awad, A. I., and Burégio, V. (2020). A guiding framework for vetting the internet of things. *Journal of Information Security and Applications*, 55:102644.
- [83] Mathew, S., Petropoulos, M., Ngo, H. Q., and Upadhyaya, S. (2010). A data-centric approach to insider attack detection in database systems. In *International Workshop on Recent Advances in Intrusion Detection*, pages 382–401. Springer.
- [84] Mazzawi, H., Dalal, G., Rozenblatz, D., Ein-Dorx, L., Niniox, M., and Lavi, O. (2017). Anomaly detection in large databases using behavioral patterning. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, pages 1140–1149. IEEE.
- [85] Megan, W. (2016). Have a smart lock? Yeah, it can probably be hacked. <https://www.cnet.com/news/have-a-smart-lock-yeah-it-can-probably-be-hacked/>. [Online; accessed 07-June-2021].
- [86] Miessler, D. (2015). Securing the internet of things: Mapping attack surface areas using the owasp iot top 10. In *RSA Conference*.
- [87] Mondal, S. and Bours, P. (2018). A continuous combination of security & forensics for mobile devices. *Journal of information security and applications*, 40:63–77.
- [88] Moritz, K., Aultman, S. S., Campbell, J. J. A., Casillas, D., Neuse, J. E., Alonzo, S. T., Buckingham, T. B., Fernandez, G. C., and Mortensen, M. K. (2015). Behavioral profiling method and system to authenticate a user. US Patent 9,185,095.
- [89] Mozer, M. C. (1998). The neural network house: An environment hat adapts to its inhabitants.
- [90] Najafabadi, M. M., Khoshgoftaar, T. M., Calvert, C., and Kemp, C. (2017). User behavior anomaly detection for application layer ddos attacks. In *2017 IEEE International Conference on Information Reuse and Integration (IRI)*, pages 154–161. IEEE.
- [91] Nespoli, P., Zago, M., Huertas Celdrán, A., Gil Pérez, M., Gómez Mármol, F., and García Clemente, F. J. (2019). Palot: profiling and authenticating users leveraging internet of things. *Sensors*, 19(12):2832.
- [92] Nguyen, T. D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., and Sadeghi, A.-R. (2019). Dīot: A federated self-learning anomaly detection system for iot. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pages 756–767. IEEE.
- [93] Notra, S., Siddiqi, M., Gharakheili, H. H., Sivaraman, V., and Boreli, R. (2014). An experimental study of security and privacy risks with emerging household appliances. In *2014 IEEE conference on communications and network security*, pages 79–84. IEEE.

- [94] Omale, G. (2018). Gartner identifies top 10 strategic iot technologies and trends. *Gartner website*.
- [95] Ongun, T., Oprea, A., Nita-Rotaru, C., Christodorescu, M., and Salajegheh, N. (2018). The house that knows you: User authentication based on iot data. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 2255–2257.
- [96] Padraig, S. (2018). Top 10 iot applications in 2020. *IoT Analytics website*.
- [97] Pahl, M.-O. and Aubet, F.-X. (2018). All eyes on you: Distributed multi-dimensional iot microservice anomaly detection. In *2018 14th International Conference on Network and Service Management (CNSM)*, pages 72–80. IEEE.
- [98] Pan, Z., Pacheco, J., Hariri, S., Chen, Y., and Liu, B. (2019). Context aware anomaly behavior analysis for smart home systems. *International Journal of Information and Communication Engineering*, 13(5):261–274.
- [99] Powell, M. P., McCarthy, J. J., Tang, C., Stouffer, K. A., Zimmerman, T. A., Barker, W. C., Ogunyale, T., and Wynne, D. M. (2020). Securing manufacturing industrial control systems: Behavioral anomaly detection.
- [100] Prarthana, T. and Gangadhar, N. (2017). User behaviour anomaly detection in multidimensional data. In *2017 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)*, pages 3–10. IEEE.
- [101] Rahmati, A., Fernandes, E., Eykholt, K., and Prakash, A. (2018). Tyche: A risk-based permission model for smart homes. In *2018 IEEE Cybersecurity Development (SecDev)*, pages 29–36. IEEE.
- [102] Ramaswamy, S., Rastogi, R., and Shim, K. (2000). Efficient algorithms for mining outliers from large data sets. In *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pages 427–438.
- [103] Rasch, K. (2014). An unsupervised recommender system for smart homes. *Journal of Ambient Intelligence and Smart Environments*, 6(1):21–37.
- [104] Rath, A. T. and Colin, J.-N. (2017). Strengthening access control in case of compromised accounts in smart home. In *2017 IEEE 13th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)*, pages 1–8. IEEE.
- [105] Robert, T., Fabre, J.-C., and Roy, M. (2009). Application of early error detection for handling degraded modes of operation.
- [106] Ruan, X., Wu, Z., Wang, H., and Jajodia, S. (2015). Profiling online social behaviors for compromised account detection. *IEEE transactions on information forensics and security*, 11(1):176–187.
- [107] Samsung (2021). Smartthings, one simple home system. w world of possibilities. <https://www.smartthings.com/>. [Online; accessed 07-June-2021].

-
- [108] Sánchez, P. M. S., Valero, J. M. J., Celdrán, A. H., Bovet, G., Pérez, M. G., and Pérez, G. M. (2020). A survey on device behavior fingerprinting: Data sources, techniques, application scenarios, and datasets. *arXiv preprint arXiv:2008.03343*.
 - [109] Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., and Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural computation*, 13(7):1443–1471.
 - [110] Shah, S. W. and Kanhere, S. S. (2019). Recent trends in user authentication—a survey. *IEEE Access*, 7:112505–112519.
 - [111] Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *ICISSp*, pages 108–116.
 - [112] Sharma, B., Pokharel, P., and Joshi, B. (2020). User behavior analytics for anomaly detection using lstm autoencoder-insider threat detection. In *Proceedings of the 11th International Conference on Advances in Information Technology*, pages 1–9.
 - [113] Sikder, A. K., Babun, L., Aksu, H., and Uluagac, A. S. (2019). Aegis: a context-aware security framework for smart home systems. In *Proceedings of the 35th Annual Computer Security Applications Conference*, pages 28–41.
 - [114] Singh, I., Kumar, N., Srinivasa, K., Sharma, T., Kumar, V., and Singhal, S. (2020). Database intrusion detection using role and user behavior based risk assessment. *Journal of Information Security and Applications*, 55:102654.
 - [115] Sivanathan, A. (2020). Iot behavioral monitoring via network traffic analysis. *arXiv preprint arXiv:2001.10632*.
 - [116] Soltan, S., Mittal, P., and Poor, H. V. (2018). Blackiot: Iot botnet of high wattage devices can disrupt the power grid. In *27th {USENIX} Security Symposium ({USENIX} Security 18)*, pages 15–32.
 - [117] Tapia, E. M., Intille, S. S., and Larson, K. (2004). Activity recognition in the home using simple and ubiquitous sensors. <https://courses.media.mit.edu/2004fall/mas622j/04.projects/home/>. [Online; accessed 07-June-2021].
 - [118] Tavallaee, M., Bagheri, E., Lu, W., and Ghorbani, A. A. (2009). A detailed analysis of the kdd cup 99 data set. In *2009 IEEE symposium on computational intelligence for security and defense applications*, pages 1–6. IEEE.
 - [119] Tian, X.-g., Gao, L.-z., Sun, C.-l., Duan, M.-Y., and Zhang, E.-Y. (2006). A method for anomaly detection of user behaviors based on machine learning. *The Journal of China Universities of Posts and Telecommunications*, 13(2):61–78.
 - [120] Tian, Y., Zhang, N., Lin, Y.-H., Wang, X., Ur, B., Guo, X., and Tague, P. (2017). Smartauth: User-centered authorization for the internet of things. In *26th {USENIX} Security Symposium ({USENIX} Security 17)*, pages 361–378.
 - [121] University, W. S. (2007). Casas: Center for advanced studies in adaptive systems. <http://casas.wsu.edu/about>. [Online; accessed 07-June-2021].

- [122] van der Meulen, R. and Rivera, J. (2014). Gartner says a typical family home could contain more than 500 smart devices by 2022. Technical report, Gartner. [Online; accessed 07-June-2021].
- [123] van Kasteren, T. L., Englebienne, G., and Kröse, B. J. (2011). Human activity recognition from wireless sensor network data: Benchmark and software. In *Activity recognition in pervasive intelligent environments*, pages 165–186. Springer.
- [124] VIAROOM (2021). Welcome to your self-learning home. <http://viaroom.com/>. [Online; accessed 07-June-2021].
- [125] Viswanath, B., Bashir, M. A., Crovella, M., Guha, S., Gummadi, K. P., Krishnamurthy, B., and Mislove, A. (2014). Towards detecting anomalous user behavior in online social networks. In *23rd {USENIX} Security Symposium ({USENIX} Security 14)*, pages 223–238.
- [126] Wang, C. and Yang, B. (2018). Composite behavioral modeling for identity theft detection in online social networks. *arXiv preprint arXiv:1801.06825*.
- [127] Wazid, M., Das, A. K., Odelu, V., Kumar, N., Conti, M., and Jo, M. (2017). Design of secure user authenticated key management protocol for generic iot networks. *IEEE Internet of Things Journal*, 5(1):269–282.
- [128] Weiser, M. (1991). The computer for the 21 st century. *Scientific american*, 265(3):94–105.
- [129] Xiao, Y., Jia, Y., Liu, C., Alrawais, A., Rekik, M., and Shan, Z. (2020). Homeshield: A credential-less authentication framework for smart home systems. *IEEE Internet of Things Journal*, 7(9):7903–7918.
- [130] Yahyazadeh, M., Podder, P., Hoque, E., and Chowdhury, O. (2019). Expat: Expectation-based policy analysis and enforcement for appified smart-home platforms. In *Proceedings of the 24th ACM Symposium on Access Control Models and Technologies*, pages 61–72.
- [131] Yamanishi, K., Takeuchi, J.-I., Williams, G., and Milne, P. (2004). On-line unsupervised outlier detection using finite mixtures with discounting learning algorithms. *Data Mining and Knowledge Discovery*, 8(3):275–300.
- [132] Yamauchi, M., Ohsita, Y., Murata, M., Ueda, K., and Kato, Y. (2020). Anomaly detection in smart home operation from user behaviors and home conditions. *IEEE Transactions on Consumer Electronics*, 66(2):183–192.
- [133] Yang, Y. (2019). User behavior-based implicit authentication.
- [134] Yang, Y. and Sun, J. S. (2018). Dynamic multi-level privilege control in behavior-based implicit authentication systems leveraging mobile devices. *arXiv preprint arXiv:1808.00638*.
- [135] Yeung, D.-Y. and Chow, C. (2002). Parzen-window network intrusion detectors. In *Object recognition supported by user interaction for service robots*, volume 4, pages 385–388. IEEE.

-
- [136] Yu, T., Li, T., Sun, Y., Nanda, S., Smith, V., Sekar, V., and Seshan, S. (2020). Learning context-aware policies from multiple smart homes via federated multi-task learning. In *2020 IEEE/ACM Fifth International Conference on Internet-of-Things Design and Implementation (IoTDI)*, pages 104–115. IEEE.
 - [137] Zarpelão, B. B., Miani, R. S., Kawakani, C. T., and de Alvarenga, S. C. (2017). A survey of intrusion detection in internet of things. *Journal of Network and Computer Applications*, 84:25–37.
 - [138] Zhang, W., Meng, Y., Liu, Y., Zhang, X., Zhang, Y., and Zhu, H. (2018). Homonit: Monitoring smart home apps from encrypted traffic. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1074–1088.
 - [139] Zhang, Z., Guan, Y., Ma, X., Yu, T., and Zhang, L. (2020). Sovereign: User-controlled smart homes. *arXiv preprint arXiv:2006.06131*.
 - [140] Ziebart, B. D., Roth, D., Campbell, R. H., and Dey, A. K. (2005). Learning automation policies for pervasive computing environments. In *Second International Conference on Autonomic Computing (ICAC'05)*, pages 193–203. IEEE.
 - [141] Zimek, A., Schubert, E., and Kriegel, H.-P. (2012). A survey on unsupervised outlier detection in high-dimensional numerical data. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 5(5):363–387.

Abstract– Smart Home Systems (SHSs) represent one of the most prevailing Internet of Things (IoT) applications. However, the unauthorized or unexpected operation of IoT-based SHSs brings new security and safety concerns that did not exist in legacy homes. The research problem this thesis address is how a Behavioral Anomaly Detection (BAD)-based approach can be devised and integrated to support the design of secure and safe SHSs. To this end, we introduce Duenna – a BAD-based conceptual and methodological security framework. We apply Duenna to two use cases. In the first, Duenna is instantiated to make sure that only authorized users are allowed to operate SHS devices. The profiled targets in such a case are the SHS users which are continuously authenticated to prevent unauthorized user-issued operation commands. In the second, Duenna is instantiated to maintain a safe SHS appified automation. The profiled targets in such a case are the automation applications (called SmartApps) which are continuously monitored to prevent unexpected SmartApps-issued operation commands. Empirical evaluations conducted on real-world datasets for both use cases have successfully shown the feasibility of profiling the behavioral patterns of users and SmartApps to maintain a secure and safe SHS operation. The obtained results of this thesis partially contribute to the success of SHS technology.

Keywords: Internet of Things (IoT), Smart Home Systems (SHSs), Behavioral Anomaly Detection (BAD), Cybersecurity, User Authentication, Home Automation.

Résumé– Les Systèmes de Maisons Intelligentes (SMIs) représentent l'une des applications de l'Internet des Objets (IdO) les plus répandues. Cependant, le fonctionnement non autorisé ou inattendu de ces systèmes pose de nouveaux problèmes de sécurité et de sûreté qui n'existaient pas dans les maisons traditionnelles. Le problème de recherche abordé par cette thèse est de savoir comment une approche basée Détection d'Anomalies Comportementale (DAC) peut être conçue et intégrée pour soutenir la conception des SMIs sécurisés et sûrs. À cette fin, nous présentons Duenna - un cadre de sécurité conceptuel et méthodologique basé DAC. Nous appliquons Duenna à deux cas d'utilisation. Dans le premier, Duenna est instancié pour s'assurer que seuls les utilisateurs légitimes sont autorisés à faire fonctionner les appareils SMI. Les cibles profilées dans ce cas sont les utilisateurs du SMI qui sont constamment authentifiés pour empêcher l'émission de commandes utilisateurs non autorisées. Dans le second, Duenna est instancié pour maintenir une automatisation SMI sûre. Les cibles profilées dans ce cas sont les applications d'automatisation (appelées SmartApps) qui sont constamment évaluées pour éviter les commandes de fonctionnement inattendues émises par celles-ci. Des évaluations empiriques menées sur des ensembles de données du monde réel pour les deux cas d'utilisation ont montré avec succès la faisabilité du profilage des schémas comportementaux des utilisateurs et des applications d'automatisation pour maintenir un fonctionnement sécurisé et sûr d'un SMI. Les résultats obtenus dans cette thèse contribuent en partie au succès de la technologie de SMI.

Mots clés : Internet des Objets (IdO), Systèmes de Maisons Intelligentes (SMIs), Détection d'Anomalies Comportementale (DAC), Cybersécurité, Authentification d'Utilisateur, Domotique.